YTÜ EEF ÇDTP · 2026-27 GÜZ · KONTROL VE OTOMASYON

İki tekerlekli denge robotu yapım kitabı

Eğitimler, parça listesi, güç ve sinyal şemaları, ana kart, kablo listesi, şasi, şarj teması, firmware, testler, dijital ikiz doğrulaması ve robotun tüm kaynak kodu.

Sürüm: v1.2 Tarih: 29.10.2026 Depo: cdtp-denge-robotu

İçindekiler

1. Eğitim 1 · Geliştirme ortamı
2. Eğitim 2 · Montaj sırası
3. Eğitim 3 · İlk çalıştırma ve kalibrasyon
4. Eğitim 4 · Model ve kazançlar
5. Eğitim 5 · Telefon ve eldiven arayüzü
6. Eğitim 6 · Sorun giderme
7. Robot parça listesi
8. Blok şema
9. Güç şeması
10. Sinyal şeması: ESP32 pinleri
11. Ana kart yerleşimi (5×10 cm delikli plaket)
12. Kablo listesi
13. Pil ve BMS bağlama
14. Şasi ölçüleri
15. Şarj teması: V-yuva ve direk
16. Montaj ve tezgâh testleri
17. Firmware
18. Ölçülecek ve ayarlanacaklar
19. Testler: C++ simülasyonu ve MuJoCo dijital ikizi
20. Doğrulama: dijital ikiz, GitHub, tutarlılık
21. Ek: tüm kodlar

Eğitim 1 · Geliştirme ortamını kurma

1. Git, Python 3.13 ve VS Code kurun. VS Code'da **PlatformIO IDE** eklentisini yükleyin.
2. Depoyu bilgisayarınıza alın: `git clone <takım-reposu>` ya da proje klasörünü kopyalayın.
3. Python paketleri: `pip install numpy scipy matplotlib mujoco`.
4. Simülasyonu derlemek için bir C++ derleyicisi gerekir (MinGW g++ ya da Linux gcc). Git Bash'te: `bash tools/run_sim.sh`. Çıktının sonunda `OZET: 20/20` görünmeli.
5. Dijital ikiz: `python twin/twin.py`. Çıktı `twin/results/` klasörüne yazılır.
6. Firmware'i derleme: VS Code'da `firmware/robot` klasörünü açın, alttaki çubukta ✓ (Build). İlk derlemede ESP32 araçları iner (~1 GB); sonraki derlemeler saniyeler sürer.

Eğitim 2 · Montaj sırası

1. **Pleksi kesimi:** 3 kat 150 × 110 mm, köşe delikleri Ø3,2; V-yuva kanatları ve kutu yuvası (bölüm "Şasi ölçüleri").
2. **Motorlar:** L braketlerle kat 1'in altına; mil ekseni = teker ekseni. Tekerleri 6 mm D göbekle takın, vidaları sıkın (gevşek göbek denge döngüsünde boşluk yapar).
3. **Kat 1 üstü:** BTS7960 × 2 (sol/sağ), MPU6050 tam ortada ve teker ekseni hizasında (X ileri, bileşenler yukarı), LM2596, sigorta, WAGO × 2, C5 (1000 µF), SR5100.
4. **Kat 2:** pil yuvası ortada, BMS yanında, ana kart, XT60 arkada. IR gözleri ön kenarda, ±15° dışa, 15 mm tüplü.
5. **V-yuva:** kat 1 ile kat 2 arasına önde; pogo pimler zeminden 102 mm (+) ve 77 mm (−).
6. **Kat 3:** acil stop arkada, kutu yuvası ön ortada.
7. **Kablolama:** Kablo listesindeki sırayla (W1 → W44). Önce pil/BMS (W1–W6), sonra 12 V (W7–W19), şarj (W20–W23), en son sinyal (W24–W30).
8. **Tezgâh testleri:** "Montaj ve tezgâh testleri" bölümündeki 10 adım. Hepsi geçmeden motorlara EN vermeyin.

Eğitim 3 · İlk çalıştırma ve kalibrasyon

1. **Yükleme:** JP1'i çıkarın, USB'yi takın, Upload. Seri monitör 115200 baud.
2. **Açılış:** "Jiroskop kalibrasyonu" yazısı çıkarken robotu oynatmayın (2 s). `Robot MAC` satırını eldiven koduna yazın.
3. **IMU yönü:** robotu öne eğin: telemetride `tilt` artmalı. Sola çevirin: `yaw` artmalı. Değilse `imu_filter.h` içindeki `AxisMap` işaretlerini değiştirin.
4. **Enkoder:** tekeri elle 10 tur ileri: x ≈ 2,04 m. Değilse `ENC_COUNTS_PER_REV`; işaret tersse `ENC_*_INVERT`.
5. **Motor yönü:** tekerler havadayken `mtest`: önce sol, sonra sağ teker ileri dönmeli. Değilse `MOTOR_*_INVERT`.
6. **Model:** parçaları tartın, `tools/mass_budget.py`'yi güncelleyin, kütle merkezini ip ile asarak doğrulayın. `python tools/lqr_design.py` → `bash tools/run_sim.sh` → `python twin/twin.py`. Hepsi geçerse yükleyin.
7. **İlk denge:** kutusuz, iki el yakında. Robotu dik tutun; 0,5 s sonra kendisi tutar. Sürekli bir yöne kayıyorsa en rahat durduğu açıda tutup `cal`.
8. **Kutulu denge, ±10°, eldiven adımları, istasyon** sırasıyla. Her aşamadan sonra `stat` ile döngü süresi ve paket kaybına bakın.
9. **Final öncesi:** `main.cpp` içinde `FREEZE = true`, son kez yükleyin. Şartname gereği finalde parametre değiştirilmez.

Eğitim 4 · Model ve kazançları güncelleme

Kazançlar elle ayarlanmaz; robotun fiziksel modelinden hesaplanır.

1. `tools/mass_budget.py`: her parçanın gerçek kütlesi ve teker ekseninden yüksekliği.
2. `tools/lqr_design.py`: motor sabitleri (Ke = 12 V / boşta hız rad/s, R = 12 V / kilit akımı), iz genişliği. Çalıştırınca `gains.h` yeniden yazılır.
3. Ağırlıkları değiştirmek isterseniz: `python tools/lqr_design.py x=0.06 v=0.08` (izin verilen en büyük hata, Bryson kuralı).
4. Her değişiklikten sonra iki doğrulama: `bash tools/run_sim.sh` (20 test) ve `python twin/twin.py` (bağımsız MuJoCo modeli).
5. Belgeler ile kodun uyumu: `python tools/check_consistency.py`.

Eğitim 5 · Telefon (BLE) ve eldiven arayüzü

NE	DEĞER
BLE adı	`CDTP-Robot`
Servis	`6e400001-cd7b-4e3a-9a40-2026cd7b0001`
Telemetri (notify, 1 Hz, JSON)	`6e400002-cd7b-4e3a-9a40-2026cd7b0001`
Komut (write, metin)	`6e400003-cd7b-4e3a-9a40-2026cd7b0001`
Telemetri alanları	seq, t, bat, v, tilt, chg, mode, mot, dir, task, dock, x, yaw, glove, ir
Görev seçimi	`task=0` statik · 1 yönlendirme · 2 şarj · 3 bütünleşik
Eldiven paketi	ESP-NOW kanal 1, 12 bayt, 20 Hz, TEAM_KEY + CRC-8; 0,3 s gelmezse robot durur
Eldiven komutları (adım modu)	ileri/geri = 0,5 m, sola/sağa = 90°, düz el = dur

Eğitim 6 · Sorun giderme

BELİRTİ	OLASI NEDEN	NE YAPILIR
Robot dengeye girer girmez bir yöne fırlıyor	Jiroskop ya da motor yönü ters	Eğitim 3, adım 3 ve 5. Hızlı öne eğince jiroskop Y pozitif olmalı.
Robot titriyor, ses yapıyor	Gevşek göbek, jumper teması, ölü bölge büyük	Göbek vidaları, jumper'ları sıcak silikonla sabitleme, `MOTOR_DEADBAND_V` ölçümü
Durduktan sonra yavaşça kayıyor	Denge noktası ofseti	Birkaç saniye bekleyince kendisi öğrenir; kalıcı için `cal`
Eldiven komutu gecikiyor	ESP-NOW kaybı, kanal farkı	`stat`: kayıp sayısı. İki kart da kanal 1.
İstasyonu bulamıyor	38 kHz ayarı, göz açısı, LED gücü	İstasyon tezgâh testi, gözler ±15°, R8 22–150 Ω
Temas var ama şarj modu yok	GPIO34 bölücüsü, diyot yönü	J9-2 gerilimi ≥ 2 V olmalı; SR5100 çizgisi BMS tarafında
Sigorta atıyor	Kilit akımı yüksek	Tekeri tutup akım ölçün; 2,5 A üstü ise 7,5 A sigorta
MPU6050 bulunamadı	I2C kablosu, AD0	SDA 21, SCL 22, AD0 → GND, 3V3 besleme

01 Robot parça listesi

Fiyatlar takım sitesinden (9–10 Ekim 2026, KDV dahil). "Yeni" işaretliler bu dosyanın eklemeleri. Kütleler tahmin; parça gelince tartıp `tools/mass_budget.py`'ye yazın.

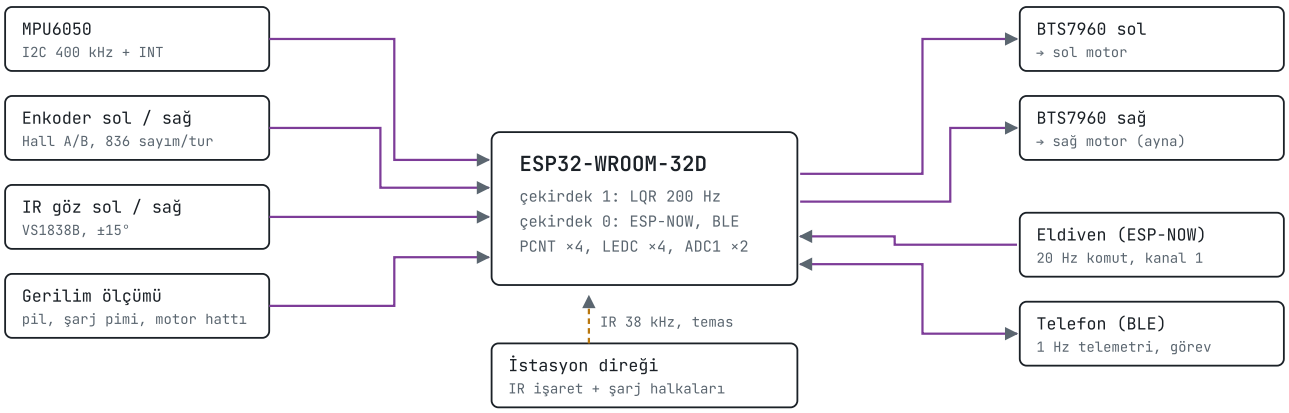
PARÇA	ADET	BİRİM TL	SATICI	NOT
BEYİN VE SENSÖRLER				
ESP32 WROOM-32D geliştirme kartı (38 pin)	1	455	Robotistan	WROVER değil: GPIO16/17 kullanılıyor
MPU6050 GY-521	1	109	Robotzade	Kat 1 üstü, teker ekseni hizasında, ortada
38 kHz IR alıcı kartı (VS1838B)	2	41	Robotistan	TSOP1838 almayın: 3,3 V'ta çalışmaz
HAREKET				
JGB37-520 12 V 530 RPM, Hall enkoderli	2	639	Direnc.net	6 mm D mil
BTS7960 motor sürücü kartı	2	162	Robotzade	VCC → 3,3 V
37 mm motor tutucu (L braket)	2	114	Keskinler	
65 mm lastik tekerlek	2	175	Robotistan	Göbek 6 mm D'ye uymalı
6 mm kaplin (2'li)	1	170	Robotistan	Göbek mile doğrudan oturuyorsa alınmaz
GÜÇ				
Orion 18650P 2450 mAh 12C	3	201	Direnc.net	Üçü aynı partiden
18650 3'lü pil yuvası	1	47	Ulutaş	İçten seri
3S 10 A BMS (HX-3S-01)	1	102	Motorobit	Şartname gereği zorunlu
LM2596 5 V 3 A regülatör	1	161	Robotistan	Ayarlıysa önce 5,0 V'a ayarlayın
Bıçak sigorta yuvası + 5 A (yedek 7,5 A)	1	40	Motorobit	Zorunlu
Acil stop Chint NP2-BE102, NC, 10 A	1	300	Motorobit	Zorunlu, kat 3'te
WAGO 221-415 (5'li)	2	30	Robotistan	WAGO-12V ve WAGO-GND
XT60 takımı	2	26	Direnc.net	
SR5100 Schottky diyot	3	6	Motorobit	Şarj girişi
1000 µF 25 V düşük ESR elektrolitik `yeni`	1	~20	Karaköy	Motor hattı (C5)
18 AWG silikon kablo kırmızı / siyah, 1 m	2+2	35	Motorobit	
ANA KART VE BAĞLANTI				
5×10 cm delikli plaket	1	16	Direnc.net	
Dişi + erkek pin header 1×40	1	45	Robotistan	ESP32 sokete takılır
Dirençler: 100k ×3, 27k ×2, 33k ×1 `33k yeni`, 10k ×1	–	~30	Robotistan	¼ W
Kondansatör: 100 µF 16 V ×1, 100 nF ×3	–	~15	Robotistan	
2'li 5,08 mm PCB klemens	3	8,72	Direnc.net	J1, J9, J10

PARÇA	ADET	BİRİM TL	SATICI	NOT
Dişi-dişi jumper 20 cm (40'lı)	1	59	Direnc.net	Yalnız sinyal
GÖVDE, YÜK, ŞARJ TEMASI				
3 mm pleksi (3 kat + V-yuva + kutu yuvası)	1	400	Trendyol	İstasyonla ortak levha
M3 30 mm + 50 mm dişi-dişi distans	8+8	9,5 / 13,5	Motorobit	Her köşede 30+50 = 80 mm
M3 10 mm vida (10'lu)	4	7,5	Motorobit	
5 mm pogo pin (10'lu) `robot tarafında`	1	87	Direnc.net	V-yuvada 2 adet, istasyonla paket ortak
Neodyum mıknatıs Ø10×3 mm N52 `yeni`	2	~40	Trendyol	V-yuvanın arka şeridine, pimlerin arasına; direkteki çelik şeride tutunur
Kapaklı ilaç kutusu	1	60	Eczane	

Robot tarafının toplamı yaklaşık **6.090 TL** (pleksi ve pogo paketi istasyonla paylaşılırsa ~5.740 TL).

02 Blok şema

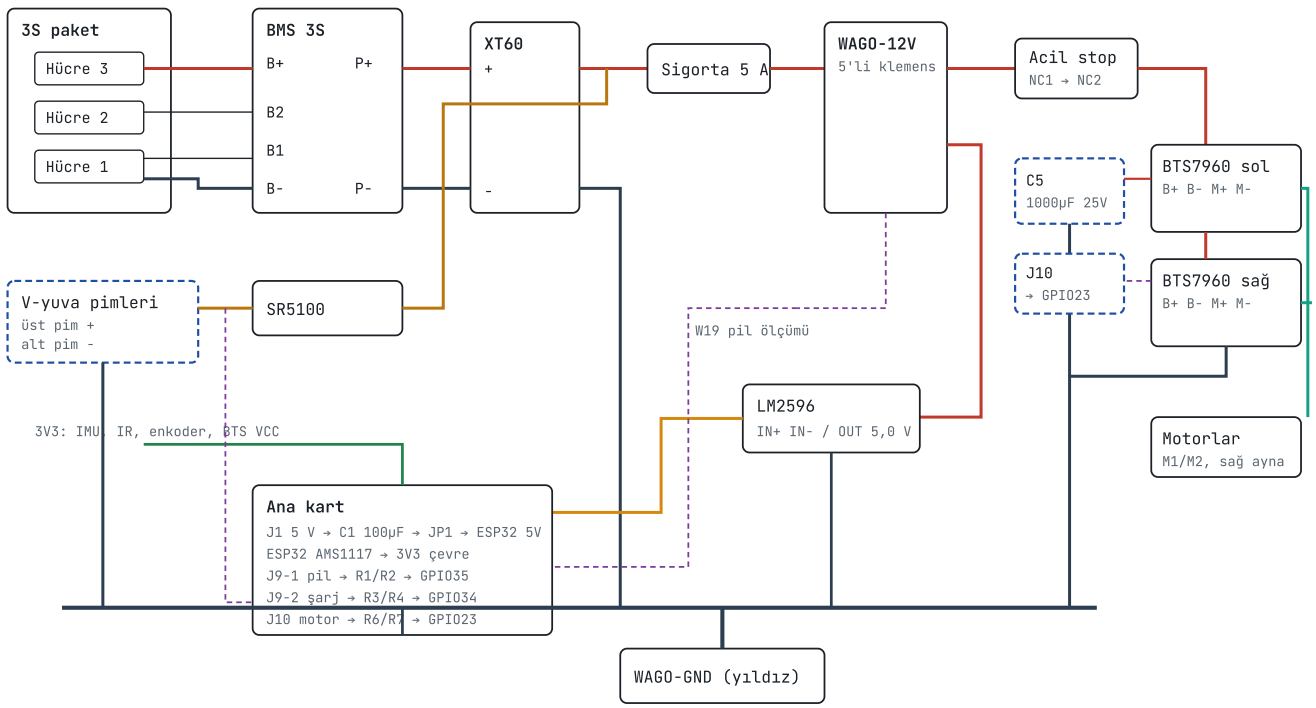
Tek beyin ESP32. Çekirdek 1 denge döngüsünü 200 Hz koşturur; çekirdek 0 radyoları (eldivenden ESP-NOW, telefona BLE) ve seri konsolu yönetir.



03 Güç şeması

Her kutuda gerçek uç adları var. Sigorta, regülatörü ve acil stopu birlikte korur. Acil stop yalnız motor hattını keser; ESP32 açık kalır. Şarj girişi sigortayı atlar, çünkü şarj kartı akımı zaten sınırlar.

12 V pil (18 AWG) · motor · 5 V · 3,3 V · GND · şarj · ölçüm



Gerilimler ve akımlar

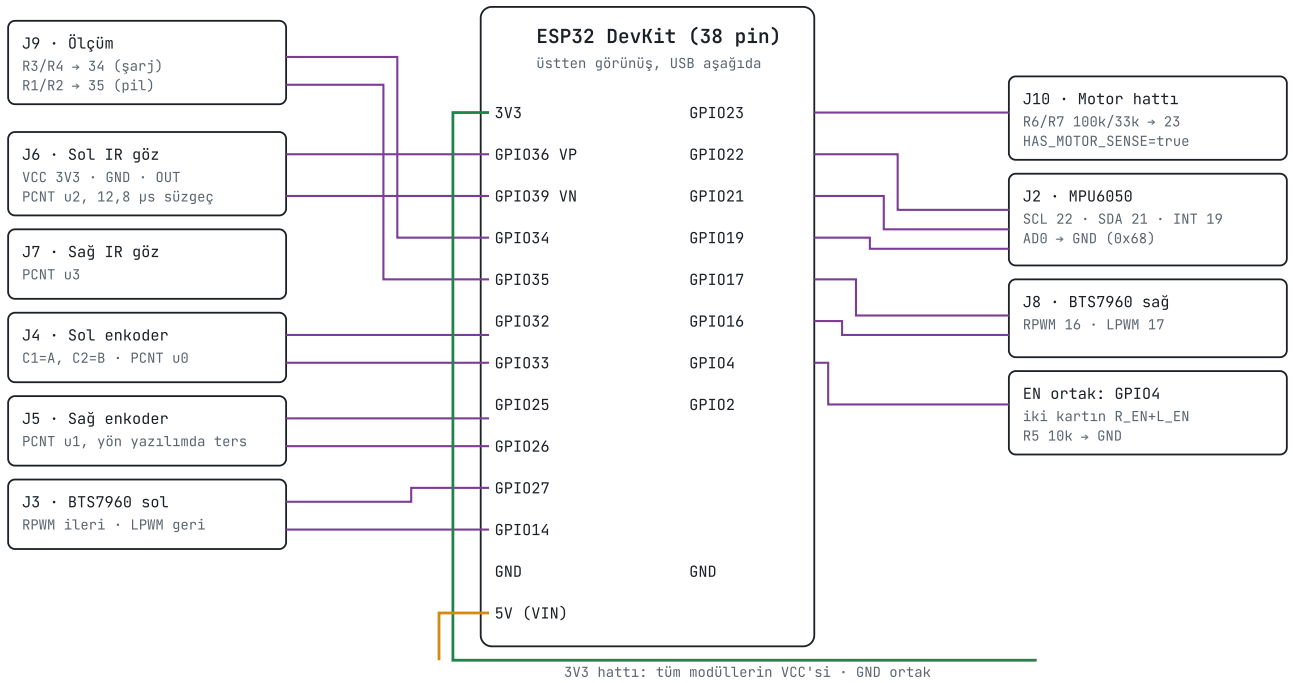
- Paket: 9,9 V boş, 11,1 V nominal, 12,6 V dolu. Şartname sınırı 24 V.
- Motor hattı tepe: iki motor kilitli ~5 A, normal dengede 0,3–1,5 A
- 5 V hattı: ~0,6 A tepe (radyo), LM2596'nın 3 A'inin beşte biri
- 3,3 V: ESP32 kartındaki AMS1117, ~0,31 A, 0,5 W ısı
- Şarj: ~1 A, diyot 0,4 V düşürür, paket ~12,2 V'ta (%90) durur

Bölücüler

- R1/R2 = 100k/27k: 12,6 V → 2,68 V (GPIO35, ADC1, 11 dB)
- R3/R4 = 100k/27k: şarj pimi → GPIO34. 9 V üstü = şarj var.
- R6/R7 = 100k/33k: acil stop sonrası 12,6 V → 3,13 V, 9,9 V → 2,46 V (GPIO23, dijital)
- Her ölçüm pinine GND'ye 100 nF (C2, C3, C4)

04 Sinyal şeması: ESP32 pinleri

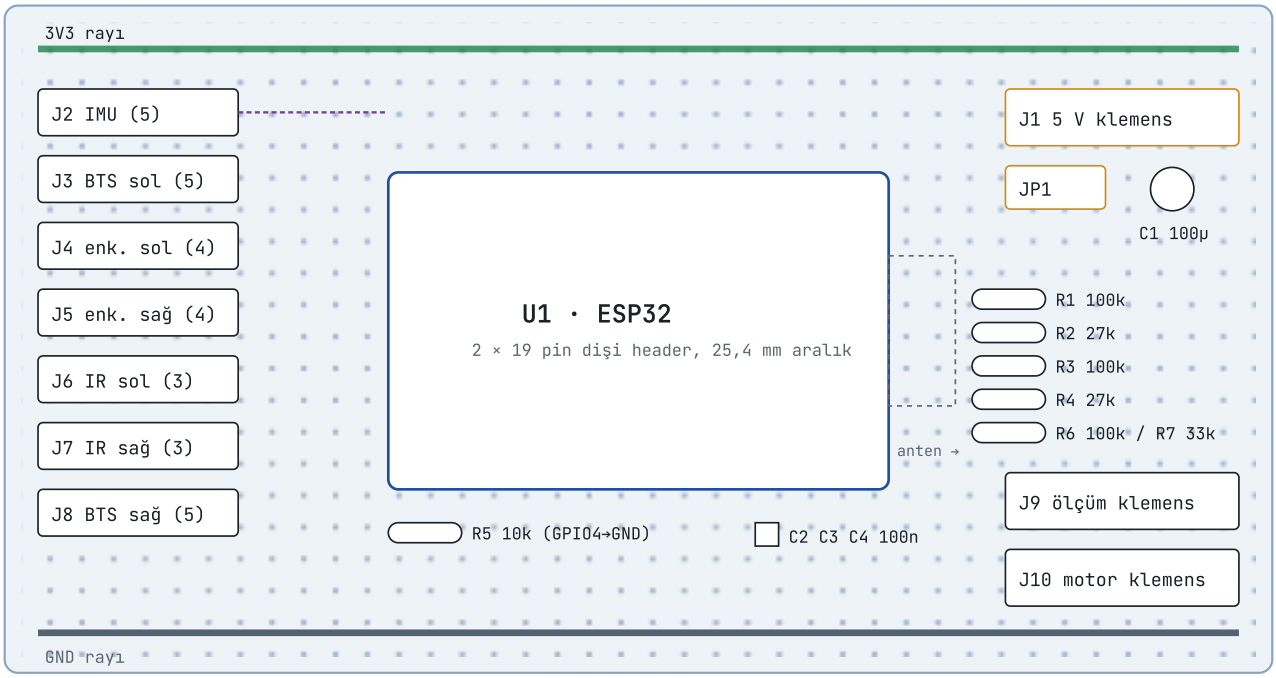
Kullanılmayan pinler: 0, 2, 5, 12, 15 açılış ayar pinleri (2 yalnız karttaki LED); 6–11 flash belleğe ait. 34–39 yalnız giriş, iç pull-up yok.



PİN	GÖREV	YAPILANDIRMA
19	MPU6050 INT	Yükselen kenar kesmesi, 200 Hz, denge döngüsünü tetikler
21 / 22	I2C SDA / SCL	400 kHz; GY-521 üstünde 4,7k pull-up var
27, 14 / 16, 17	Sol / sağ RPWM, LPWM	LEDC kanal 0–3, 20 kHz, 10 bit. GPIO14 açılışta kısa darbe verir; EN kapalıyken zararsız.
4	EN (iki kart)	Çıkış; 10k GND'ye: açılışta ve ESP32 sıfırlanırken motorlar kapalı
32, 33 / 25, 26	Enkoder A, B	PCNT dörtlü sayım (4x), 1,25 µs süzgeç, iç pull-up
36 / 39	IR göz sol / sağ	PCNT düşen kenar, 12,8 µs süzgeç (errata 3.11'deki 80 ns darbeleri sayılmaz)
35 / 34	Pil / şarj pimi gerilimi	ADC1, 11 dB, 50 Hz, `analogReadMilliVolts`
23	Acil stop sonrası motor hattı	Dijital giriş; 0 ise robot BEKLE moduna geçer
2	Karttaki LED	Dengede sabit, beklerken yanıp söner. Şarj göstergesi değil.

05 Ana kart yerleşimi (5×10 cm delikli plaket)

Üstten görünüş, 2,54 mm aralık, 20 × 38 delik. ESP32 iki sıra dişi header üstünde; arızada sökmeden değişir. Anten kart kenarından taşar, altından kablo geçirmeyin. Kalın çizgiler lehim tarafındaki kalaylı hatlar.



HEADER	PİN SIRASI (BEYAZ KARE = 1. PİN)	GİTTİĞİ YER
J1	+5V, GND	LM2596 OUT+ / OUT− (22 AWG)
JP1	2 pin + jumper	5 V'u ESP32'ye verir. **Kod yüklerken çıkarın** (USB ve LM2596 çakışmasın).
J2	3V3, GND, G22, G21, G19	MPU6050: VCC, GND, SCL, SDA, INT
J3	G27, G14, G4, 3V3, GND	BTS sol: RPWM, LPWM, R_EN+L_EN (köprülü), VCC, GND. IS uçları boş.
J8	G16, G17, G4, 3V3, GND	BTS sağ: aynı sıra
J4	3V3, GND, G32, G33	Sol enkoder: VCC, GND, C1 (A), C2 (B)
J5	3V3, GND, G25, G26	Sağ enkoder
J6 / J7	3V3, GND, G36 / G39	Sol / sağ IR göz: VCC, GND, OUT
J9	1: pil, 2: şarj pimi	1 → WAGO-12V (W19), 2 → üst şarj pimi (W23)
J10	1: motor hattı	Acil stop NC2 / BTS sol B+ (W43). GND J1'den.

06 Kablo listesi

Takım sitesinin numaraları korunur. W20–W23 şarj teması değiştiği için yeniden yazıldı, W41–W44 yeni. Her kablonun iki ucuna numarasını kâğıt bantla yazın. Güç hattında jumper kullanmayın; her lehime makaron.

NO	NEREDEN	NEREYE	TAŞIDIĞI	KABLO	BAĞLANTI
PİL VE BMS (BAĞLAMA SIRASI İÇİN BÖLÜM 07)					
W1	Hücre 1 −	BMS B−	Pil −	18 AWG siyah	Lehim
W2	Hücre 1+/2− köprüsü	BMS B1	3,7 V denge	22–26 AWG	Lehim (yuvanın köprüsüne)
W3	Hücre 2+/3− köprüsü	BMS B2	7,4 V denge	22–26 AWG	Lehim
W4	Hücre 3 +	BMS B+	12,6 V	18 AWG kırmızı	Lehim
W5	BMS B+ (= P+)	XT60 dişi +	Paket çıkışı	18 AWG kırmızı	Lehim + makaron
W6	BMS P−	XT60 dişi −	Paket çıkışı	18 AWG siyah	Lehim + makaron
12 V DAĞITIM					
W7	XT60 erkek +	Sigorta girişi	12,6 V	18 AWG kırmızı	Lehim
W8	Sigorta çıkışı	WAGO-12V	Sigortalı 12 V	18 AWG kırmızı	Klemens
W9	WAGO-12V	Acil stop NC1	12 V	18 AWG kırmızı	Vidalı
W10	Acil stop NC2	BTS sol B+	Motor beslemesi	18 AWG kırmızı	Vidalı
W11	BTS sol B+	BTS sağ B+	Motor beslemesi	18 AWG kırmızı	Zincirleme
W12	WAGO-12V	LM2596 IN+	12 V, stoptan önce	18 AWG kırmızı	Lehim
W13	XT60 erkek −	WAGO-GND	GND	18 AWG siyah	Klemens
W14	WAGO-GND	BTS sol B− ve sağ B−	GND	18 AWG siyah ×2	Vidalı
W15	WAGO-GND	LM2596 IN−	GND	18 AWG siyah	Lehim
W16	BTS sol M+ / M−	Sol motor M1 / M2	Motor	Motorun kablosu, bükülü	Vidalı
W17	BTS sağ M+ / M−	Sağ motor M1 / M2	Motor	Motorun kablosu, bükülü	Vidalı
W18	LM2596 OUT+ / OUT−	Ana kart J1	5 V	22 AWG	Klemens
W19	WAGO-12V	J9-1	Pil ölçümü	26 AWG	Klemens
ŞARJ TEMASI (DEĞİŞTİ)					
W20	Üst pogo pim (+)	SR5100 anot	Şarj +	18 AWG kırmızı	Lehim
W21	SR5100 katot (çizgili)	XT60 erkek + hattı (W7 başı)	Şarj +, sigortadan önce	18 AWG kırmızı	Lehim + makaron
W22	Alt pogo pim (−)	WAGO-GND	Şarj −	18 AWG siyah	Klemens
W23	Üst pogo pim (+)	J9-2	Şarj algılama	26 AWG	Klemens
SİNYAL (DİŞİ-DİŞİ JUMPER)					
W24	J2	MPU6050	I2C + INT	5 jumper	Header, AD0 → GND

NO	NEREDEN	NEREYE	TAŞIDIĞI	KABLO	BAĞLANTI
W25	J3	BTS sol	PWM + EN	5 jumper	Header
W26	J8	BTS sağ	PWM + EN	5 jumper	Header
W27	J4	Sol enkoder	Enkoder	4 jumper	Motor soketi
W28	J5	Sağ enkoder	Enkoder	4 jumper	Motor soketi
W29	J6	Sol IR göz	IR	3 jumper	Header
W30	J7	Sağ IR göz	IR	3 jumper	Header
YENİ					
W41	BTS sol B+	C5 + (1000 µF)	Motor hattı	Kısa, 18 AWG	Vidalı, kutup dikkat
W42	C5 −	BTS sol B−	GND	Kısa, 18 AWG	Vidalı
W43	BTS sol B+	J10-1	Motor hattı algılama	26 AWG	Klemens
W44	Ana kart GND rayı	WAGO-GND	Sinyal GND referansı	22 AWG	J1 üzerinden

> Motor kablolarını iki teli birbirine sararak götürün; enkoder ve I2C kablolarından en az 2 cm uzak tutun. Jumper uçlarını ilk denemeden sonra ESP32 tarafında sıcak silikonla sabitleyin; denge titreşimi gevşetir.

07 Pil ve BMS bağlama

Sıra

1. Hücreleri tek tek ölçün: 3,0–4,2 V, aralarında ≤ 0,05 V fark.
2. B− (W1), sonra B1 (W2), B2 (W3), en son B+ (W4). Sökerken tersi.
3. B1 ve B2'yi yuvanın hücre arası metal köprülerine lehimleyin, hücreye değil.
4. B−→B1, B1→B2, B2→B+ her biri 3,0–4,2 V olmalı.
5. P− ile B+ arası 0 V görünürse BMS uykudadır; bir kez şarj gerilimi verince uyanır.

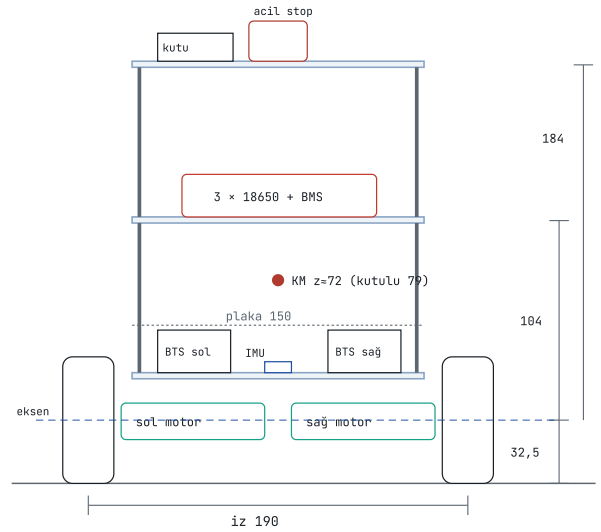
Kurallar

- Hiçbir yükü doğrudan B−'ye bağlamayın: BMS devre dışı kalır.
- Yük ve şarj aynı uçtan: B+ (=P+) ile P−.
- Bu BMS dengeleme yapmaz; ayda bir hücreleri ölçün.
- Pil yuvası kat 2'de ortada, cırt bantla değil M3 vidayla sabit.
- Robotu kullanmadığınızda XT60'ı çıkarın: ESP32 ve LM2596 sürekli ~0,3 A çeker.

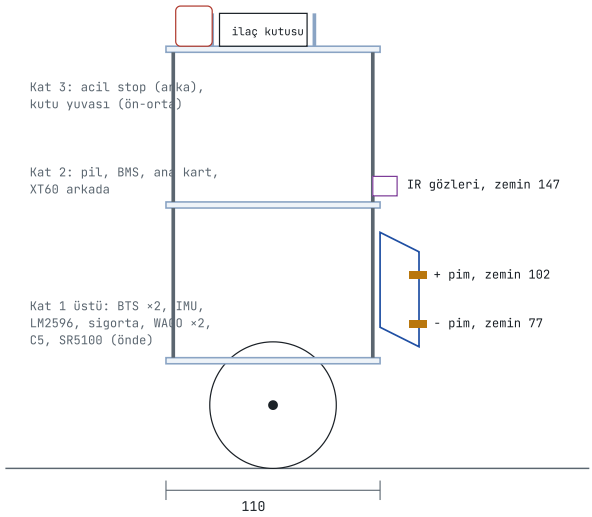
08 Şasi ölçüleri

Üç kat 3 mm pleksi, 150 × 110 mm, köşelerde 30+50 mm distans (katlar arası 80 mm). Ölçüler milimetre; z, teker ekseninden yukarı. Motorlar alt katın altına L braketle asılır, mil ekseni teker eksenidir.

Önden görünüş



Yandan görünüş (ön sağda)



Kesim listesi (3 mm pleksi)

- Kat 1, 2, 3: 150 × 110 mm, köşelerden 6 mm içeride Ø3,2 delik
- Kat 1: motor braketleri için 4 × Ø3,2; kablo için ortada 20 × 40 mm pencere
- V-yuva: 2 adet 40 × 80 mm kanat, 90° açıyla; arkada pimler için 20 × 80 mm dikey şerit, Ø5 delik 2 adet (25 mm aralık)
- Kutu yuvası: kutunun 2 mm büyüğü cep, 4 duvar 25 mm. Kutu vidalanmaz, cebe oturur.
- IR göz tutucu: iki gözü ±15° dışa bakacak açılı küçük braket, her göze 15 mm siyah büzüşmeli makaron tüp

Kütle bütçesi

- Motorlar + braketler 380 g (z 0)
- Kat 1 ve üstü 255 g (z 24–45)
- Kat 2 ve üstü 330 g (z 90–120)
- Kat 3 ve üstü 160 g (z 184–205)
- Distans, vida, kablo 120 g
- **Toplam 1,245 kg, KM 72 mm**; kutuyla 1,305 kg, 79 mm

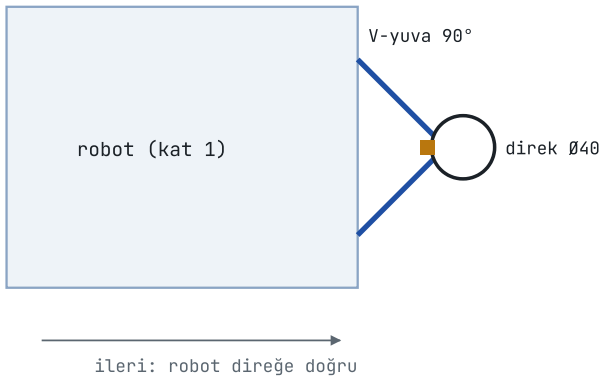
Sitedeki öneri 80–120 mm. Simülasyon 54–115 mm arasında geçti. Daha yavaş düşen (kolay dengelenen) robot isterseniz pili kat 3'e alın.

09 Şarj teması: V-yuva ve direk

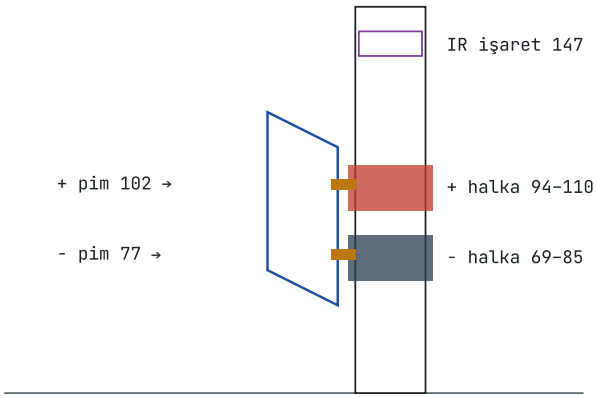
Robot önündeki V-yuvayla istasyonun yuvarlak direğine yaslanır. Direk yuvarlak olduğu için hangi açıdan gelirse gelsin temas aynıdır; V-yuva yanal ±3 cm hatayı kendisi ortalar. Pimler robotta, halkalar direkte. Pimler SR5100 diyotun arkasında olduğu için robotta açıkta pil gerilimi yoktur.

> Motorlar kesilince tekerler serbest kalır ve robot yuvarlak direkten geri yuvarlanır. Dijital ikizde eğim 11,6°'ye çıktı ve pim temastan ayrıldı. Bu yüzden V-yuvanın arka şeridinde iki neodyum mıknatıs, direkte halkaların arasında çelik şerit var. Şartname manyetik sabitlemeye izin veriyor. Mıknatıs ~10 N tutar; elle çekmek ve motorla geri gitmek (en fazla ~35 N) bunu rahat yener.

Üstten



Yandan (zeminden mm)



Robot tarafı

- Pogo pimler V-yuvanın arka şeridine geçirilir, arkasından lehimlenip sıcak silikonla sabitlenir; uçlar şeritten 3 mm dışarıda.
- Üst pim → SR5100 → XT60 + hattı (W20, W21) ve J9-2 (W23). Alt pim → WAGO-GND (W22).
- Robot dengede ±2° sallanır; pim yüksekliği ±0,5 mm değişir, halka 16 mm.
- Arka şerit 20 mm geniş olmalı: Ø40 direk önce şeride (pimlere) değer, kanatlar yalnız yanal hatayı düzeltir. Sivri V'de direk pimlere 8 mm kala kanatlara takılır (ikizde görüldü).
- İki Ø10×3 mm mıknatıs şeridin üstüne ve altına, pimlerle aynı hizada yapıştırılır.

Direk tarafı (Elektrik ekibine)

- Ø40 mm PVC boru, 200 mm, ağır taban (robot 6° yaslanır).
- Halkalar bakır banttan, aralarında 9 mm boşluk. Şarj kartı OUT+ → 1,1 A PTC → üst halka, OUT− → alt halka.
- IR LED'ler 147 mm'de, ±20° açılı, robot gözleriyle aynı yükseklikte.
- Robot bölgesine bakan yarım çevrede halkalar; arka yarım bant olmadan bırakılabilir.
- Halkaların arasına (85–94 mm) ince galvaniz çelik şerit sarılır; robotun mıknatısı her açıdan buna tutunur.

10 Montaj ve tezgâh testleri

1. **Pil:** bölüm 07'deki sıra. B+ ile P− arası paket gerilimi.
2. **Regülatör yüksüz:** LM2596'yı sigortadan besleyin, çıkışı 5,0–5,1 V'a ayarlayın. ESP32 takılı değil.
3. **Ana kart:** ESP32 takılı değilken J1'e 5 V verin, soketteki 5V ve 3V3 noktalarını ölçün (3V3 boş: ESP32 regülatörü takılınca gelir). GPIO4 ile GND arası 10k.
4. **ESP32 tak, JP1 çıkık, USB ile yükle** (bölüm 11). Seri monitörde MPU6050 ve `Robot MAC` görünmeli.
5. **Ölçümler:** `stat` ve telemetride v değerini multimetreyle karşılaştırın; fark %3'ten büyükse `vscale=`.
6. **Motor hattı:** EN kablosu takılmadan acil stopa basıp bırakın; J10 gerilimi 0 ↔ ~3,1 V olmalı.
7. **Motorlar havada:** robotu tekerler boşta kalacak şekilde destekleyin, EN'i takın, `mtest`: önce sol, sonra sağ teker 1 s ileri.
8. **Kilit akımı:** tekeri elle 1 s tutun, sürücü girişinde akım 2,5 A'i geçmiyorsa 5 A sigorta yeter.
9. **Enkoder:** tekeri elle 10 tur ileri; x ≈ 2,04 m.
10. **IMU:** robotu öne eğin `tilt` artar; sola çevirin `yaw` artar.

11 Firmware

Yükleme

1. VS Code + PlatformIO eklentisi.
2. `firmware/robot` klasörünü açın. `platformio.ini` ESP32 platform 6.9.0 (Arduino 2.0.17) ve NimBLE 1.4.3'ü kendisi indirir.
3. JP1'i çıkarın, USB'yi takın, Upload.
4. Seri monitör 115200. Açılışta robot kıpırdamasın: jiroskop 2 s kalibre olur.

Derleme: flash %80, RAM %17, uyarı yok.

Çalışma

- MPU6050 her 5 ms'de INT verir → kesme → çekirdek 1'deki görev uyanır.
- IMU, enkoder (PCNT), IR sayaçları, ADC (50 Hz), eldiven paketi okunur.
- `RobotLogic::step()` çıkışı: EN ve iki PWM oranı.
- Her 200 adımda bir telemetri JSON'u üretilir; çekirdek 0 BLE ile yollar.
- IMU 3 kez üst üste okunamazsa motorlar kapanır.
- Eldivenden 0,3 s paket gelmezse süren adım yumuşakça kesilir, robot durur.
- Enkoder tekerin gövdeye göre dönüşünü sayar; yere göre yol için $r \cdot \theta$ eklenir.
- Robot yerinde dururken denge açısını yavaşça öğrenir (en fazla ±3°). Kütle merkezi tam eksen üstünde değilse durduktan sonraki kaymayı bu giderir.

MOD	NE YAPAR	GEÇİŞ
BEKLE	Motorlar kapalı	Elle ±3° içinde 0,5 s tutulunca DENGE. Şarj gerilimi varsa ŞARJ.
DENGE	LQR denge; eldiven adımları	Görev uygunsa ve işaret görülürse YANAŞMA. $\|\theta\| > 40°$ → DÜŞTÜ. Acil stop → BEKLE.
YANAŞMA	ARA → HİZALAN → YAKLAŞ → YERLEŞ	Pimde 9 V'tan fazla 0,1 s → 4° öne eğilip motorları keser → ŞARJ. 60 s'de bitmezse DENGE.
ŞARJ	Motorlar kapalı, mıknatısla direğe yaslı	Eldivenle "geri" → DENGE + 0,5 m geri; sonraki 10 s yeniden yanaşmaz. Şarj 2 s kesilirse BEKLE.
DÜŞTÜ	Motorlar kapalı	$\|\theta\| < 20°$ olunca BEKLE.

SERİ / BLE KOMUTU	ETKİ
`stat`	Döngü süresi, IMU hata, eldiven paket/kayıp sayısı
`mtest`	BEKLE'deyken sol, sonra sağ teker 1 s %15 ileri
`task=0..3`	Statik / yönlendirme / şarj / bütünleşik (NVS'e kaydedilir)
`glove=step | cont`	Adım modu (0,5 m / 90°) ya da sürekli sürüş
`cal`	Şu anki eğimi denge noktası kabul et (NVS)
`vscale=1.000`	Pil ölçüm katsayısı

12 Ölçülecek ve ayarlanacaklar

PARAMETRE	DOSYA	ŞU AN	NASIL BULUNUR
Gövde kütlesi, KM, atalet	`tools/mass_budget.py`	`1,245 kg, 72 mm`	Parçaları tartın; KM için robotu ipe iki noktadan asın
Enkoder sayım/tur	`cdtp_config.h`	`836`	Tekeri 10 tur çevirip sayımı okuyun
İz genişliği	`cdtp_config.h + lqr_design.py`	`0,190 m`	Teker temas noktaları arası; yerinde 10 tur dönüp enkoder farkından
Motor yönleri, enkoder yönleri	`cdtp_config.h`	`sağ ters`	`mtest` ve elle çevirme
IMU eksenleri	`imu_filter.h AxisMap`	`X ileri, Z yukarı, jiroskop Y +`	MPU6050'yi X ileri, bileşenler yukarı bakacak yapıştırın. Öne eğince `tilt` artmalı; hızlı öne eğerken jiroskop Y pozitif okunur. Farklı montajda `AxisMap` işaretlerini değiştirin.
Olu bölge gerilimi	`cdtp_config.h`	`0,45 V`	Teker havada; PWM'i yavaş artırıp dönmeye başladığı gerilim
Denge noktası ofseti	`NVS (cal)`	`0`	Robotu en rahat durduğu açıda tutup `cal`
Motor Ke, R	`lqr_design.py`	`0,216, 4,5 Ω`	Ke = 12 V / boşta hız (rad/s); R = 12 V / kilit akımı

> Bir değer değişince: `python tools/lqr_design.py` → `bash tools/run_sim.sh` (hepsi geçmeli) → firmware'i yükleyin. Final öncesi `main.cpp`'de `FREEZE = true`.

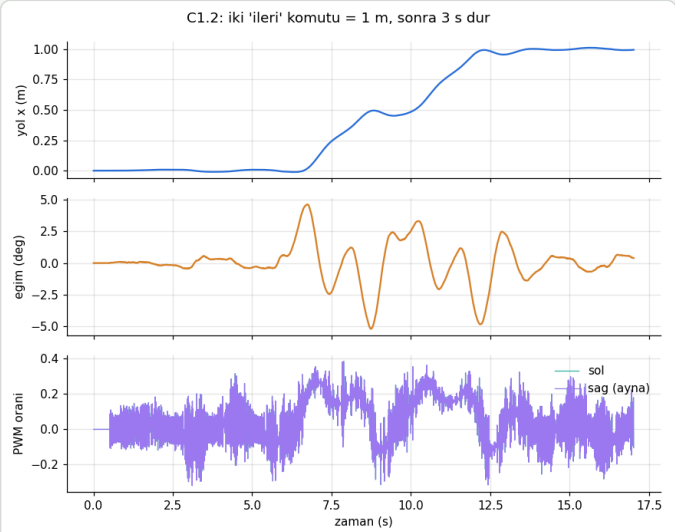
13 Testler: C++ simülasyonu ve MuJoCo dijital ikizi

TEST	C++ SİMÜLASYONU	SONUÇ
ON Enerjisiz +-10 deg'den dik kalmamali	40 deg'yi gecme suresi en fazla 0.30 s (sinir: 5 s icinde dusmeli)	geçti
C1.1 30 s statik denge (kutulu)	en buyuk egim 0.73 deg, en buyuk kayma 1.5 cm	geçti
C1.2 1 m ileri + 3 s dur	yol 0.994 m, 3 s durmada en buyuk kayma 10.2 mm (hiz tepe 0.052 m/s), en buyuk egim 5.2 deg	geçti
C1.3 1 m geri + 3 s dur	yol 1.004 m, 3 s durmada en buyuk kayma 9.0 mm (hiz tepe 0.052 m/s), en buyuk egim 5.0 deg	geçti
C1.4 +10 ve -10 deg'den 10 s icinde toparlanma	+10: 1.44 s (yol 14 cm), -10: 1.46 s (yol 14 cm)	geçti
GUV.2 Acil stop: denge birakilir, motorlar kapanir	stop sonrasi mod BEKLE, bekleme dogru	geçti
C2.1 Eldiven: ileri ~0,5 m	tepki 0.75 s, yol 0.497 m	geçti
C2.2 Eldiven: geri ~0,5 m	tepki 0.77 s, yol -0.498 m	geçti
C2.3 Eldiven: sola ~90 deg	tepki 0.29 s, donus 90.0 deg	geçti
C2.4 Eldiven: saga ~90 deg	tepki 0.29 s, donus -90.0 deg	geçti
C2.5 Dur: tum komutlardan sonra yerinde kalir	son 3 s kayma 0.5 mm, donme 0.46 deg	geçti
GUV.1 Eldiven sustu (0,3 s): adim kesilir, robot durur	durdugu yol 0.309 m (tam adim 0,5 m)	geçti
C3.1-2 IR ile bul, hizalan, yerles (63 baslangic, 30 cm/90 deg)	63/63 basarili, en uzun 14.8 s	geçti
C3.3 Temastan sonra sarj + motor durur <= 5 s	temas -> SARJ modu en fazla 0.50 s	geçti
C3.x Istasyon gorus disindayken arayip bulma	3/3 basarili (90, -90, 180 deg ters bakis)	geçti
C5.1 Butunlesik gorev, tekrar 1	hedefte bekleme 5.6 s, erken yanasma yok, sarj basladi, temas->sarj 0.50 s	geçti
C5.2 Butunlesik gorev, tekrar 2	hedefte bekleme 5.6 s, erken yanasma yok, sarj basladi, temas->sarj 0.50 s	geçti
C5.3 Butunlesik gorev, tekrar 3	hedefte bekleme 5.6 s, erken yanasma yok, sarj basladi, temas->sarj 0.50 s	geçti
C5.4 Butunlesik gorev, tekrar 4	hedefte bekleme 5.6 s, erken yanasma yok, sarj basladi, temas->sarj 0.50 s	geçti
SAG Belirsizlik: kutle +-25%, KM -25/+60%, ...	statik 40/40, toparlanma 40/40 (en uzun 1.7 s), 1 m: +-5 cm durmada 40/40, +-3 cm durmada 35/40	geçti

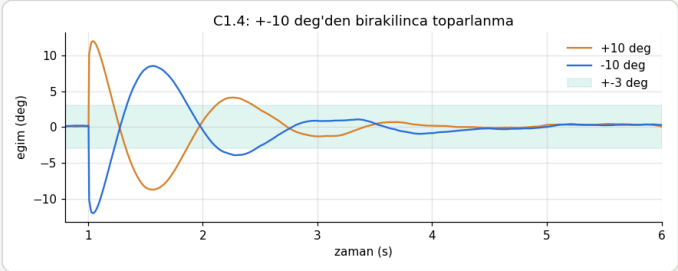
MuJoCo dijital ikizi

TEST	DİJİTAL İKİZ	SONUÇ
ON Enerjisiz +-10 deg'den dik kalmamali	40 deg'ye en gec 0.31 s'de dustu	geçti
C1.1 30 s statik denge (kutulu)	en buyuk egim 1.77 deg, kayma 4.0 cm, kutu yerinde	geçti
C1.2 1 m ileri + 3 s dur	yol 0.990 m, 3 s durmada kayma 6.9 mm, egim tepe 5.8 deg, kutu yerinde	geçti
C1.3 1 m geri + 3 s dur	yol 1.008 m, 3 s durmada kayma 13.9 mm, egim tepe 6.0 deg, kutu yerinde	geçti
C1.4 +10 ve -10 deg'den 10 s icinde toparlanma	+10: 1.29 s, -10: 1.57 s	geçti
C2.3-4 Eldiven: sola / saga ~90 deg	sol +89.8 deg, sag -89.8 deg	geçti
GUV.2 Acil stop: denge birakilir, motorlar kapanir	mod=BEKLE	geçti

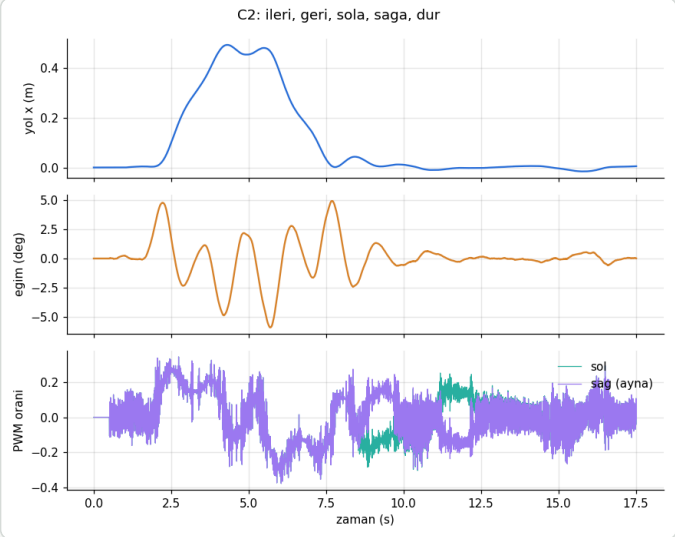
TEST	DİJİTAL İKİZ	SONUÇ
REF GitHub referans LQI kazanclari ikizde dengeliyor	baslangic 0 deg: dengede (son -0.9 deg), baslangic 8 deg: dengede (son -1.1 deg)	geçti
C3 IR ile yanasma, V-yuva + direk (30 baslangic)	30/30 basarili, en uzun temas 14.0 s, temas->sarj modu 0.50 s	geçti
C3.5 Eldivenle 'geri': miknatistan kopup istasyondan ayrilir	once SARJ, sonra mod DENGE, direge uzaklik 50 cm	geçti
C5 Butunlesik gorev (kutu, eldiven, yanasma)	son mod SARJ, erken yanasma yok, kutu yerinde	geçti



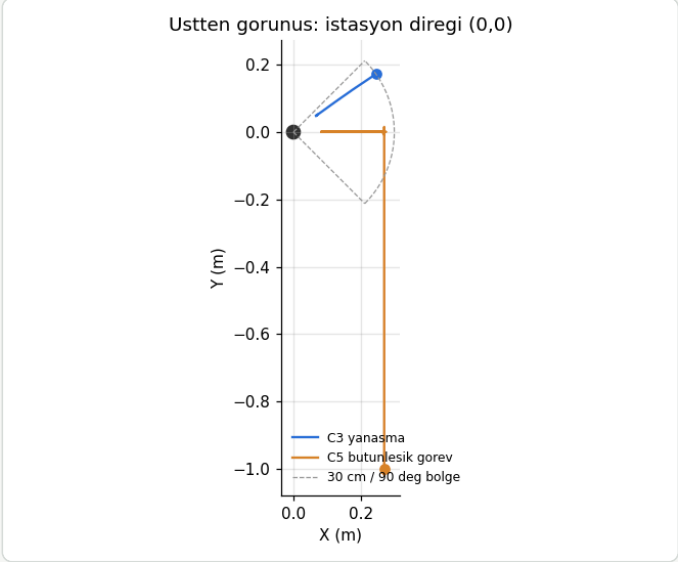
C1.2: iki "ileri" adımı ve 3 s durma



C1.4: ±10°'den toparlanma



C2: beş komut



C3 ve C5 yolları, üstten

> Simülasyon gerçek robotun yerini tutmaz. Gerçek robotta ilk denge denemesini kutusuz, iki el yakında yapın.

Doğrulama

Yazılan ve çizilen her şey üç bağımsız yolla kontrol edildi: GitHub'daki çalışan robotlarla karşılaştırma, belge ile kodun otomatik tutarlılık denetimi ve MuJoCo fizik motoruyla kurulan dijital ikiz.

Dijital ikizde bulunan ve düzeltilen hatalar

HATA	ETKİ	DÜZELTME
Jiroskop işareti ters. Kod, X ileri / Z yukarı monte edilmiş MPU6050'de öne düşüşü jiroskop Y'de negatif varsayıyordu. Gerçekte pozitif okunur (sağ el kuralı, ikizde ölçüldü). İlk C++ simülasyonu aynı yanlış varsayımla yazıldığı için fark etmedi.	Gerçek robot ilk anda düşerdi: tümleyen filtre jiroskopla ivmeölçeri ters yönde birleştirir.	`AxisMap.pitchSign = +1`; C++ simülasyonu da fiziksel kurala çevrildi.
Enkoder bağıl açı ölçer. Enkoder tekerin gövdeye göre dönüşünü sayar; kod bunu yere göre yol sanıyordu.	Eğim değişirken hız ölçümünde $r\cdot\dot{\theta}$ kadar hata (~0,03 m/s), K_V ile ~2 V yanlış komut.	`x = r·(φ_bağıl) + r·θ`; simülasyonun enkoder modeli de bağıl açıya çevrildi.
Durduktan sonra kayma. IR gözleri ve V-yuva öne 1,4 mm kütle kaydırıyor; robot −1,1° eğik dengede durur, LQR bunu 3,6 cm konum hatasıyla karşılar.	1 m testinde durmada 5 cm'ye yakın kayma.	Robot dururken denge açısını yavaşça öğrenir (±3° sınırlı). LQI (konum integrali) de denendi; yanaşma ve dönüşü bozduğu için alınmadı.
Şarjda geri yuvarlanma. Motorlar kesilince tekerler serbest kalıyor; robot yuvarlak direkten geri yuvarlandı, eğim 11,6°'ye çıktı, pim temastan ayrıldı.	Şarj göstergesi söner, robot 2 s sonra BEKLE'ye düşer; C3 ve C5 başarısız olur.	V-yuvaya iki neodyum mıknatıs, direğe çelik şerit (şartname manyetik sabitlemeye izin veriyor). İkizde robot 0,2° eğimle temaslı kaldı.
Sivri V-yuva. İkizde ilk kurulan sivri V'de Ø40 direk pimlere 8 mm kala kanatlara takıldı.	Hiç temas yok.	Belgedeki 20 mm düz arka şerit zorunlu hale getirildi ve açıklandı.
Ayrıldıktan sonra yeniden yanaşma. Şarj görevinde eldivenle "geri" deyince robot 1 s sonra kendiliğinden yeniden yanaşıyordu.	Kullanıcı robotu istasyondan alamaz.	Ayrıldıktan sonra 10 s yanaşma beklemesi (`UNDOCK_HOLD_S`).
Belgede eksik bilgi. Robot yapım dosyasında eldiven zaman aşımı yazmıyordu.	Belge ile kod arasında boşluk.	Eklendi; tutarlılık denetimi 41/41.

GitHub karşılaştırması

GitHub'da en çok yıldızlı ve bize en yakın 6 denge robotu indirildi. Kontrol yöntemi, döngü hızı, filtre, düşme sınırı ve kazançlar karşılaştırıldı. En yakın örnek (suelioalencar, ESP32, 65 mm teker, enkoderli DC motor, gerilim cinsinden LQI) gerçek robotta çalışan kazançlar kullanıyor. Bu kazançlar bizim dijital ikizimizde de robotu dengede tuttu; yani ikiz gerçekçi bir bölgede. Kazançlarımız aynı işaret yapısında ve aynı büyüklükte.

REPO	DONANIM	KONTROL	DÖNGÜ	AÇI KESTİRİMİ	DÜŞME SINIRI	KAZANÇ / NOT	BİZİMLE FARK
Bu proje (cdtp-denge-robotu)	ESP32, MPU6050, JGB37-520 enkoderli DC, BTS7960, 65 mm teker	LQR (gerilim), 4 durum + yön PD, referans süzgeci, denge açısı öğrenme	200 Hz, MPU INT ile	Tümleyen 0,98, ivme güveni 1 g'den sapınca düşer	40°	K = [52,4 64,7 107,2 13,2] V; modelden, simülasyon + ikizle doğrulandı	—

REPO	DONANIM	KONTROL	DÖNGÜ	AÇI KESTİRİMİ	DÜŞME SINIRI	KAZANÇ / NOT	BİZİMLE FARK
suelioalencar/Two-Wheels-Inverted-Pendulum-with-LQI (★4)	ESP32 (WEMOS D32), MPU6050, enkoderli DC (1:56), TB6612, 65 mm teker	LQI (gerilim × PWM katsayısı), tekerlere ayrı yön LQR'ı	125 Hz (8 ms), vTaskDelayUntil	MPU6050_tockn kütüphanesi	30°	k1 = [100,5 69,9 83,6 9,31] V + integral 0,107. Bizimkine en yakın; aynı işaret yapısı ve büyüklük. İkizimizde de dengeliyor.	Enkoder kesmeyle (CHANGE); biz PCNT donanım sayacıyla. Bizde integral yok, yerine denge açısı öğrenme.
dungvd0309/lqr-self-balancing-robot (★1)	ESP32, MPU6050, enkoderli DC, L298 tipi sürücü	LQR, 6 durum (ileri, eğim, yön), PWM'e map	~166 Hz (6 ms, loop içinde micros)	Kalman süzgeci	30°	K = [12 11 300 14] PWM biriminde, elle ayarlı. MIN_PWM 140/255 ölü bölge telafisi.	Kazanç elle; biz modelden. Ölü bölgeyi biz gerilimde (0,45 V) telafi ediyoruz.
trandi/esp32-self_balancing_robot (★88)	ESP32, MPU6050, step motor (A4988)	PID (Kp 100, Ki 5, Kd 130) + otomatik denge açısı	250 Hz (4 ms)	Tümleyen 0,999, DLPF 43 Hz	—	Denge açısını robot hareket ederken ayarlıyor (YABR yöntemi).	Step motorda enkoder gerekmez. Bizim denge açısı öğrenmemiz aynı fikrin konum hatasıyla yapılan hali.
Rokasbarasa1/ESP32-self-balancing-robot (★38)	ESP32-WROOM-32D, optik enkoder, DC motor	Kademeli PID (eğim, yön, hız)	400 Hz	Tümleyen, oran döngü hızına bağlı	—	pitch P 15, I 350, D 0,31; hız döngüleri kapalı (0).	PID ile konum tutmuyor; biz konum + hız durumlarıyla tutuyoruz.
simplefoc/Arduino-FOC-balancer (★277)	BLDC gimbal motor, FOC sürücü, MPU6050 DMP	Kademeli: hız PI → eğim hedefi → eğim PID → gerilim	FOC döngüsü	MPU6050 DMP	—	Eğim PID P 30, I 100, D 1; hız PI çıkışı ±π/10 ile sınırlı.	Bizim referans süzgecine benzer alçak geçirgen süzgeçler (0,07–0,5 s) kullanıyor.
bluino/esp32_wifi_balancing_robot (★56)	ESP32, MPU6050, step motor	PD eğim + PI hız (jjrobots B-ROBOT türevi)	~200 Hz	Tümleyen	—	KP 0,32, KD 0,05; MAX_TARGET_ANGLE 14°.	Hedef eğimi ±14° ile sınırlıyor; bizde ivme ön-beslemesi ve LQR doğal olarak sınırlıyor.

Belge ↔ kod tutarlılık denetimi

`tools/check_consistency.py`: firmware pinleri, takım sitesindeki pin şeması, yapım dosyasındaki pin tablosu ve header sıraları, ayar sabitleri, LQR kazançları,

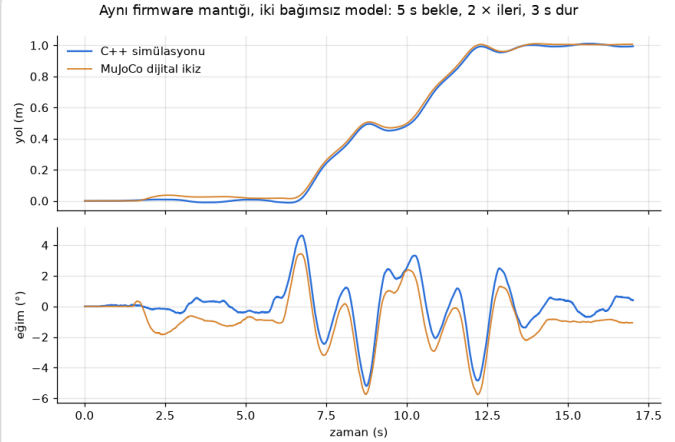
BLE UUID'leri ve bölücü hesapları karşılaştırıldı.

KONU	BEKLENEN	BULUNAN	DURUM
Site semasi: SDA	[21]	[21]	geçti
Site semasi: SCL	[22]	[22]	geçti
Site semasi: INT	[19]	[19]	geçti
Site semasi: OUT	[36, 39]	[36, 39]	geçti
Site semasi: RPWM	[27, 16]	[27, 16]	geçti
Site semasi: LPWM	[14, 17]	[14, 17]	geçti
Site semasi: R_EN + L_EN	[4, 4]	[4, 4]	geçti
Site semasi: C1 (A)	[32, 25]	[32, 25]	geçti
Site semasi: C2 (B)	[33, 26]	[33, 26]	geçti
Site semasi: pil olcumu	35	35	geçti
Site semasi: sarj algilama	34	34	geçti
Belge pin tablosu: MPU6050 INT	[19]	[19]	geçti
Belge pin tablosu: I2C SDA / SCL	[21, 22]	[21, 22]	geçti
Belge pin tablosu: Sol / sağ RPWM, LPWM	[27, 14, 16, 17]	[27, 14, 16, 17]	geçti
Belge pin tablosu: EN (iki kart)	[4]	[4]	geçti
Belge pin tablosu: Enkoder A, B	[32, 33, 25, 26]	[32, 33, 25, 26]	geçti
Belge pin tablosu: IR göz sol / sağ	[36, 39]	[36, 39]	geçti
Belge pin tablosu: Pil / şarj pimi gerilimi	[35, 34]	[35, 34]	geçti
Belge pin tablosu: Acil stop sonrası motor hattı	[23]	[23]	geçti
Belge pin tablosu: Karttaki LED	[2]	[2]	geçti
Ana kart J2 pin sirasi	3V3, GND, G22, G21, G19	3V3, GND, G22, G21, G19	geçti
Ana kart J3 pin sirasi	G27, G14, G4, 3V3, GND	G27, G14, G4, 3V3, GND	geçti
Ana kart J8 pin sirasi	G16, G17, G4, 3V3, GND	G16, G17, G4, 3V3, GND	geçti
Ana kart J4 pin sirasi	3V3, GND, G32, G33	3V3, GND, G32, G33	geçti
Ana kart J5 pin sirasi	3V3, GND, G25, G26	3V3, GND, G25, G26	geçti
Ana kart J6/J7	[36, 39]	[36, 39]	geçti
Iz genisligi (kod <-> belge 190 mm)	190	190	geçti
Enkoder sayim/tur (kod <-> belge 836)	836	836	geçti
Dusme acisi 40 deg	40	40	geçti
Dengeye girme +-3 deg, 0,5 s	3 / 0.5	3 / 0.5	geçti
Eldiven zaman asimi 0,3 s	0.3	0.3	geçti

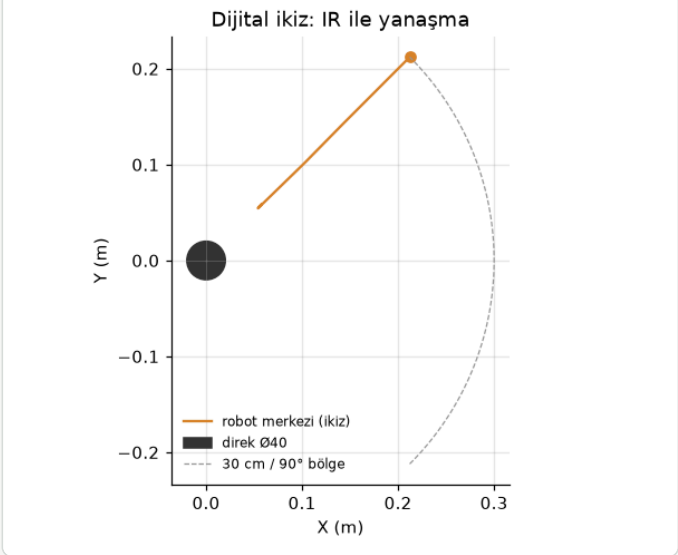
KONU	BEKLENEN	BULUNAN	DURUM
Adim 0,5 m / 90 deg	0.5 / 90	0.5 / 90	geçti
Sarj algilama esigi 9 V	9	9.0	geçti
Butunlesik gorevde bos bekleme 6 s	6	6.0	geçti
LQR kazanclari (gains.h <-> plan sayfasi)	u = 52,4·x + 64,7·v + 107,2·θ + 13,2·ω	plan sayfasinda var	geçti
BLE CDTP_SVC_UUID	6e400001-cd7b-4e3a-9a40-2026cd7b0001	belgede var	geçti
BLE CDTP_TELEM_UUID	6e400002-cd7b-4e3a-9a40-2026cd7b0001	belgede var	geçti
BLE CDTP_CMD_UUID	6e400003-cd7b-4e3a-9a40-2026cd7b0001	belgede var	geçti
Pil boluculeri 100k/27k	100/27	100/27	geçti
12,6 V -> ADC gerilimi < 3,1 V	< 3.1	2.68	geçti
Motor hatti bolucu 100k/33k -> < 3,3 V	< 3.3	3.13	geçti

Dijital ikiz nasıl kuruldu

- Gövde, kütle bütçesindeki her parçadan ayrı kutu olarak kuruldu. MuJoCo kütle merkezini ve ataleti kendisi hesaplar; C++ modelindeki elle girilen sayılar kullanılmaz.
- Teker–zemin teması, kayma, ilaç kutusunun cepte durması ve V-yuvanın direğe oturması gerçek temas hesabıyla çözülür.
- MPU6050 ölçümleri MuJoCo'nun ivmeölçer ve jiroskop sensörlerinden alınır. Bunlara gürültü ve 42 Hz süzgeç eklenir. Enkoder, teker eklemi ile gövde arasındaki bağıl açıdan hesaplanır.
- Karar mantığı ESP32'deki `RobotLogic` sınıfının kendisidir: `twin_bridge.exe` her 5 ms'de bir sensör satırı alır ve motor komutunu döndürür.



C++ simülasyonu ile dijital ikiz: aynı senaryo, aynı firmware mantığı



Dijital ikizde V-yuva ile direğe yanaşma, üstten

Ek · Tüm kodlar

Robotta çalışan firmware, ortak çekirdek, tasarım araçları, simülasyon ve dijital ikiz.

Robot firmware

firmware/robot/platformio.ini 23 satır

```
; Robot ESP32 (ESP32-WROOM-32D gelistirme karti)
; VS Code + PlatformIO eklentisiyle bu klasoru acip "Upload" deyin.
; Kod yuklerken ana karttaki JP1 koprusunu cikarin (USB ve LM2596 5 V'u cakismasin).

[platformio]
default_envs = robot

[env:robot]
platform = espressif32@6.9.0          ; Arduino-ESP32 2.0.17
board = esp32dev
framework = arduino
monitor_speed = 115200
upload_speed = 921600
lib_extra_dirs = ../../common
lib_deps =
  h2zero/NimBLE-Arduino@1.4.3
build_unflags = -std=gnu++11
build_flags =
  -std=gnu++17
  -O2
  -DCORE_DEBUG_LEVEL=1
  -DCONFIG_BT_NIMBLE_MAX_CONNECTIONS=1
```

firmware/robot/src/main.cpp 219 satır

```
// CDTP iki tekerlekli hasta destek robotu: robot ESP32 firmware'i.
// Cekirdek 1: 200 Hz denge dongusu (MPU6050 veri-hazir kesmesiyle tetiklenir).
// Cekirdek 0: ESP-NOW (eldiven), BLE (telefon), seri konsol.
// Tum karar mantigi common/cdtp_core/RobotLogic'tedir; PC simulasyonu ayni kodu test eder.
#include <Arduino.h>
#include <Preferences.h>

#include "hw.h"
#include "links.h"
#include "robot_logic.h"

using namespace cdtp;

// ---- Finalde true yapin: BLE/seri uzerinden ayar degisikligi kapanir (sartname: finalde parametre degismez) ----
static bool FREEZE = false;

static Mpu6050 imu;
static PcntCounter encL, encR, irL, irR;
static Motors motors;
static RobotLogic logic;
static Preferences prefs;

static SemaphoreHandle_t imuSem;
static portMUX_TYPE telemMux = portMUX_INITIALIZER_UNLOCKED;
static char telemJson[2][320];
static volatile int telemIdx = 0;
static volatile uint32_t loopMaxUs = 0, imuErrors = 0, overruns = 0;
static volatile int pendingTask = -1, pendingGlove = -1;
static volatile float pendingOffset = NAN;
static volatile int motorTestTicks = 0;  // mtest: once sol, sonra sag teker 1'er s ileri
static float vbatScale = 1.0f;

static void IRAM_ATTR onImuInt() {
  BaseType_t woke = pdFALSE;
  xSemaphoreGiveFromISR(imuSem, &woke);
  if (woke) portYIELD_FROM_ISR();
}

static void controlTask(void*) {
  RobotInputs in;
  uint32_t tick = 0, seq = 0;
  float vbat = 11.1f, vchg = 0;
  int failStreak = 0;
  const uint32_t t0 = micros();
  for (;;) {
    // veri-hazir kesmesi 5 ms'de bir gelir; 10 ms'de gelmezse yine de oku
    if (xSemaphoreTake(imuSem, pdMS_TO_TICKS(10)) != pdTRUE) overruns = overruns + 1;
    const uint32_t ts = micros();

    if (imu.read(in.imu)) {
      failStreak = 0;
    } else {
      imuErrors = imuErrors + 1;
      if (++failStreak >= 3) {  // IMU yok: guvenli taraf
        motors.set(false, 0, 0);
        continue;
      }
    }
    in.t = (ts - t0) * 1e-6f;
    in.encL = encL.read();
    in.encR = encR.read();
```

```
    in.irL = irL.read();
    in.irR = irR.read();
    if (tick % 4 == 0) {  // ADC 50 Hz
      vbat = readVolts(PIN_VBAT, vbatScale);
      vchg = readVolts(PIN_VCHG, vbatScale);
    }
    in.vbat = vbat;
    in.vcharge = vchg;
    in.motorPower = !HAS_MOTOR_SENSE || digitalRead(PIN_MOTOR_SENSE);
    in.gloveNew = gloverx::poll(in.glove);

    if (pendingTask >= 0) { logic.setTask((Task)pendingTask); pendingTask = -1; }
    if (pendingGlove >= 0) { logic.setGloveMode((GloveMode)pendingGlove); pendingGlove = -1; }
    if (!isnan(pendingOffset)) { logic.setPitchOffset(pendingOffset); pendingOffset = NAN; }

    RobotOutputs out = logic.step(in);
    if (motorTestTicks > 0) {
      // yalniz BEKLE modunda ve tekerler havadayken; %15 ileri, ters takilma bayraklari uygulanmis
      if (logic.mode() == Mode::STANDBY) {
        const float d = 0.15f;
        const bool left = motorTestTicks > 200;
        out.motorEnable = true;
        out.dutyL = left ? (cfg::MOTOR_LEFT_INVERT ? -d : d) : 0;
        out.dutyR = left ? 0 : (cfg::MOTOR_RIGHT_INVERT ? -d : d);
      }
      motorTestTicks = motorTestTicks - 1;
    }
    motors.set(out.motorEnable, out.dutyL, out.dutyR);

    // 1 Hz telemetri: JSON'u burada uret, radyo gorevi kopyalar
    if (++tick % 200 == 0) {
      int w = telemIdx ^ 1;
      logic.telemetryJson(telemJson[w], sizeof(telemJson[w]), seq++);
      portENTER_CRITICAL(&telemMux);
      telemIdx = w;
      portEXIT_CRITICAL(&telemMux);
    }
    digitalWrite(PIN_LED, logic.mode() == Mode::BALANCE || logic.mode() == Mode::DOCK ? HIGH : ((tick / 50) & 1));
    uint32_t dt = micros() - ts;
    if (dt > loopMaxUs) loopMaxUs = dt;
  }
}

// Seri ve BLE icin ortak metin komutlari
static void handleCommand(const char* c, Print& out) {
  if (!strncmp(c, "task=", 5)) {
    int t = atoi(c + 5);
    if (t >= 0 && t <= 3) {
      pendingTask = t;
      prefs.putUChar("task", t);
      out.printf("ok gorev=%s\n", taskName((Task)t));
    }
    return;
  }
  if (!strcmp(c, "mtest")) {
    motorTestTicks = 400;
    out.println("motor testi: sol 1 s, sag 1 s ileri (tekerler havada olsun)");
    return;
  }
  if (!strcmp(c, "stat")) {
    out.printf("dongu max %lu us, IMU hata %lu, gecikme %lu, eldiven paket %lu (kayip %lu, bozuk %lu), MPU=0x%02X\n",
               (unsigned long)loopMaxUs, (unsigned long)imuErrors, (unsigned long)overruns,
               (unsigned long)gloverx::count, (unsigned long)gloverx::lost, (unsigned long)gloverx::bad, imu.whoAmI);
    loopMaxUs = 0;
    return;
  }
  if (FREEZE) {
    out.println("FREEZE: ayar degisikligi kapali");
    return;
  }
  if (!strncmp(c, "glove=", 6)) {
    int m = !strcmp(c + 6, "cont") ? 1 : 0;
    pendingGlove = m;
    prefs.putUChar("glove", m);
    out.printf("ok eldiven modu=%s\n", m ? "surekli" : "adim");
  } else if (!strcmp(c, "cal")) {
    // Robotu elle denge noktasinda (en az caba ile durdugu acida) tutun, sonra "cal" yazin.
    float raw = logic.thetaDeg() + prefs.getFloat("ofs", 0);
    pendingOffset = raw * DEG;
    prefs.putFloat("ofs", raw);
    out.printf("ok egim ofseti=%.2f deg\n", raw);
  } else if (!strncmp(c, "vscale=", 7)) {
    vbatScale = atof(c + 7);
    prefs.putFloat("vscale", vbatScale);
    out.printf("ok pil olcek=%.4f\n", vbatScale);
  } else {
    out.println("komutlar: stat | mtest | task=0..3 | glove=step|cont | cal | vscale=1.000");
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(PIN_LED, OUTPUT);
  if (HAS_MOTOR_SENSE) pinMode(PIN_MOTOR_SENSE, INPUT);
```

```
  motors.begin();  // once motorlari kapali konuma al
  analogSetPinAttenuation(PIN_VBAT, ADC_11db);
  analogSetPinAttenuation(PIN_VCHG, ADC_11db);

  prefs.begin("cdtp", false);
  logic.setTask((Task)prefs.getUChar("task", (uint8_t)Task::INTEGRATED));
  logic.setGloveMode((GloveMode)prefs.getUChar("glove", 0));
  logic.setPitchOffset(prefs.getFloat("ofs", cfg::PITCH_OFFSET_DEG_DEFAULT) * DEG);
  vbatScale = prefs.getFloat("vscale", 1.0f);

  if (!imu.begin()) {
    Serial.println("HATA: MPU6050 bulunamadi (SDA=21, SCL=22, AD0=GND?)");
    for (;;) { digitalWrite(PIN_LED, !digitalRead(PIN_LED)); delay(100); }
  }
  Serial.println("Jiroskop kalibrasyonu: robotu kipirdatmayin...");
  while (!imu.calibrateGyro()) Serial.println("  hareket algilandi, tekrar");
  Serial.printf("  bias: %.4f %.4f %.4f rad/s\n", imu.bias[0], imu.bias[1], imu.bias[2]);

  encL.beginQuadrature(PCNT_UNIT_0, PIN_ENC_L_A, PIN_ENC_L_B, 100);
  encR.beginQuadrature(PCNT_UNIT_1, PIN_ENC_R_A, PIN_ENC_R_B, 100);
  irL.beginFallingEdges(PCNT_UNIT_2, PIN_IR_L, 1023);
  irR.beginFallingEdges(PCNT_UNIT_3, PIN_IR_R, 1023);

  if (!gloverx::begin()) Serial.println("HATA: ESP-NOW baslatilamadi");
  Serial.printf("Robot MAC (eldiven firmware'ine yazin): %s, kanal %d\n", WiFi.macAddress().c_str(), ESPNOW_CHANNEL);
  ble::begin("CDTP-Robot");

  snprintf(telemJson[0], sizeof telemJson[0], "{}");
  imuSem = xSemaphoreCreateBinary();
  pinMode(PIN_IMU_INT, INPUT);
  attachInterrupt(PIN_IMU_INT, onImuInt, RISING);
  xTaskCreatePinnedToCore(controlTask, "ctl", 6144, nullptr, configMAX_PRIORITIES - 1, nullptr, 1);
  Serial.println("Hazir. Robotu elle dik tutun: 0,5 s sonra dengeye girer.");
}

void loop() {
  static uint32_t lastTel = 0;
  static char line[64];
  static int n = 0;
  // 1 Hz: BLE bildirimi + seri kopya
  if (millis() - lastTel >= 1000) {
    lastTel = millis();
    char buf[320];
    portENTER_CRITICAL(&telemMux);
    memcpy(buf, telemJson[telemIdx], sizeof buf);
    portEXIT_CRITICAL(&telemMux);
    ble::publish(buf);
    Serial.println(buf);
  }
  if (ble::cmdReady) {
    handleCommand(ble::cmdBuf, Serial);
    ble::cmdReady = false;
  }
  while (Serial.available()) {
    char ch = Serial.read();
    if (ch == '\n' || ch == '\r') {
      if (n) { line[n] = 0; handleCommand(line, Serial); n = 0; }
    } else if (n < (int)sizeof(line) - 1) {
      line[n++] = ch;
    }
  }
  delay(5);
}
```

firmware/robot/src/pins.h 38 satır

```
// Robot ESP32 pin haritasi. Sitedeki (cdtp-robot.netlify.app > Sema > Robot pinleri) ile aynidir.
#pragma once

// IMU (MPU6050, I2C 400 kHz, adres 0x68: AD0 -> GND)
constexpr int PIN_SDA = 21;
constexpr int PIN_SCL = 22;
constexpr int PIN_IMU_INT = 19;

// BTS7960 x 2. R_EN ve L_EN iki kartta da GPIO4'e bagli; GPIO4 -> 10 kOhm -> GND
constexpr int PIN_L_RPWM = 27;  // sol motor ileri
constexpr int PIN_L_LPWM = 14;  // sol motor geri (acilista kisa darbe verir; EN kapaliyken zararsiz)
constexpr int PIN_R_RPWM = 16;  // sag motor ileri  (WROVER kartta 16/17 PSRAM'e gider: WROOM kullanin)
constexpr int PIN_R_LPWM = 17;  // sag motor geri
constexpr int PIN_MOTOR_EN = 4;

// Enkoderler (Hall, 3,3 V ile beslenir)
constexpr int PIN_ENC_L_A = 32;
constexpr int PIN_ENC_L_B = 33;
constexpr int PIN_ENC_R_A = 25;
constexpr int PIN_ENC_R_B = 26;

// IR gozleri (VS1838B; TSOP1838 4,5 V ister, 3,3 V'ta calismaz). Yalniz-giris pinleri.
// ESP32 hatasi (errata 3.11): ADC/Wi-Fi acikken GPIO36/39'da ~80 ns'lik dusuk darbeler olur.
// Kesme yerine PCNT + 12,8 us cam filtresi kullanildigi icin bu darbeler sayilmaz.
constexpr int PIN_IR_L = 36;
```

```
constexpr int PIN_IR_R = 39;

// Olcumler (ADC1; Wi-Fi/BLE acikken calisir). 100k / 27k bolucu: 12,6 V -> 2,68 V
constexpr int PIN_VBAT = 35;
constexpr int PIN_VCHG = 34;
constexpr float DIVIDER = (100.0f + 27.0f) / 27.0f;

// Acil stop sonrasi motor hatti: 100k/33k bolucu -> GPIO23 (12,6 V -> 3,1 V). Takili degilse false yapin.
constexpr bool HAS_MOTOR_SENSE = false;
constexpr int PIN_MOTOR_SENSE = 23;

constexpr int PIN_LED = 2;  // kart uzerindeki mavi LED (yalniz durum gosterir; sarj gostergesi DEGIL)
```

firmware/robot/src/hw.h 212 satır

```
// Robot donanim katmani: MPU6050, PCNT sayaclari, BTS7960 PWM, ADC.
#pragma once
#include <Arduino.h>
#include <Wire.h>
#include <driver/pcnt.h>

#include "imu_filter.h"
#include "pins.h"

// ---------------- MPU6050 ----------------
class Mpu6050 {
 public:
  bool begin() {
    Wire.begin(PIN_SDA, PIN_SCL, 400000);
    uint8_t who = read8(0x75);
    // Orijinali 0x68 doner; bazi klon cipler 0x70/0x72/0x98 doner, yine de calisir
    if (who == 0x00 || who == 0xFF) return false;
    whoAmI = who;
    write8(0x6B, 0x80);  // sifirla
    delay(100);
    write8(0x6B, 0x01);  // saat: PLL (jiroskop X)
    write8(0x1A, 0x03);  // DLPF ~42 Hz
    write8(0x19, 4);     // 1 kHz / (1+4) = 200 Hz
    write8(0x1B, 0x08);  // +-500 deg/s -> 65,5 LSB/(deg/s)
    write8(0x1C, 0x08);  // +-4 g -> 8192 LSB/g
    write8(0x37, 0x10);  // INT: aktif yuksek, okuyunca temizlenir
    write8(0x38, 0x01);  // veri hazir kesmesi
    return true;
  }
  // ham olcumu robot birimlerine cevirir (m/s^2, rad/s)
  bool read(cdtp::ImuSample& s) {
    uint8_t b[14];
    Wire.beginTransmission(ADDR);
    Wire.write(0x3B);
    if (Wire.endTransmission(false) != 0) return false;
    if (Wire.requestFrom((int)ADDR, 14) != 14) return false;
    for (int i = 0; i < 14; ++i) b[i] = Wire.read();
    auto w = [&](int i) { return (int16_t)((b[i] << 8) | b[i + 1]); };
    const float ka = 9.80665f / 8192.0f, kg = (3.14159265f / 180.0f) / 65.5f;
    for (int i = 0; i < 3; ++i) {
      s.acc[i] = w(i * 2) * ka;
      s.gyro[i] = w(8 + i * 2) * kg - bias[i];
    }
    return true;
  }
  // Acilista robot hareketsizken jiroskop sapmasini olcer (yatik da olabilir)
  bool calibrateGyro(int n = 400) {
    double sum[3] = {0, 0, 0}, sq[3] = {0, 0, 0};
    for (int i = 0; i < 3; ++i) bias[i] = 0;
    cdtp::ImuSample s;
    for (int k = 0; k < n; ++k) {
      if (!read(s)) return false;
      for (int i = 0; i < 3; ++i) {
        sum[i] += s.gyro[i];
        sq[i] += s.gyro[i] * s.gyro[i];
      }
      delay(5);
    }
    for (int i = 0; i < 3; ++i) {
      double m = sum[i] / n, var = sq[i] / n - m * m;
      if (var > 0.0004) return false;  // ~1,1 deg/s std: robot kipirdadi, tekrar
      bias[i] = (float)m;
    }
    return true;
  }
  float bias[3] = {0, 0, 0};
  uint8_t whoAmI = 0;

 private:
  static constexpr uint8_t ADDR = 0x68;
  void write8(uint8_t r, uint8_t v) {
    Wire.beginTransmission(ADDR);
    Wire.write(r);
    Wire.write(v);
    Wire.endTransmission();
  }
```

```
  uint8_t read8(uint8_t r) {
    Wire.beginTransmission(ADDR);
    Wire.write(r);
    Wire.endTransmission(false);
    Wire.requestFrom((int)ADDR, 1);
    return Wire.available() ? Wire.read() : 0xFF;
  }
};

// ---------------- PCNT: enkoder (4x) ve IR darbe sayaci ----------------
// 16 bitlik donanim sayacini yazilimda 32 bite genisletir. Her 5 ms okunmasi yeterli.
class PcntCounter {
 public:
  void beginQuadrature(pcnt_unit_t u, int a, int b, uint16_t filter) {
    unit_ = u;
    pcnt_config_t c = {};
    c.unit = u;
    c.counter_h_lim = LIM;
    c.counter_l_lim = -LIM;
    // kanal 0: A kenarlari, yon B'den
    c.channel = PCNT_CHANNEL_0;
    c.pulse_gpio_num = a;
    c.ctrl_gpio_num = b;
    c.pos_mode = PCNT_COUNT_DEC;
    c.neg_mode = PCNT_COUNT_INC;
    c.lctrl_mode = PCNT_MODE_REVERSE;
    c.hctrl_mode = PCNT_MODE_KEEP;
    pcnt_unit_config(&c);
    // kanal 1: B kenarlari, yon A'dan
    c.channel = PCNT_CHANNEL_1;
    c.pulse_gpio_num = b;
    c.ctrl_gpio_num = a;
    c.pos_mode = PCNT_COUNT_INC;
    c.neg_mode = PCNT_COUNT_DEC;
    pcnt_unit_config(&c);
    gpio_pullup_en((gpio_num_t)a);
    gpio_pullup_en((gpio_num_t)b);
    finish(filter);
  }
  void beginFallingEdges(pcnt_unit_t u, int pin, uint16_t filter) {
    unit_ = u;
    pcnt_config_t c = {};
    c.unit = u;
    c.channel = PCNT_CHANNEL_0;
    c.pulse_gpio_num = pin;
    c.ctrl_gpio_num = PCNT_PIN_NOT_USED;
    c.pos_mode = PCNT_COUNT_DIS;
    c.neg_mode = PCNT_COUNT_INC;  // alici cikisi patlama sirasinda dusuk
    c.lctrl_mode = PCNT_MODE_KEEP;
    c.hctrl_mode = PCNT_MODE_KEEP;
    c.counter_h_lim = LIM;
    c.counter_l_lim = -LIM;
    pcnt_unit_config(&c);
    finish(filter);
  }
  int32_t read() {
    int16_t v = 0;
    pcnt_get_counter_value(unit_, &v);
    int32_t d = (int32_t)v - last_;
    if (d > LIM / 2) d -= LIM;  // sayac +-LIM'e ulasinca 0'a doner
    if (d < -LIM / 2) d += LIM;
    last_ = v;
    acc_ += d;
    return acc_;
  }

 private:
  static constexpr int16_t LIM = 30000;
  void finish(uint16_t filter) {
    pcnt_set_filter_value(unit_, filter);  // APB 80 MHz cevrimi; 1023 = 12,8 us
    pcnt_filter_enable(unit_);
    pcnt_counter_pause(unit_);
    pcnt_counter_clear(unit_);
    pcnt_counter_resume(unit_);
  }
  pcnt_unit_t unit_ = PCNT_UNIT_0;
  int16_t last_ = 0;
  int32_t acc_ = 0;
};

// ---------------- BTS7960 x 2 ----------------
class Motors {
 public:
  void begin() {
    pinMode(PIN_MOTOR_EN, OUTPUT);
    digitalWrite(PIN_MOTOR_EN, LOW);
    const int pins[4] = {PIN_L_RPWM, PIN_L_LPWM, PIN_R_RPWM, PIN_R_LPWM};
    for (int i = 0; i < 4; ++i) {
#if ESP_ARDUINO_VERSION_MAJOR >= 3
      ledcAttach(pins[i], FREQ, BITS);
#else
      ledcSetup(i, FREQ, BITS);
      ledcAttachPin(pins[i], i);
#endif
```

```
      write(i, 0);
    }
  }
  // duty: -1..1. enable=false: iki surucu de yuksek empedans (motor serbest)
  void set(bool enable, float dl, float dr) {
    if (!enable) {
      digitalWrite(PIN_MOTOR_EN, LOW);
      for (int i = 0; i < 4; ++i) write(i, 0);
      return;
    }
    side(0, dl);
    side(2, dr);
    digitalWrite(PIN_MOTOR_EN, HIGH);
  }

 private:
  static constexpr int FREQ = 20000, BITS = 10, MAXD = (1 << BITS) - 1;
  void side(int ch, float d) {
    int v = (int)(fabsf(d) * MAXD + 0.5f);
    if (v > MAXD) v = MAXD;
    // asla iki yari kopru ayni anda surulmez
    if (d >= 0) {
      write(ch + 1, 0);
      write(ch, v);
    } else {
      write(ch, 0);
      write(ch + 1, v);
    }
  }
  void write(int ch, int v) {
#if ESP_ARDUINO_VERSION_MAJOR >= 3
    const int pins[4] = {PIN_L_RPWM, PIN_L_LPWM, PIN_R_RPWM, PIN_R_LPWM};
    ledcWrite(pins[ch], v);
#else
    ledcWrite(ch, v);
#endif
  }
};

// ---------------- ADC ----------------
inline float readVolts(int pin, float scale) { return analogReadMilliVolts(pin) * 0.001f * DIVIDER * scale; }
```

firmware/robot/src/links.h 114 satır

```
// Kablosuz baglantilar: eldivenden ESP-NOW, telefona BLE.
#pragma once
#include <Arduino.h>
#include <NimBLEDevice.h>
#include <WiFi.h>
#include <esp_now.h>
#include <esp_wifi.h>

#include "protocol.h"

constexpr uint8_t ESPNOW_CHANNEL = 1;  // eldivenle ayni olmali

// ---------------- ESP-NOW alici ----------------
namespace gloverx {
static portMUX_TYPE mux = portMUX_INITIALIZER_UNLOCKED;
static cdtp::GlovePacket last;
static volatile uint32_t count = 0, bad = 0, lost = 0;
static uint32_t seenCount = 0;
static uint16_t lastSeq = 0;

static void onPacket(const uint8_t* data, int len) {
  cdtp::GlovePacket p;
  if (!cdtp::checkPacket(data, len, p)) {
    bad = bad + 1;
    return;
  }
  portENTER_CRITICAL(&mux);
  if (count > 0) lost = lost + (uint16_t)(p.seq - lastSeq - 1);
  lastSeq = p.seq;
  last = p;
  count = count + 1;
  portEXIT_CRITICAL(&mux);
}
#if ESP_ARDUINO_VERSION_MAJOR >= 3
static void cb(const esp_now_recv_info_t*, const uint8_t* d, int n) { onPacket(d, n); }
#else
static void cb(const uint8_t*, const uint8_t* d, int n) { onPacket(d, n); }
#endif

inline bool begin() {
  WiFi.mode(WIFI_STA);
  WiFi.disconnect();
  esp_wifi_set_channel(ESPNOW_CHANNEL, WIFI_SECOND_CHAN_NONE);
  if (esp_now_init() != ESP_OK) return false;
  esp_now_register_recv_cb(cb);
  return true;
}
```

```
// Kontrol dongusu cagirir: yeni paket varsa true
inline bool poll(cdtp::GlovePacket& out) {
  bool fresh = false;
  portENTER_CRITICAL(&mux);
  if (count != seenCount) {
    seenCount = count;
    out = last;
    fresh = true;
  }
  portEXIT_CRITICAL(&mux);
  return fresh;
}
}  // namespace gloverx

// ---------------- BLE: telefon uygulamasi ----------------
// Servis ve karakteristik UUID'leri uygulama ekibiyle paylasilir (docs/BLE_PROTOKOL.md).
#define CDTP_SVC_UUID "6e400001-cd7b-4e3a-9a40-2026cd7b0001"
#define CDTP_TELEM_UUID "6e400002-cd7b-4e3a-9a40-2026cd7b0001"  // notify + read, JSON, 1 Hz
#define CDTP_CMD_UUID "6e400003-cd7b-4e3a-9a40-2026cd7b0001"    // write, metin komut

namespace ble {
static NimBLECharacteristic* telem = nullptr;
static volatile bool connected = false;
static char cmdBuf[64];
static volatile bool cmdReady = false;

class ServerCb : public NimBLEServerCallbacks {
  void onConnect(NimBLEServer*) override { connected = true; }
  void onDisconnect(NimBLEServer*) override {
    connected = false;
    NimBLEDevice::startAdvertising();
  }
};
class CmdCb : public NimBLECharacteristicCallbacks {
  void onWrite(NimBLECharacteristic* c) override {
    std::string v = c->getValue();
    if (cmdReady) return;  // onceki komut islenmedi
    size_t n = v.size() < sizeof(cmdBuf) - 1 ? v.size() : sizeof(cmdBuf) - 1;
    memcpy(cmdBuf, v.data(), n);
    cmdBuf[n] = 0;
    cmdReady = true;
  }
};

inline void begin(const char* name) {
  NimBLEDevice::init(name);
  NimBLEDevice::setMTU(247);
  NimBLEDevice::setPower(ESP_PWR_LVL_P9);
  NimBLEServer* s = NimBLEDevice::createServer();
  s->setCallbacks(new ServerCb());
  NimBLEService* svc = s->createService(CDTP_SVC_UUID);
  telem = svc->createCharacteristic(CDTP_TELEM_UUID, NIMBLE_PROPERTY::READ | NIMBLE_PROPERTY::NOTIFY);
  NimBLECharacteristic* cmd = svc->createCharacteristic(CDTP_CMD_UUID, NIMBLE_PROPERTY::WRITE | NIMBLE_PROPERTY::WRITE_NR);
  cmd->setCallbacks(new CmdCb());
  svc->start();
  NimBLEAdvertising* adv = NimBLEDevice::getAdvertising();
  adv->addServiceUUID(CDTP_SVC_UUID);
  adv->setScanResponse(true);
  adv->start();
}
inline void publish(const char* json) {
  if (!telem) return;
  telem->setValue((const uint8_t*)json, strlen(json));
  if (connected) telem->notify();
}
}  // namespace ble
```

Ortak çekirdek (robotta çalışan karar mantığı)

common/cdtp_core/library.json 8 satır

```
{
  "name": "cdtp_core",
  "version": "1.0.0",
  "description": "CDTP denge robotu: donanimdan bagimsiz kontrol, gorev ve protokol kodu. Hem ESP32 firmware'i hem PC simulasyonu bunu kullanir.",
  "frameworks": "*",
  "platforms": "*"
}
```

common/cdtp_core/src/cdtp_config.h 80 satır

```
// Robot parametreleri. "OLC" yazanlar parca gelince olculup guncellenecek.
// Bu dosya hem firmware'de hem simulasyonda ayni degerlerle kullanilir.
#pragma once

namespace cdtp {
namespace cfg {

// ---- Mekanik (OLC) ----
constexpr float WHEEL_RADIUS_M   = 0.0325f;  // 65 mm teker
constexpr float TRACK_WIDTH_M    = 0.190f;   // iki teker temas noktasi arasi (sasi cizimi; OLC)
constexpr float ENC_COUNTS_PER_REV = 836.0f; // 11 darbe x ~19 disli x 4 kenar (OLC: tekeri 10 tur cevir)
constexpr bool  ENC_LEFT_INVERT  = false;
constexpr bool  ENC_RIGHT_INVERT = true;     // sag motor ayna takili
constexpr bool  MOTOR_LEFT_INVERT  = false;
constexpr bool  MOTOR_RIGHT_INVERT = true;

// ---- Dongu ----
constexpr float LOOP_HZ = 200.0f;
constexpr float DT      = 1.0f / LOOP_HZ;

// ---- IMU filtresi ----
constexpr float COMP_ALPHA = 0.98f;           // tumleyen filtre: 0.98 jiroskop, 0.02 ivme
constexpr float PITCH_OFFSET_DEG_DEFAULT = 0.0f; // denge noktasi ofseti (NVS'ten yuklenir)

// ---- Motor / guc ----
constexpr float VBAT_NOMINAL   = 11.1f;
constexpr float VBAT_EMPTY     = 9.9f;        // 3.3 V/hucre
constexpr float VBAT_FULL      = 12.6f;
constexpr float DUTY_MAX       = 0.95f;
constexpr float MOTOR_DEADBAND_V = 0.45f;     // kucuk gerilimde dislinin donmemesi (OLC)
constexpr float MOTOR_KE       = 0.216f;      // V/(rad/s), cikis milinde: 12 V / 55.5 rad/s (530 RPM)

// ---- Kontrol ----
constexpr float TRIM_RATE = 0.1f;      // rad/(m s): dururken denge acisi ogrenme hizi
constexpr float TRIM_MAX_DEG = 3.0f;   // ogrenilen aci siniri
constexpr float REF_SHAPING_W = 5.0f;  // rad/s, konum referansi suzgeci
constexpr float TH_FF_GAIN = 1.0f;   // ivme -> egim on-beslemesinin ne kadari kullanilsin (sim ile secildi)

// ---- Guvenlik ----
constexpr float FALL_ANGLE_DEG    = 40.0f;    // bunun ustunde motorlar kapanir
constexpr float FALL_HOLD_S       = 0.05f;
constexpr float ARM_ANGLE_DEG     = 3.0f;     // elle dik tutulunca bu acida dengeye girer
constexpr float ARM_HOLD_S        = 0.5f;
constexpr float GLOVE_TIMEOUT_S   = 0.30f;    // bu sure paket yoksa dur

// ---- Hareket ----
constexpr float STEP_DIST_M     = 0.50f;      // ileri/geri komutu basina yol
constexpr float STEP_VMAX       = 0.25f;      // m/s
constexpr float STEP_ACCEL      = 0.40f;      // m/s^2
constexpr float TURN_ANGLE_DEG  = 90.0f;
constexpr float TURN_RATE_DEG   = 90.0f;      // deg/s
constexpr float TURN_ACCEL_DEG  = 240.0f;     // deg/s^2
constexpr float CONT_V          = 0.20f;      // surekli modda hiz
constexpr float CONT_YAW_DEG    = 60.0f;      // surekli modda donus hizi

// ---- Yanasma ----
constexpr float DOCK_SEARCH_YAW_DEG = 30.0f;
constexpr float DOCK_ALIGN_K_DEG    = 40.0f;  // e=1 icin deg/s
constexpr float DOCK_ALIGN_TOL      = 0.25f;
constexpr float DOCK_APPROACH_V     = 0.08f;
constexpr int   DOCK_MIN_COUNT      = 6;      // bir gozun "gordu" sayilmasi icin cerceve basina darbe
constexpr float DOCK_TRIGGER_S      = 1.0f;   // gozlerden biri isareti bu kadar gorurse yanasmaya gec
constexpr float DOCK_IDLE_S         = 6.0f;   // eldiven bu kadar "dur"daysa yanasma serbest (hedefte 5 s bekleme bozulmasin)
constexpr float DOCK_LOST_S         = 1.0f;
constexpr float DOCK_TIMEOUT_S      = 60.0f;
constexpr float UNDOCK_HOLD_S       = 10.0f;  // eldivenle istasyondan ayrilinca bu sure yeniden yanasmaz
constexpr float CHARGE_DETECT_V     = 9.0f;   // plakada bu gerilim varsa sarj var
constexpr float CHARGE_CONFIRM_S    = 0.10f;
constexpr float DOCK_LEAN_DEG       = 4.0f;   // motor kesmeden once sirta dogru egilme

// ---- Eldiven jest ----
constexpr float GEST_ENTER_DEG   = 25.0f;
constexpr float GEST_EXIT_DEG    = 15.0f;
constexpr float GEST_NEUTRAL_DEG = 12.0f;
constexpr float GEST_HOLD_S      = 0.15f;     // komuta girmek icin bekleme
constexpr float GEST_NEUTRAL_S   = 0.20f;
```

```
}  // namespace cfg
}  // namespace cdtp
```

common/cdtp_core/src/gains.h 17 satır

```
// OTOMATIK URETILDI: tools/lqr_design.py. Elle degistirmeyin; modeli guncelleyip yeniden calistirin.
// Durum: [x (m), v (m/s), theta (rad), omega (rad/s)], cikis: her motora gerilim (V).
// u = -K (s - s_ref)
#pragma once
namespace cdtp {
namespace gains {
constexpr float K_X = -52.40356f;
constexpr float K_V = -64.68360f;
constexpr float K_TH = -107.16416f;
constexpr float K_OM = -13.15230f;
constexpr float YAW_KP = 2.73684f;
constexpr float YAW_KD = 0.61579f;
// a_ref -> egim on-besleme (rad / (m/s^2))
constexpr float TH_PER_ACC = 0.15266f;
}  // namespace gains
}  // namespace cdtp
```

common/cdtp_core/src/robot_logic.h 119 satır

```
// Robotun tum karar mantigi: mod makinesi, denge (LQR), hareket, yanasma.
// Donanimdan bagimsizdir: firmware her 5 ms'de bir step() cagirir, simulasyon da ayni sinifi kullanir.
#pragma once
#include <stdint.h>
#include "battery.h"
#include "cdtp_config.h"
#include "core_math.h"
#include "dock_fsm.h"
#include "imu_filter.h"
#include "ir_frame.h"
#include "protocol.h"

namespace cdtp {

enum class Mode : uint8_t { STANDBY, BALANCE, DOCK, CHARGING, FALLEN };
enum class Task : uint8_t { STATIC = 0, GUIDANCE = 1, CHARGING = 2, INTEGRATED = 3 };
enum class GloveMode : uint8_t { STEP = 0, CONTINUOUS = 1 };
enum class MotionState : uint8_t { STATIC, MOVING, TURNING, RECOVERY };

const char* modeName(Mode m);
const char* taskName(Task t);
const char* motionName(MotionState m);

struct RobotInputs {
  float t = 0;               // s
  ImuSample imu{};           // ham eksen, gyro bias cikarilmis
  int32_t encL = 0, encR = 0;  // kumulatif sayim (ham)
  float vbat = 11.1f;        // V
  float vcharge = 0;         // plaka gerilimi, V
  int32_t irL = 0, irR = 0;  // IR dusen kenar sayaclari (kumulatif)
  bool motorPower = true;    // acil stop sonrasi 12 V var mi (olculmuyorsa hep true)
  bool gloveNew = false;     // bu adimda yeni paket geldi mi
  GlovePacket glove{};
};

struct RobotOutputs {
  bool motorEnable = false;
  float dutyL = 0, dutyR = 0;  // -1..1, + = ileri (yon cevirme dahil edilmis)
};

class RobotLogic {
 public:
  RobotLogic();
  void setAxisMap(const AxisMap& m) { axes_ = m; }
  void setPitchOffset(float rad) { tilt_.setOffset(rad); }
  void setTask(Task t) { task_ = t; }
  void setGloveMode(GloveMode m) { gloveMode_ = m; }

  RobotOutputs step(const RobotInputs& in);

  // Telemetri / izleme
  Mode mode() const { return mode_; }
  Task task() const { return task_; }
  MotionState motion() const { return motion_; }
  const char* direction() const { return dir_; }
  DockState dockState() const { return dock_.state(); }
  float thetaDeg() const { return tilt_.theta() / DEG; }
  float trimDeg() const { return thTrim_ / DEG; }
  float x() const { return x_; }
  float v() const { return v_; }
  float psiDeg() const { return psi_ / DEG; }
  bool charging() const { return charging_; }
  int batteryPct() const { return bat_.percent(); }
  float batteryV() const { return bat_.volts(); }
```

```
  int irNL() const { return irL_.n(); }
  int irNR() const { return irR_.n(); }
  Cmd lastGloveCmd() const { return gloveCmd_; }
  bool gloveAlive() const { return gloveAlive_; }
  int telemetryJson(char* buf, int len, uint32_t seq) const;

 private:
  void enterBalance();
  void handleGlove(const RobotInputs& in);
  void startStep(Cmd c);
  bool dockAllowed() const;
  RobotOutputs balanceControl(float vbat, float leanRad);
  void updateMotionState();

  AxisMap axes_;
  TiltFilter tilt_;
  BatteryMonitor bat_;
  IrFrameCounter irL_, irR_;
  DockFsm dock_;

  Mode mode_ = Mode::STANDBY;
  Task task_ = Task::INTEGRATED;
  GloveMode gloveMode_ = GloveMode::STEP;
  MotionState motion_ = MotionState::STATIC;
  const char* dir_ = "-";

  float t_ = 0, modeT_ = 0, armT_ = 0, fallT_ = 0;
  // olcumler
  int32_t encL0_ = 0, encR0_ = 0;
  bool encInit_ = false;
  float x_ = 0, v_ = 0, psi_ = 0;
  float xHist_[4] = {0, 0, 0, 0};
  int histIdx_ = 0;
  // referanslar
  float xRaw_ = 0, vRaw_ = 0;  // bicimlendirme oncesi referans
  float thTrim_ = 0;           // ogrenilen denge acisi (rad); acilista sifir, cal komutu kalici ofseti verir
  float xRef_ = 0, vRef_ = 0, aRef_ = 0, psiRef_ = 0, rRef_ = 0;
  float xStart_ = 0, psiStart_ = 0;
  Trapezoid lin_, rot_;
  bool linActive_ = false, rotActive_ = false;
  float vCmd_ = 0, rCmd_ = 0;  // hiz modu hedefleri (surekli mod / yanasma)
  // eldiven
  Cmd gloveCmd_ = Cmd::STOP, prevGloveCmd_ = Cmd::STOP;
  float gloveLastT_ = -10, gloveIdleT_ = 0;
  bool gloveAlive_ = false;
  bool pendingBack_ = false;
  // yanasma tetik
  float beaconT_ = 0;
  float undockHoldT_ = 0;  // eldivenle ayrildiktan sonra otomatik yanasma beklemesi
  bool charging_ = false;
  float chargeLostT_ = 0;
};

}  // namespace cdtp
```

common/cdtp_core/src/robot_logic.cpp 325 satır

```
#include "robot_logic.h"

#include <stdio.h>

#include "gains.h"

namespace cdtp {

const char* modeName(Mode m) {
  switch (m) {
    case Mode::STANDBY: return "BEKLE";
    case Mode::BALANCE: return "DENGE";
    case Mode::DOCK: return "YANASMA";
    case Mode::CHARGING: return "SARJ";
    case Mode::FALLEN: return "DUSTU";
  }
  return "?";
}
const char* taskName(Task t) {
  switch (t) {
    case Task::STATIC: return "statik";
    case Task::GUIDANCE: return "yonlendirme";
    case Task::CHARGING: return "sarj";
    case Task::INTEGRATED: return "butunlesik";
  }
  return "?";
}
const char* motionName(MotionState m) {
  switch (m) {
    case MotionState::STATIC: return "statik";
    case MotionState::MOVING: return "ilerliyor";
    case MotionState::TURNING: return "donuyor";
    case MotionState::RECOVERY: return "toparlaniyor";
  }
```

```
  return "?";
}

RobotLogic::RobotLogic() { tilt_.setOffset(cfg::PITCH_OFFSET_DEG_DEFAULT * DEG); }

void RobotLogic::enterBalance() {
  mode_ = Mode::BALANCE;
  modeT_ = 0;
  xRef_ = xRaw_ = x_;
  vRef_ = aRef_ = vRaw_ = 0;
  psiRef_ = psi_;
  rRef_ = 0;
  linActive_ = rotActive_ = false;
  vCmd_ = rCmd_ = 0;
  dock_.abort();
  armT_ = beaconT_ = 0;
}

bool RobotLogic::dockAllowed() const { return task_ == Task::CHARGING || task_ == Task::INTEGRATED; }

void RobotLogic::startStep(Cmd c) {
  if (c == Cmd::FWD || c == Cmd::BACK) {
    xStart_ = xRaw_;
    lin_.start(c == Cmd::FWD ? cfg::STEP_DIST_M : -cfg::STEP_DIST_M, cfg::STEP_VMAX, cfg::STEP_ACCEL);
    linActive_ = true;
  } else if (c == Cmd::LEFT || c == Cmd::RIGHT) {
    psiStart_ = psiRef_;
    float a = cfg::TURN_ANGLE_DEG * DEG;
    rot_.start(c == Cmd::LEFT ? a : -a, cfg::TURN_RATE_DEG * DEG, cfg::TURN_ACCEL_DEG * DEG);
    rotActive_ = true;
  }
}

void RobotLogic::handleGlove(const RobotInputs& in) {
  const float dt = cfg::DT;
  Cmd cmd = gloveCmd_;
  if (in.gloveNew) {
    gloveLastT_ = in.t;
    cmd = (Cmd)in.glove.cmd;
  }
  const bool wasAlive = gloveAlive_;
  gloveAlive_ = (in.t - gloveLastT_) < cfg::GLOVE_TIMEOUT_S;
  if (!gloveAlive_) cmd = Cmd::STOP;
  const bool edge = cmd != prevGloveCmd_;
  prevGloveCmd_ = cmd;
  gloveCmd_ = cmd;
  const bool motionCmd = cmd == Cmd::FWD || cmd == Cmd::BACK || cmd == Cmd::LEFT || cmd == Cmd::RIGHT;
  gloveIdleT_ = motionCmd ? 0 : gloveIdleT_ + dt;

  if (mode_ == Mode::CHARGING) {
    // istasyondan eldivenle ayrilma: "geri" komutu
    if (edge && cmd == Cmd::BACK) {
      enterBalance();
      pendingBack_ = true;
      undockHoldT_ = cfg::UNDOCK_HOLD_S;  // kullanici ayirdi: hemen geri yanasma
    }
    return;
  }
  if (mode_ != Mode::BALANCE || task_ == Task::STATIC) return;

  if (gloveMode_ == GloveMode::CONTINUOUS) {
    vCmd_ = cmd == Cmd::FWD ? cfg::CONT_V : (cmd == Cmd::BACK ? -cfg::CONT_V : 0.0f);
    rCmd_ = cmd == Cmd::LEFT ? cfg::CONT_YAW_DEG * DEG : (cmd == Cmd::RIGHT ? -cfg::CONT_YAW_DEG * DEG : 0.0f);
    return;
  }
  // Adim modu: her hareket jesti bir adim (0,5 m ya da 90 deg). Notr el ("dur") adimi kesmez, robot adimi bitirip durur.
  // Adim surerken farkli bir hareket jesti ya da baglanti kopmasi adimi yumusakca keser.
  if (wasAlive && !gloveAlive_ && (linActive_ || rotActive_)) {
    lin_.stopSoft();
    rot_.stopSoft();
  }
  if (edge && motionCmd) {
    if (linActive_ || rotActive_) {
      lin_.stopSoft();
      rot_.stopSoft();
    } else {
      startStep(cmd);
    }
  }
}

RobotOutputs RobotLogic::balanceControl(float vbat, float leanRad) {
  using namespace gains;
  const float r = cfg::WHEEL_RADIUS_M;
  const float thRef = cfg::TH_FF_GAIN * TH_PER_ACC * aRef_ + leanRad;
  const float ex = clampf(x_ - xRef_, -0.25f, 0.25f);
  // Denge noktasi ogrenme: kutle merkezi eksenin tam ustunde degilse robot egik dengede durur ve LQR bunu
  // kalici konum hatasiyla karsilar (dururken yavas kayma). Robot yerinde dururken egim hedefi yavasca
  // bu hatayi sifirlayacak yone kaydirilir. Hareket, donus ve yanasmada donmus kalir.
  const bool still = !linActive_ && !rotActive_ && fabsf(vRef_) < 0.01f && fabsf(rRef_) < 0.05f && mode_ == Mode::BALANCE;
  if (still) thTrim_ = clampf(thTrim_ - cfg::TRIM_RATE * ex * cfg::DT, -cfg::TRIM_MAX_DEG * DEG, cfg::TRIM_MAX_DEG * DEG);
  float u = -(K_X * ex + K_V * (v_ - vRef_) + K_TH * (tilt_.theta() - thRef - thTrim_) + K_OM * tilt_.pitchRate());
  u += cfg::MOTOR_KE * vRef_ / r;  // zit-EMK on-beslemesi
  float ud = YAW_KP * wrapPi(psiRef_ - psi_) + YAW_KD * (rRef_ - tilt_.yawRate());
```

```
  ud += cfg::MOTOR_KE * (rRef_ * cfg::TRACK_WIDTH_M * 0.5f) / r;

  vbat = fmaxf(vbat, 9.0f);
  const float vmax = cfg::DUTY_MAX * vbat;
  u = clampf(u, -vmax, vmax);
  const float udMax = fmaxf(0.0f, vmax - fabsf(u));  // denge onceliklidir, donus payi kalandan
  ud = clampf(ud, -udMax, udMax);

  float vl = u - ud, vr = u + ud;
  // olu bolge telafisi (yumusak)
  vl += cfg::MOTOR_DEADBAND_V * clampf(vl / 0.3f, -1, 1);
  vr += cfg::MOTOR_DEADBAND_V * clampf(vr / 0.3f, -1, 1);

  RobotOutputs o;
  o.motorEnable = true;
  o.dutyL = clampf(vl / vbat, -cfg::DUTY_MAX, cfg::DUTY_MAX);
  o.dutyR = clampf(vr / vbat, -cfg::DUTY_MAX, cfg::DUTY_MAX);
  if (cfg::MOTOR_LEFT_INVERT) o.dutyL = -o.dutyL;
  if (cfg::MOTOR_RIGHT_INVERT) o.dutyR = -o.dutyR;
  return o;
}

void RobotLogic::updateMotionState() {
  const float th = fabsf(tilt_.theta()), om = fabsf(tilt_.pitchRate());
  if (mode_ != Mode::BALANCE && mode_ != Mode::DOCK) {
    motion_ = MotionState::STATIC;
    dir_ = "-";
    return;
  }
  if (th > 5 * DEG || om > 40 * DEG) motion_ = MotionState::RECOVERY;
  else if (rotActive_ || fabsf(rRef_) > 0.1f) motion_ = MotionState::TURNING;
  else if (linActive_ || fabsf(vRef_) > 0.02f) motion_ = MotionState::MOVING;
  else motion_ = MotionState::STATIC;
  if (motion_ == MotionState::TURNING) dir_ = rRef_ > 0 ? "sol" : "sag";
  else if (motion_ == MotionState::MOVING) dir_ = vRef_ > 0 ? "ileri" : "geri";
  else dir_ = "-";
}

RobotOutputs RobotLogic::step(const RobotInputs& in) {
  const float dt = cfg::DT;
  t_ = in.t;
  modeT_ += dt;

  tilt_.update(in.imu, axes_, dt);
  bat_.update(in.vbat, dt);
  irL_.sample(in.irL, in.t);
  irR_.sample(in.irR, in.t);
  charging_ = in.vcharge > cfg::CHARGE_DETECT_V;

  if (!encInit_) {
    encL0_ = in.encL;
    encR0_ = in.encR;
    encInit_ = true;
  }
  const float mPerCount = 2 * PI_F * cfg::WHEEL_RADIUS_M / cfg::ENC_COUNTS_PER_REV;
  const float xl = (cfg::ENC_LEFT_INVERT ? -1 : 1) * (float)(in.encL - encL0_) * mPerCount;
  const float xr = (cfg::ENC_RIGHT_INVERT ? -1 : 1) * (float)(in.encR - encR0_) * mPerCount;
  // Enkoder tekerin govdeye gore donusunu sayar; yere gore teker yolu icin govde egimi eklenir: x = r*(phi_bagil + theta)
  x_ = 0.5f * (xl + xr) + cfg::WHEEL_RADIUS_M * tilt_.theta();
  // hiz: 4 ornek (20 ms) uzerinden fark; tek sayim kuantalamasini 4 kat azaltir
  const float xOld = xHist_[histIdx_];
  xHist_[histIdx_] = x_;
  histIdx_ = (histIdx_ + 1) & 3;
  v_ = (x_ - xOld) / (4 * dt);
  psi_ += tilt_.yawRate() * dt;

  handleGlove(in);

  const float th = tilt_.theta();
  RobotOutputs out;  // varsayilan: motorlar kapali

  if (mode_ == Mode::BALANCE || mode_ == Mode::DOCK) {
    fallT_ = fabsf(th) > cfg::FALL_ANGLE_DEG * DEG ? fallT_ + dt : 0;
    if (fallT_ > cfg::FALL_HOLD_S) {
      mode_ = Mode::FALLEN;
      modeT_ = 0;
      dock_.abort();
      linActive_ = rotActive_ = false;
      pendingBack_ = false;
    }
  }

  // Acil stop basili: motorlara guc yok. Denge birakilir; stop acilinca robot yeniden dik tutulup baslatilir,
  // boylece birikmis hata ile motorlar aniden donmez.
  if (!in.motorPower && (mode_ == Mode::BALANCE || mode_ == Mode::DOCK)) {
    mode_ = Mode::STANDBY;
    modeT_ = 0;
    armT_ = 0;
    dock_.abort();
    linActive_ = rotActive_ = false;
    pendingBack_ = false;
  }

  switch (mode_) {
```

```
    case Mode::STANDBY:
      armT_ = (in.motorPower && fabsf(th) < cfg::ARM_ANGLE_DEG * DEG) ? armT_ + dt : 0;
      if (armT_ > cfg::ARM_HOLD_S) enterBalance();
      else if (charging_) { mode_ = Mode::CHARGING; modeT_ = 0; chargeLostT_ = 0; }
      break;

    case Mode::FALLEN:
      if (fabsf(th) < 20 * DEG) { mode_ = Mode::STANDBY; modeT_ = 0; armT_ = 0; }
      break;

    case Mode::CHARGING:
      chargeLostT_ = charging_ ? 0 : chargeLostT_ + dt;
      if (chargeLostT_ > 2.0f) { mode_ = Mode::STANDBY; modeT_ = 0; armT_ = 0; }
      break;

    case Mode::BALANCE:
    case Mode::DOCK: {
      float lean = 0;
      if (mode_ == Mode::DOCK) {
        DockCmd dc = dock_.step(irL_.n(), irR_.n(), charging_, dt);
        if (dock_.state() == DockState::DONE) {
          mode_ = Mode::CHARGING;
          modeT_ = 0;
          chargeLostT_ = 0;
          break;
        }
        if (dock_.state() == DockState::FAIL) {
          enterBalance();
        } else {
          vCmd_ = dc.v;
          rCmd_ = dc.yawRate;
          lean = dc.leanRad;
        }
      } else {
        if (pendingBack_ && modeT_ > 1.0f && fabsf(th) < 3 * DEG) {
          pendingBack_ = false;
          startStep(Cmd::BACK);
        }
        const bool idle = !linActive_ && !rotActive_ && fabsf(vCmd_) < 1e-3f && fabsf(rCmd_) < 1e-3f;
        const bool seen = irL_.n() >= cfg::DOCK_MIN_COUNT || irR_.n() >= cfg::DOCK_MIN_COUNT;
        // Butunlesik gorevde hedefte 5 s bekleme bozulmasin diye eldiven uzun sure bos kalmali; sarj testinde 1 s yeter
        const float idleNeed = task_ == Task::INTEGRATED ? cfg::DOCK_IDLE_S : 1.0f;
        // Sarj gorevinde robot istasyon bolgesine konur: isareti gormese de 1 s sonra aramaya baslar
        const bool look = seen || task_ == Task::CHARGING;
        undockHoldT_ = fmaxf(0.0f, undockHoldT_ - dt);
        beaconT_ = (dockAllowed() && idle && gloveIdleT_ > idleNeed && look && !pendingBack_ && undockHoldT_ <= 0)
                       ? beaconT_ + dt
                       : 0;
        if (beaconT_ > cfg::DOCK_TRIGGER_S) {
          mode_ = Mode::DOCK;
          modeT_ = 0;
          dock_.start();
        }
      }
      // --- referans uretimi ---
      if (linActive_) {
        lin_.step(dt);
        xRaw_ = xStart_ + lin_.pos();
        if (!lin_.active()) linActive_ = false;
      } else {
        vRaw_ += clampf(vCmd_ - vRaw_, -cfg::STEP_ACCEL * dt, cfg::STEP_ACCEL * dt);
        xRaw_ += vRaw_ * dt;
        xRaw_ = clampf(xRaw_, x_ - 0.15f, x_ + 0.15f);  // takilirsa referans kacmasin
      }
      {
        // referans bicimlendirme: kritik sonumlu 2. derece suzgec. Ivme referansi surekli olur,
        // egim on-beslemesi basamak yapmaz; model hatasinda durmadaki salinim azalir.
        const float w = cfg::REF_SHAPING_W;
        aRef_ = w * w * (xRaw_ - xRef_) - 2 * w * vRef_;
        vRef_ += aRef_ * dt;
        xRef_ += vRef_ * dt;
      }
      if (rotActive_) {
        rot_.step(dt);
        psiRef_ = psiStart_ + rot_.pos();
        rRef_ = rot_.vel();
        if (!rot_.active()) { rotActive_ = false; rRef_ = 0; }
      } else {
        const float dr = clampf(rCmd_ - rRef_, -cfg::TURN_ACCEL_DEG * DEG * dt, cfg::TURN_ACCEL_DEG * DEG * dt);
        rRef_ += dr;
        psiRef_ += rRef_ * dt;
      }
      out = balanceControl(in.vbat, lean);
      break;
    }
  }
  updateMotionState();
  return out;
}

int RobotLogic::telemetryJson(char* buf, int len, uint32_t seq) const {
  return snprintf(buf, len,
                  "{\"seq\":%lu,\"t\":%lu,\"bat\":%d,\"v\":%.2f,\"tilt\":%.1f,\"chg\":%d,\"mode\":\"%s\","
                  "\"mot\":\"%s\",\"dir\":\"%s\",\"task\":\"%s\",\"dock\":\"%s\",\"x\":%.3f,\"yaw\":%.0f,"
```

```
                  "\"glove\":\"%s\",\"ir\":[%d,%d]}",
                  (unsigned long)seq, (unsigned long)(t_ * 1000.0f), batteryPct(), batteryV(), thetaDeg(),
                  charging_ ? 1 : 0, modeName(mode_), motionName(motion_), dir_, taskName(task_),
                  dockName(dock_.state()), x_, psiDeg(), gloveAlive_ ? cmdName(gloveCmd_) : "yok", irL_.n(),
                  irR_.n());
}

}  // namespace cdtp
```

common/cdtp_core/src/imu_filter.h 66 satır

```
// MPU6050 eksen eslemesi + tumleyen filtre.
#pragma once
#include "cdtp_config.h"
#include "core_math.h"

namespace cdtp {

// Ham IMU eksenlerini robot eksenlerine esleme. idx: 0=X 1=Y 2=Z, sign: +1/-1.
// Robot ekseni: ileri (on), yukari (govde boyunca), yunuslama hizi (one egilme +), sapma hizi (sola donus +).
// Varsayilan: MPU6050 X ileri, Z yukari (Y sola). Bu montajda one dusus jiroskop Y'de POZITIF okunur
// (sag el kurali; MuJoCo dijital ikiziyle dogrulandi).
struct AxisMap {
  int fwdIdx = 0;   int fwdSign = 1;
  int upIdx = 2;    int upSign = 1;
  int pitchIdx = 1; int pitchSign = 1;
  int yawIdx = 2;   int yawSign = 1;
};

struct ImuSample {
  float acc[3];   // m/s^2 (ham eksen)
  float gyro[3];  // rad/s (ham eksen, bias cikarilmis)
};

class TiltFilter {
 public:
  void setOffset(float offsetRad) { offset_ = offsetRad; }
  float offset() const { return offset_; }

  void reset(const ImuSample& s, const AxisMap& m) {
    theta_ = accAngle(s, m);
    initialized_ = true;
  }

  // Donus: govde egimi (rad), one egik +.
  void update(const ImuSample& s, const AxisMap& m, float dt) {
    pitchRate_ = m.pitchSign * s.gyro[m.pitchIdx];
    yawRate_ = m.yawSign * s.gyro[m.yawIdx];
    float thAcc = accAngle(s, m);
    if (!initialized_) {
      theta_ = thAcc;
      initialized_ = true;
    }
    // ivme olcer, hizlanma sirasinda yanilir; buyuklugu 1 g'den cok sapinca ona daha az guven
    float an = sqrtf(s.acc[0] * s.acc[0] + s.acc[1] * s.acc[1] + s.acc[2] * s.acc[2]);
    float trust = fabsf(an - 9.81f) < 2.0f ? (1.0f - cfg::COMP_ALPHA) : 0.002f;
    theta_ = (1.0f - trust) * (theta_ + pitchRate_ * dt) + trust * thAcc;
  }

  float theta() const { return theta_ - offset_; }
  float thetaRaw() const { return theta_; }
  float pitchRate() const { return pitchRate_; }
  float yawRate() const { return yawRate_; }

 private:
  static float accAngle(const ImuSample& s, const AxisMap& m) {
    float fwd = m.fwdSign * s.acc[m.fwdIdx];
    float up = m.upSign * s.acc[m.upIdx];
    // one egilince yercekiminin govde-ileri bileseni negatif okunur
    return atan2f(-fwd, up);
  }
  float theta_ = 0, pitchRate_ = 0, yawRate_ = 0, offset_ = 0;
  bool initialized_ = false;
};

}  // namespace cdtp
```

common/cdtp_core/src/dock_fsm.h 112 satır

```
// Otonom yanasma durum makinesi. Istasyonda yazilim yok; robot iki IR gozunun sayimiyla yon bulur.
#pragma once
#include "cdtp_config.h"
#include "core_math.h"

namespace cdtp {

enum class DockState { IDLE, SEARCH, ALIGN, APPROACH, CONTACT, DONE, FAIL };

inline const char* dockName(DockState s) {
```

```
  switch (s) {
    case DockState::SEARCH: return "ARA";
    case DockState::ALIGN: return "HIZALAN";
    case DockState::APPROACH: return "YAKLAS";
    case DockState::CONTACT: return "YERLES";
    case DockState::DONE: return "SARJ";
    case DockState::FAIL: return "BASARISIZ";
    default: return "-";
  }
}

struct DockCmd {
  float v = 0;          // m/s
  float yawRate = 0;    // rad/s, sola +
  float leanRad = 0;    // ek egim hedefi
  bool motorsOff = false;
};

class DockFsm {
 public:
  void start() {
    state_ = DockState::SEARCH;
    t_ = 0;
    stateT_ = 0;
    alignedT_ = 0;
    lostT_ = 0;
    chargeT_ = 0;
  }
  void abort() { state_ = DockState::IDLE; }
  DockState state() const { return state_; }
  bool active() const { return state_ != DockState::IDLE && state_ != DockState::DONE && state_ != DockState::FAIL; }

  DockCmd step(int nL, int nR, bool charge, float dt) {
    DockCmd c;
    t_ += dt;
    stateT_ += dt;
    const bool seeL = nL >= cfg::DOCK_MIN_COUNT, seeR = nR >= cfg::DOCK_MIN_COUNT;
    const float sum = (float)(nL + nR);
    const float e = (seeL || seeR) ? (nL - nR) / fmaxf(sum, 1.0f) : 0.0f;
    if (seeL || seeR) lastSide_ = e >= 0 ? 1.0f : -1.0f;
    chargeT_ = charge ? chargeT_ + dt : 0;
    const float kAlign = cfg::DOCK_ALIGN_K_DEG * DEG;

    if (active() && t_ > cfg::DOCK_TIMEOUT_S) go(DockState::FAIL);

    switch (state_) {
      case DockState::SEARCH:
        c.yawRate = lastSide_ * cfg::DOCK_SEARCH_YAW_DEG * DEG;
        if (seeL || seeR) go(DockState::ALIGN);
        else if (stateT_ > 15.0f) go(DockState::FAIL);  // ~1,2 tur dondu, isaret yok
        break;
      case DockState::ALIGN:
        c.yawRate = kAlign * e;
        alignedT_ = (seeL && seeR && fabsf(e) < cfg::DOCK_ALIGN_TOL) ? alignedT_ + dt : 0;
        lostT_ = (seeL || seeR) ? 0 : lostT_ + dt;
        if (alignedT_ > 0.3f) go(DockState::APPROACH);
        else if (lostT_ > cfg::DOCK_LOST_S) go(DockState::SEARCH);
        break;
      case DockState::APPROACH:
        if (seeL && seeR) {
          c.v = cfg::DOCK_APPROACH_V;
          c.yawRate = kAlign * e;
        } else if (seeL || seeR) {
          c.v = 0.4f * cfg::DOCK_APPROACH_V;  // tek goz: yavasla, don
          c.yawRate = kAlign * e;
        } else {
          c.v = 0.5f * cfg::DOCK_APPROACH_V;  // cok yakinda gozler isareti kaybedebilir: duz devam
        }
        lostT_ = (seeL || seeR) ? 0 : lostT_ + dt;
        if (chargeT_ >= cfg::CHARGE_CONFIRM_S) go(DockState::CONTACT);
        else if (lostT_ > cfg::DOCK_LOST_S) go(DockState::SEARCH);
        break;
      case DockState::CONTACT: {
        // sirta dogru yavasca egil, sonra motorlari kes
        float k = clampf(stateT_ / 0.3f, 0, 1);
        c.leanRad = k * cfg::DOCK_LEAN_DEG * DEG;
        if (!charge && stateT_ > 0.05f) go(DockState::APPROACH);
        else if (stateT_ > 0.4f) go(DockState::DONE);
        break;
      }
      case DockState::DONE:
        c.motorsOff = true;
        break;
      default:
        break;
    }
    return c;
  }

 private:
  void go(DockState s) {
    state_ = s;
    stateT_ = 0;
    alignedT_ = 0;
```

```
    lostT_ = 0;
  }
  DockState state_ = DockState::IDLE;
  float t_ = 0, stateT_ = 0, alignedT_ = 0, lostT_ = 0, chargeT_ = 0, lastSide_ = 1;
};

}  // namespace cdtp
```

common/cdtp_core/src/ir_frame.h 40 satır

```
// IR gozu cerceve sayaci.
// Istasyon 45 ms boyunca ~1 kHz patlama yollar, 33 ms susar. Donanim sayaci (PCNT) dusen kenarlari sayar.
// Bu sinif sayaci periyodik okur; 10 ms'den uzun sessizlik = cerceve sonu, o cercevedeki darbe sayisi n.
#pragma once
#include <stdint.h>

namespace cdtp {

class IrFrameCounter {
 public:
  // count: PCNT'nin kumulatif degeri, t: saniye
  void sample(int32_t count, float t) {
    if (count != lastCount_) {
      if (!inFrame_) {
        inFrame_ = true;
        frameStart_ = lastCount_;
      }
      lastCount_ = count;
      lastEdgeT_ = t;
    } else if (inFrame_ && t - lastEdgeT_ > GAP_S) {
      inFrame_ = false;
      n_ = count - frameStart_;
      frameT_ = t;
    }
    if (t - frameT_ > STALE_S) n_ = 0;  // isaret kayboldu
  }
  int n() const { return n_; }
  float lastFrameT() const { return frameT_; }

 private:
  static constexpr float GAP_S = 0.018f;  // isaret 33 ms susar; zayif isarette tek tuk kacan patlama cerceveyi bolmesin
  static constexpr float STALE_S = 0.25f;
  int32_t lastCount_ = 0, frameStart_ = 0;
  float lastEdgeT_ = -1, frameT_ = -1;
  bool inFrame_ = false;
  int n_ = 0;
};

}  // namespace cdtp
```

common/cdtp_core/src/core_math.h 65 satır

```
#pragma once
#include <math.h>

namespace cdtp {

constexpr float PI_F = 3.14159265358979f;
constexpr float DEG = PI_F / 180.0f;

inline float clampf(float x, float lo, float hi) { return x < lo ? lo : (x > hi ? hi : x); }
inline float signf(float x) { return x > 0 ? 1.0f : (x < 0 ? -1.0f : 0.0f); }
inline float wrapPi(float a) {
  while (a > PI_F) a -= 2 * PI_F;
  while (a < -PI_F) a += 2 * PI_F;
  return a;
}

// Tek eksenli trapez hiz profili: 0'dan hedef mesafeye, dur.
class Trapezoid {
 public:
  void start(float dist, float vmax, float amax) {
    dist_ = fabsf(dist);
    dir_ = dist >= 0 ? 1.0f : -1.0f;
    vmax_ = vmax;
    amax_ = amax;
    t_ = 0;
    pos_ = 0;
    vel_ = 0;
    active_ = dist_ > 1e-6f;
    stopping_ = false;
  }
  // Yumusak durdurma: kalan yolu fren mesafesine indir.
  void stopSoft() {
    if (!active_) return;
    float brake = vel_ * vel_ / (2 * amax_);
    dist_ = pos_ + brake;
    stopping_ = true;
  }
```

```
  void step(float dt) {
    if (!active_) return;
    float remaining = dist_ - pos_;
    float vBrake = sqrtf(fmaxf(0.0f, 2 * amax_ * remaining));
    float vTarget = fminf(vmax_, vBrake);
    float v0 = vel_;
    if (vel_ < vTarget) vel_ = fminf(vTarget, vel_ + amax_ * dt);
    else vel_ = fmaxf(vTarget, vel_ - amax_ * dt);
    acc_ = (vel_ - v0) / dt;
    pos_ += vel_ * dt;
    if (remaining <= 1e-4f || (vel_ <= 1e-4f && remaining < 2e-3f)) {
      pos_ = dist_;
      vel_ = 0;
      active_ = false;
    }
  }
  bool active() const { return active_; }
  float pos() const { return dir_ * pos_; }
  float vel() const { return dir_ * vel_; }
  float acc() const { return dir_ * acc_; }

 private:
  float dist_ = 0, dir_ = 1, vmax_ = 0, amax_ = 1, t_ = 0, pos_ = 0, vel_ = 0, acc_ = 0;
  bool active_ = false, stopping_ = false;
};

}  // namespace cdtp
```

common/cdtp_core/src/battery.h 33 satır

```
// 3S Li-ion: gerilimden yaklasik doluluk yuzdesi.
#pragma once
#include "core_math.h"

namespace cdtp {

inline float socFromCellV(float v) {
  // dinlenme egrisi (18650, yaklasik)
  static const float V[] = {3.30f, 3.50f, 3.60f, 3.70f, 3.75f, 3.80f, 3.90f, 4.00f, 4.10f, 4.20f};
  static const float S[] = {0, 5, 10, 25, 40, 50, 65, 78, 90, 100};
  if (v <= V[0]) return 0;
  for (int i = 1; i < 10; ++i)
    if (v <= V[i]) return S[i - 1] + (S[i] - S[i - 1]) * (v - V[i - 1]) / (V[i] - V[i - 1]);
  return 100;
}

class BatteryMonitor {
 public:
  void update(float vbat, float dt) {
    if (v_ <= 0) v_ = vbat;
    float a = dt / (TAU + dt);
    v_ += a * (vbat - v_);
  }
  float volts() const { return v_; }
  int percent() const { return (int)(socFromCellV(v_ / 3.0f) + 0.5f); }

 private:
  static constexpr float TAU = 3.0f;  // motor akimindaki dalgalanmayi suzer
  float v_ = 0;
};

}  // namespace cdtp
```

common/cdtp_core/src/protocol.h 65 satır

```
// Eldiven -> robot ESP-NOW paketi ve ortak tipler.
#pragma once
#include <stdint.h>
#include <string.h>

namespace cdtp {

enum class Cmd : uint8_t { NONE = 0, FWD = 1, BACK = 2, LEFT = 3, RIGHT = 4, STOP = 5 };

inline const char* cmdName(Cmd c) {
  switch (c) {
    case Cmd::FWD: return "ileri";
    case Cmd::BACK: return "geri";
    case Cmd::LEFT: return "sol";
    case Cmd::RIGHT: return "sag";
    case Cmd::STOP: return "dur";
    default: return "yok";
  }
}

// Ayni sinifta baska takimlarin eldiveni de ESP-NOW yayini yapar.
// Robot yalniz bu anahtari tasiyan paketi kabul eder. Takim icin degistirin.
constexpr uint32_t TEAM_KEY = 0xCD7B0001u;
```

```
constexpr uint8_t PROTO_VERSION = 1;

#pragma pack(push, 1)
struct GlovePacket {
  uint32_t key;       // TEAM_KEY
  uint8_t version;    // PROTO_VERSION
  uint8_t cmd;        // Cmd
  uint16_t seq;       // her pakette +1
  int8_t pitchDeg;    // hata ayiklama icin
  int8_t rollDeg;
  uint8_t batPct;     // eldiven pili
  uint8_t crc;        // onceki baytlarin CRC-8'i
};
#pragma pack(pop)
static_assert(sizeof(GlovePacket) == 12, "paket boyutu degismemeli");

inline uint8_t crc8(const uint8_t* d, size_t n) {
  uint8_t c = 0;
  for (size_t i = 0; i < n; ++i) {
    c ^= d[i];
    for (int b = 0; b < 8; ++b) c = (c & 0x80) ? (uint8_t)((c << 1) ^ 0x07) : (uint8_t)(c << 1);
  }
  return c;
}

inline void sealPacket(GlovePacket& p) {
  p.key = TEAM_KEY;
  p.version = PROTO_VERSION;
  p.crc = crc8(reinterpret_cast<const uint8_t*>(&p), sizeof(p) - 1);
}

inline bool checkPacket(const uint8_t* data, int len, GlovePacket& out) {
  if (len != (int)sizeof(GlovePacket)) return false;
  memcpy(&out, data, sizeof(out));
  if (out.key != TEAM_KEY || out.version != PROTO_VERSION) return false;
  if (out.crc != crc8(data, sizeof(out) - 1)) return false;
  return out.cmd <= (uint8_t)Cmd::STOP;
}

}  // namespace cdtp
```

common/cdtp_core/src/gesture.h 49 satır

```
// Eldiven: el egimini bes komuta ceviren siniflandirici (histerezisli).
// Pitch: el one egik +. Roll: sag tarafa yatik +. Aci olculeri acilistaki notr durusa goredir.
#pragma once
#include "cdtp_config.h"
#include "core_math.h"
#include "protocol.h"

namespace cdtp {

class GestureClassifier {
 public:
  Cmd update(float pitchDeg, float rollDeg, float dt) {
    Cmd cand = candidate(pitchDeg, rollDeg);
    if (cand == current_) {
      pendingT_ = 0;
      pending_ = cand;
      return current_;
    }
    if (cand != pending_) {
      pending_ = cand;
      pendingT_ = 0;
    }
    pendingT_ += dt;
    float need = (cand == Cmd::STOP) ? cfg::GEST_NEUTRAL_S : cfg::GEST_HOLD_S;
    if (cand != Cmd::NONE && pendingT_ >= need) current_ = cand;
    return current_;
  }
  Cmd current() const { return current_; }

 private:
  Cmd candidate(float p, float r) const {
    float ap = fabsf(p), ar = fabsf(r);
    // Suren komut, cikis esiginin (15 deg) altina inene kadar korunur
    if (current_ == Cmd::FWD && p > cfg::GEST_EXIT_DEG && ap >= ar) return Cmd::FWD;
    if (current_ == Cmd::BACK && p < -cfg::GEST_EXIT_DEG && ap >= ar) return Cmd::BACK;
    if (current_ == Cmd::RIGHT && r > cfg::GEST_EXIT_DEG && ar >= ap) return Cmd::RIGHT;
    if (current_ == Cmd::LEFT && r < -cfg::GEST_EXIT_DEG && ar >= ap) return Cmd::LEFT;
    if (ap < cfg::GEST_NEUTRAL_DEG && ar < cfg::GEST_NEUTRAL_DEG) return Cmd::STOP;
    // Yeni komut icin giris esigi (25 deg) ve baskin eksen 1.5 kat buyuk olmali (capraz el karismasin)
    if (ap >= cfg::GEST_ENTER_DEG && ap > 1.5f * ar) return p > 0 ? Cmd::FWD : Cmd::BACK;
    if (ar >= cfg::GEST_ENTER_DEG && ar > 1.5f * ap) return r > 0 ? Cmd::RIGHT : Cmd::LEFT;
    return Cmd::NONE;  // belirsiz bolge: mevcut komut degismez
  }
  Cmd current_ = Cmd::STOP, pending_ = Cmd::STOP;
  float pendingT_ = 0;
};

}  // namespace cdtp
```

Tasarım araçları

tools/mass_budget.py 29 satır

```
"""Robot govdesinin kutle butcesi: kutle merkezi yuksekligi ve ataleti (teker ekseni z=0).
Parca agirliklari tahmindir (OLC): parca gelince terazide tartip guncelleyin."""
# (ad, kutle kg, z m (eksenden yukseklik), x m (one +), boyut z m (atalet icin))
PARTS = [
    ("2x JGB37-520 motor + braket", 0.380, 0.000, 0.0, 0.037),
    ("Kat 1 pleksi 150x110x3",      0.060, 0.024, 0.0, 0.003),
    ("2x BTS7960",                  0.130, 0.045, 0.0, 0.035),
    ("LM2596 + sigorta + 2 WAGO",   0.060, 0.035, 0.0, 0.020),
    ("MPU6050 (kat 1 ustu, ortada)", 0.005, 0.030, 0.0, 0.003),
    ("Kat 2 pleksi",                0.060, 0.104, 0.0, 0.003),
    ("3x18650 + yuva + BMS",        0.190, 0.120, 0.0, 0.022),
    ("Ana kart + ESP32",            0.045, 0.115, 0.0, 0.015),
    ("IR gozleri + V-yuva + pimler", 0.035, 0.090, 0.05, 0.060),
    ("Kat 3 pleksi",                0.060, 0.184, 0.0, 0.003),
    ("Acil stop",                   0.070, 0.205, 0.0, 0.040),
    ("Kutu yuvasi",                 0.030, 0.195, 0.0, 0.020),
    ("16x distans + vida + kablo",  0.120, 0.100, 0.0, 0.160),
]
BOX = ("Ilac kutusu", 0.060, 0.215)
m = sum(p[1] for p in PARTS)
l = sum(p[1] * p[2] for p in PARTS) / m
I = sum(p[1] * ((p[2] - l) ** 2 + (p[4] ** 2 + 0.11 ** 2) / 12) for p in PARTS)
if __name__ == "__main__":
    for p in PARTS:
        print(f"{p[0]:32s} {p[1]*1000:5.0f} g  z={p[2]*1000:4.0f} mm")
    print(f"govde (kutusuz): {m:.3f} kg, KM eksenin {l*1000:.0f} mm ustunde, I={I:.4f} kg m^2")
    mt = m + BOX[1]; lt = (m * l + BOX[1] * BOX[2]) / mt
    print(f"kutulu: {mt:.3f} kg, KM {lt*1000:.0f} mm")
```

tools/lqr_design.py 173 satır

```
"""Denge robotu modeli ve LQR kazanc tasarimi.

Calistir:  python tools/lqr_design.py
Uretir:    common/cdtp_core/src/gains.h   (firmware + simulasyon kazanclari)
           sim/plant_nominal.h            (simulasyonun nominal fiziksel modeli)

Parca gelince "OLC" yazan degerleri tartip olcup bu dosyada guncelleyin, yeniden calistirin.
"""
import math
import pathlib

import numpy as np
from scipy.linalg import expm, solve_discrete_are

import mass_budget

ROOT = pathlib.Path(__file__).resolve().parents[1]

# ---- Fiziksel model (OLC) ----
P = dict(
    # govde degerleri tools/mass_budget.py'deki parca listesinden (kutu haric; kutuyu simulasyon ekler)
    m_body=round(mass_budget.m, 3),   # kg, tekerler haric her sey (motor govdeleri dahil)
    l_com=round(mass_budget.l, 4),    # m, teker ekseninden govde kutle merkezine
    I_body=round(mass_budget.I, 5),   # kg m^2, kutle merkezi etrafinda yunuslama ataleti
    m_wheel=0.045,      # kg, tek teker + gobek
    r=0.0325,           # m
    J_rotor=5.0e-7,     # kg m^2, motor rotoru (disli oncesi)
    gear=19.0,          # JGB37-520 530 RPM ~ 1:19
    Ke=0.216,           # V s/rad (cikis mili) = Kt (N m/A)
    R=4.5,              # ohm, sargi + surucu
    b_visc=0.002,       # N m s/rad, disli viskoz surtunme (motor basina)
    tau_coulomb=0.02,   # N m, disli kuru surtunme (motor basina)
    track=0.19,         # m, teker merkezleri arasi (sasi cizimi; OLC)
    I_yaw=0.012,        # kg m^2
    vbat=11.1,
)

DT = 1.0 / 200.0


def linear_model(p):
    """Durum s = [x, v, theta, omega], giris u = her motora ayni gerilim (V)."""
    r, mb, l = p["r"], p["m_body"], p["l_com"]
    Iw = 2 * 0.5 * p["m_wheel"] * r**2
    Jm = 2 * p["J_rotor"] * p["gear"] ** 2
    a = 2 * p["m_wheel"] * r**2 + Iw + mb * r**2 + Jm
    b = mb * r * l - Jm
    c = mb * l**2 + p["I_body"] + Jm
    M = np.array([[a, b], [b, c]])
    Minv = np.linalg.inv(M)
    k = 2 * p["Ke"] / p["R"]                      # iki motor, tork/gerilim
    d = 2 * (p["Ke"] ** 2 / p["R"] + p["b_visc"])  # iki motor, tork/(bagil hiz)
    g = 9.81
    # q = [phi, theta]; tork tau = k u - d (phi_dot - theta_dot), tekerlege +tau, govdeye -tau
    # M qdd = [tau, -tau + mb g l theta]
```

```
    # phi = x / r
    A = np.zeros((4, 4))
    B = np.zeros((4, 1))
    # qdd = Minv @ (F_u * u + F_phid * phid + F_thd * thd + F_th * th)
    F_u = np.array([k, -k])
    F_phid = np.array([-d, d])
    F_thd = np.array([d, -d])
    F_th = np.array([0.0, mb * g * l])
    acc_u = Minv @ F_u
    acc_phid = Minv @ F_phid
    acc_thd = Minv @ F_thd
    acc_th = Minv @ F_th
    # x' = v ; v' = r phidd ; phid = v / r
    A[0, 1] = 1
    A[1, 1] = r * acc_phid[0] / r
    A[1, 2] = r * acc_th[0]
    A[1, 3] = r * acc_thd[0]
    B[1, 0] = r * acc_u[0]
    A[2, 3] = 1
    A[3, 1] = acc_phid[1] / r
    A[3, 2] = acc_th[1]
    A[3, 3] = acc_thd[1]
    B[3, 0] = acc_u[1]
    return A, B


def discretize(A, B, dt):
    n, m = A.shape[0], B.shape[1]
    big = np.zeros((n + m, n + m))
    big[:n, :n] = A
    big[:n, n:] = B
    e = expm(big * dt)
    return e[:n, :n], e[:n, n:]


def dlqr(Ad, Bd, Q, R):
    X = solve_discrete_are(Ad, Bd, Q, R)
    return np.linalg.solve(Bd.T @ X @ Bd + R, Bd.T @ X @ Ad)


# Agirliklar: izin verilen en buyuk hata (Bryson kurali, agirlik = 1/hata^2)
W = dict(x=0.06, v=0.08, th_deg=4.0, om_dps=60.0, u_volt=5.0)  # sim ile secildi (sim/sim_main.cpp SAG testi)


def design(p, w=None):
    w = w or W
    A, B = linear_model(p)
    Ad, Bd = discretize(A, B, DT)
    Q = np.diag([1 / w["x"] ** 2, 1 / w["v"] ** 2, 1 / math.radians(w["th_deg"]) ** 2, 1 / math.radians(w["om_dps"]) ** 2])
    R = np.array([[1 / w["u_volt"] ** 2]])
    K = dlqr(Ad, Bd, Q, R)
    cl = np.linalg.eigvals(Ad - Bd @ K)
    return A, B, K.flatten(), cl


def yaw_gains(p):
    # yaw: I psi_dd = (track/2r) * 2*(Ke/R) * u_d - damping ; PD ile ~6 rad/s bant
    r, tr = p["r"], p["track"]
    kyaw = (tr / (2 * r)) * 2 * p["Ke"] / p["R"]
    wn, zeta = 8.0, 0.9
    kp = p["I_yaw"] * wn**2 / kyaw
    kd = p["I_yaw"] * 2 * zeta * wn / kyaw
    return kp, kd


def main():
    import sys
    for arg in sys.argv[1:]:  # ornek: x=0.05 v=0.1
        k, v = arg.split("=")
        W[k] = float(v)
    A, B, K, cl = design(P)
    print("agirliklar:", W)
    kp_y, kd_y = yaw_gains(P)
    print("A =\n", np.round(A, 3))
    print("B =", np.round(B.flatten(), 3))
    print("acik dongu kutuplar:", np.round(np.linalg.eigvals(A), 2))
    print("K [x, v, theta, omega] =", np.round(K, 3))
    print("kapali dongu |z|:", np.round(np.abs(cl), 4))
    print(f"yaw Kp={kp_y:.3f} V/rad  Kd={kd_y:.3f} V/(rad/s)")

    gains = f"""// OTOMATIK URETILDI: tools/lqr_design.py. Elle degistirmeyin; modeli guncelleyip yeniden calistirin.
// Durum: [x (m), v (m/s), theta (rad), omega (rad/s)], cikis: her motora gerilim (V).
// u = -K (s - s_ref)
#pragma once
namespace cdtp {{
namespace gains {{
constexpr float K_X = {K[0]:.5f}f;
constexpr float K_V = {K[1]:.5f}f;
constexpr float K_TH = {K[2]:.5f}f;
constexpr float K_OM = {K[3]:.5f}f;
constexpr float YAW_KP = {kp_y:.5f}f;
constexpr float YAW_KD = {kd_y:.5f}f;
// a_ref -> egim on-besleme (rad / (m/s^2))
constexpr float TH_PER_ACC = {th_per_acc(P):.5f}f;
```

```
}}  // namespace gains
}}  // namespace cdtp
"""
    (ROOT / "common/cdtp_core/src/gains.h").write_text(gains, encoding="utf-8")

    plant = "// OTOMATIK URETILDI: tools/lqr_design.py\n#pragma once\nstruct PlantNominal {\n"
    for k, v in P.items():
        plant += f"  static constexpr double {k} = {v!r};\n"
    plant += "};\n"
    (ROOT / "sim/plant_nominal.h").write_text(plant, encoding="utf-8")
    print("yazildi: gains.h, plant_nominal.h")


def th_per_acc(p):
    # sabit ivme a icin dengedeki egim: sin(th) = a (mb l + J/r) / (mb g l)
    r, mb, l = p["r"], p["m_body"], p["l_com"]
    Iw = 2 * 0.5 * p["m_wheel"] * r**2
    J = 2 * p["m_wheel"] * r**2 + Iw + mb * r**2
    return (mb * l + J / r) / (mb * 9.81 * l)


if __name__ == "__main__":
    main()
```

tools/check_consistency.py 130 satır

```
"""Cizilen/yazilan belgeler ile kodun tutarlilik denetimi.

Karsilastirir:
  1. firmware/robot/src/pins.h  <->  takim sitesinin pin semasi (tools/ref/site_wiring.js)
  2. pins.h                     <->  docs/robot_src.html pin tablosu ve ana kart header siralari
  3. cdtp_config.h / gains.h    <->  belgelerde yazan sayilar
  4. links.h BLE UUID'leri      <->  docs/BLE_PROTOKOL.md
  5. bolucu oranlari            <->  pins.h DIVIDER ve belge
Cikti: docs/tutarlilik.md (+ ekrana ozet). Hata varsa cikis kodu 1.
"""
import pathlib
import re
import sys

sys.stdout.reconfigure(encoding="utf-8")

ROOT = pathlib.Path(__file__).resolve().parents[1]
rd = lambda p: (ROOT / p).read_text(encoding="utf-8")

rows = []  # (konu, beklenen, bulunan, durum)


def check(topic, expected, found, ok=None):
    if ok is None:
        ok = str(expected) == str(found)
    rows.append((topic, str(expected), str(found), "uygun" if ok else "UYUMSUZ"))


pins_h = rd("firmware/robot/src/pins.h")
pins = {m[0]: int(m[1]) for m in re.findall(r"constexpr int (PIN_\w+) = (\d+);", pins_h)}

# ---- 1. site pin semasi ----
site = rd("tools/ref/site_wiring.js")
robot_block = site[site.index("export const ROBOT_PINS"):site.index("export const GLOVE_PINS")]
site_map = {}
for name, gpio in re.findall(r"\['([^']+)', 'GPIO(\d+)", robot_block):
    site_map.setdefault(name, []).append(int(gpio))
expect_site = {
    "SDA": ("PIN_SDA",), "SCL": ("PIN_SCL",), "INT": ("PIN_IMU_INT",),
    "OUT": ("PIN_IR_L", "PIN_IR_R"),
    "RPWM": ("PIN_L_RPWM", "PIN_R_RPWM"), "LPWM": ("PIN_L_LPWM", "PIN_R_LPWM"),
    "R_EN + L_EN": ("PIN_MOTOR_EN", "PIN_MOTOR_EN"),
    "C1 (A)": ("PIN_ENC_L_A", "PIN_ENC_R_A"), "C2 (B)": ("PIN_ENC_L_B", "PIN_ENC_R_B"),
}
for sig, names in expect_site.items():
    got = site_map.get(sig, [])
    want = [pins[n] for n in names]
    check(f"Site semasi: {sig}", want, got, sorted(want) == sorted(got))
meas = dict(re.findall(r"t: '(Pil gerilimi|Şarj algılama)'.*?'GPIO(\d+)'", robot_block))
check("Site semasi: pil olcumu", pins["PIN_VBAT"], meas.get("Pil gerilimi"))
check("Site semasi: sarj algilama", pins["PIN_VCHG"], meas.get("Şarj algılama"))

# ---- 2. robot_src.html pin tablosu ve header siralari ----
doc = rd("docs/robot_src.html")
sig_sec = doc[doc.index('id="sinyal"'):doc.index('id="anakart"')]
tbl = {}
for pinsTxt, task in re.findall(r'<td class="num">([\d ,/]+)</td><td>([^<]+)</td>', sig_sec):
    tbl[task.strip()] = [int(x) for x in re.findall(r"\d+", pinsTxt)]
doc_expect = {
    "MPU6050 INT": [pins["PIN_IMU_INT"]],
    "I2C SDA / SCL": [pins["PIN_SDA"], pins["PIN_SCL"]],
    "Sol / sağ RPWM, LPWM": [pins["PIN_L_RPWM"], pins["PIN_L_LPWM"], pins["PIN_R_RPWM"], pins["PIN_R_LPWM"]],
    "EN (iki kart)": [pins["PIN_MOTOR_EN"]],
    "Enkoder A, B": [pins["PIN_ENC_L_A"], pins["PIN_ENC_L_B"], pins["PIN_ENC_R_A"], pins["PIN_ENC_R_B"]],
    "IR göz sol / sağ": [pins["PIN_IR_L"], pins["PIN_IR_R"]],
```

```
    "Pil / şarj pimi gerilimi": [pins["PIN_VBAT"], pins["PIN_VCHG"]],
    "Acil stop sonrası motor hattı": [pins["PIN_MOTOR_SENSE"]],
    "Karttaki LED": [pins["PIN_LED"]],
}
for task, want in doc_expect.items():
    check(f"Belge pin tablosu: {task}", want, tbl.get(task))

hdr_sec = doc[doc.index('id="anakart"'):doc.index('id="kablo"')]
headers = dict(re.findall(r'<td class="num">(J\d+)</td><td>([^<]+)</td>', hdr_sec))
hdr_expect = {
    "J2": f"3V3, GND, G{pins['PIN_SCL']}, G{pins['PIN_SDA']}, G{pins['PIN_IMU_INT']}",
    "J3": f"G{pins['PIN_L_RPWM']}, G{pins['PIN_L_LPWM']}, G{pins['PIN_MOTOR_EN']}, 3V3, GND",
    "J8": f"G{pins['PIN_R_RPWM']}, G{pins['PIN_R_LPWM']}, G{pins['PIN_MOTOR_EN']}, 3V3, GND",
    "J4": f"3V3, GND, G{pins['PIN_ENC_L_A']}, G{pins['PIN_ENC_L_B']}",
    "J5": f"3V3, GND, G{pins['PIN_ENC_R_A']}, G{pins['PIN_ENC_R_B']}",
}
for j, want in hdr_expect.items():
    check(f"Ana kart {j} pin sirasi", want, headers.get(j, "").strip())
ir = re.search(r'<td class="num">J6 / J7</td><td>3V3, GND, G(\d+) / G(\d+)</td>', hdr_sec)
check("Ana kart J6/J7", [pins["PIN_IR_L"], pins["PIN_IR_R"]], [int(ir[1]), int(ir[2])] if ir else None)

# ---- 3. sayilar ----
cfg = rd("common/cdtp_core/src/cdtp_config.h")
num = lambda name: float(re.search(rf"{name}\s*=\s*([\d.]+)f", cfg)[1])
gains = rd("common/cdtp_core/src/gains.h")
g = {k: float(v) for k, v in re.findall(r"constexpr float (\w+) = (-?[\d.]+)f;", gains)}
plan = rd("docs/plan.html")
check("Iz genisligi (kod <-> belge 190 mm)", 190, round(num("TRACK_WIDTH_M") * 1000), "iz 190" in doc and num("TRACK_WIDTH_M") == 0.19)
check("Enkoder sayim/tur (kod <-> belge 836)", 836, int(num("ENC_COUNTS_PER_REV")), "836" in doc)
check("Dusme acisi 40 deg", 40, int(num("FALL_ANGLE_DEG")), "40°" in doc)
check("Dengeye girme +-3 deg, 0,5 s", "3 / 0.5", f"{num('ARM_ANGLE_DEG'):g} / {num('ARM_HOLD_S'):g}", "±3° içinde 0,5 s" in doc)
check("Eldiven zaman asimi 0,3 s", 0.3, num("GLOVE_TIMEOUT_S"), num("GLOVE_TIMEOUT_S") == 0.3 and "0,3 s paket" in doc)
check("Adim 0,5 m / 90 deg", "0.5 / 90", f"{num('STEP_DIST_M'):g} / {num('TURN_ANGLE_DEG'):g}", "0,5 m" in doc and "90°" in doc)
check("Sarj algilama esigi 9 V", 9, num("CHARGE_DETECT_V"), "9 V" in doc)
check("Butunlesik gorevde bos bekleme 6 s", 6, num("DOCK_IDLE_S"), "6 s" in plan)
fmt = lambda v: f"{abs(v):.1f}".replace(".", ",")
plan_gain = f"u = {fmt(g['K_X'])}·x + {fmt(g['K_V'])}·v + {fmt(g['K_TH'])}·θ + {fmt(g['K_OM'])}·ω"
check("LQR kazanclari (gains.h <-> plan sayfasi)", plan_gain, "plan sayfasinda var" if plan_gain in plan else "YOK", plan_gain in plan)

# ---- 4. BLE ----
links = rd("firmware/robot/src/links.h")
ble_doc = rd("docs/BLE_PROTOKOL.md")
for name in ("CDTP_SVC_UUID", "CDTP_TELEM_UUID", "CDTP_CMD_UUID"):
    u = re.search(rf'#define {name} "([0-9a-f-]+)"', links)[1]
    check(f"BLE {name}", u, "belgede var" if u in ble_doc else "YOK", u in ble_doc)

# ---- 5. bolucu ----
div = re.search(r"DIVIDER = \(([\d.]+)f \+ ([\d.]+)f\) / ([\d.]+)f", pins_h)
check("Pil boluculeri 100k/27k", "100/27", f"{float(div[1]):g}/{float(div[2]):g}", "100k/27k" in doc)
v_top = 12.6 * float(div[2]) / (float(div[1]) + float(div[2]))
check("12,6 V -> ADC gerilimi < 3,1 V", "< 3.1", f"{v_top:.2f}", v_top < 3.1)
v_ms = 12.6 * 33 / 133
check("Motor hatti bolucu 100k/33k -> < 3,3 V", "< 3.3", f"{v_ms:.2f}", v_ms < 3.3 and "100k/33k" in doc)

# ---- rapor ----
bad = [r for r in rows if r[3] != "uygun"]
lines = ["# Belge ve kod tutarlilik denetimi", "", f"{len(rows)} kontrol, {len(bad)} uyumsuz.", "",
         "| Konu | Beklenen | Bulunan | Durum |", "|---|---|---|---|"]
lines += [f"| {a} | {b} | {c} | {d} |" for a, b, c, d in rows]
(ROOT / "docs/tutarlilik.md").write_text("\n".join(lines) + "\n", encoding="utf-8")
for r in rows:
    print(f"[{r[3]:7s}] {r[0]}: beklenen {r[1]} / bulunan {r[2]}")
print(f"\n{len(rows)} kontrol, {len(bad)} uyumsuz")
sys.exit(1 if bad else 0)
```

tools/run_sim.sh 11 satır

```
#!/usr/bin/env bash
# Simulasyonu derler ve tum testleri kosar. Proje kokunden calistirin: bash tools/run_sim.sh
set -e
cd "$(dirname "$0")/.."
mkdir -p sim/results build
g++ -std=gnu++14 -O2 -Wall -Wno-unused-variable \
  -Icommon/cdtp_core/src -Isim \
  sim/sim_main.cpp common/cdtp_core/src/robot_logic.cpp \
  -o build/sim.exe
./build/sim.exe sim/results/
```

tools/plot_sim.py 94 satır

```
"""Simulasyon izlerini cizer: sim/results/*.csv -> docs/img/*.png"""
import csv
import pathlib

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
```

```
ROOT = pathlib.Path(__file__).resolve().parents[1]
RES = ROOT / "sim/results"
OUT = ROOT / "docs/img"
OUT.mkdir(parents=True, exist_ok=True)


def load(name):
    with open(RES / name, encoding="utf-8") as f:
        rows = list(csv.DictReader(f))
    return {k: [float(r[k]) for r in rows] for k in rows[0]}


def style(ax):
    ax.grid(True, alpha=0.3)
    ax.spines[["top", "right"]].set_visible(False)


def motion(fname, title, out):
    d = load(fname)
    fig, ax = plt.subplots(3, 1, figsize=(8, 6.4), sharex=True)
    ax[0].plot(d["t"], d["x"], color="#2a6fdb")
    ax[0].set_ylabel("yol x (m)")
    ax[1].plot(d["t"], d["theta_deg"], color="#d9822b")
    ax[1].set_ylabel("egim (deg)")
    ax[2].plot(d["t"], d["dutyL"], color="#2bb3a3", lw=0.8, label="sol")
    ax[2].plot(d["t"], [-v for v in d["dutyR"]], color="#9b7cf0", lw=0.8, label="sag (ayna)")
    ax[2].set_ylabel("PWM orani")
    ax[2].set_xlabel("zaman (s)")
    ax[2].legend(frameon=False, loc="upper right")
    for a in ax:
        style(a)
    fig.suptitle(title)
    fig.tight_layout()
    fig.savefig(OUT / out, dpi=110)
    plt.close(fig)


def recovery():
    fig, ax = plt.subplots(figsize=(8, 3.2))
    for fn, lab, c in (("c1_toparlanma_arti.csv", "+10 deg", "#d9822b"), ("c1_toparlanma_eksi.csv", "-10 deg", "#2a6fdb")):
        d = load(fn)
        ax.plot(d["t"], d["theta_deg"], label=lab, color=c)
    ax.axhspan(-3, 3, color="#2bb3a3", alpha=0.12, label="+-3 deg")
    ax.set_xlabel("zaman (s)")
    ax.set_ylabel("egim (deg)")
    ax.set_xlim(0.8, 6)
    ax.legend(frameon=False)
    style(ax)
    ax.set_title("C1.4: +-10 deg'den birakilinca toparlanma")
    fig.tight_layout()
    fig.savefig(OUT / "c1_toparlanma.png", dpi=110)
    plt.close(fig)


def dock():
    import math

    fig, ax = plt.subplots(figsize=(6, 5))
    for fn, c in (("c3_yanasma.csv", "#2a6fdb"), ("c5_butunlesik.csv", "#d9822b")):
        d = load(fn)
        ax.plot(d["X"], d["Y"], color=c, label="C3 yanasma" if "c3" in fn else "C5 butunlesik gorev")
        ax.plot(d["X"][0], d["Y"][0], "o", color=c)
    ax.add_patch(plt.Circle((0, 0), 0.02, color="#333"))
    th = [math.radians(a) for a in range(-45, 46)]
    ax.plot([0.3 * math.cos(a) for a in th], [0.3 * math.sin(a) for a in th], "--", color="#999", lw=0.8)
    ax.plot([0, 0.3 * math.cos(th[0])], [0, 0.3 * math.sin(th[0])], "--", color="#999", lw=0.8)
    ax.plot([0, 0.3 * math.cos(th[-1])], [0, 0.3 * math.sin(th[-1])], "--", color="#999", lw=0.8, label="30 cm / 90 deg bolge")
    ax.set_aspect("equal")
    ax.set_xlabel("X (m)")
    ax.set_ylabel("Y (m)")
    ax.legend(frameon=False, fontsize=8, loc="lower left")
    style(ax)
    ax.set_title("Ustten gorunus: istasyon diregi (0,0)")
    fig.tight_layout()
    fig.savefig(OUT / "yanasma_yolu.png", dpi=110)
    plt.close(fig)


if __name__ == "__main__":
    motion("c1_ileri.csv", "C1.2: iki 'ileri' komutu = 1 m, sonra 3 s dur", "c1_ileri.png")
    motion("c2_eldiven.csv", "C2: ileri, geri, sola, saga, dur", "c2_eldiven.png")
    recovery()
    dock()
    print("cizildi:", sorted(p.name for p in OUT.glob("*.png")))
```

C++ simülasyonu

sim/plant.h 154 satır

```
// Iki tekerlekli ters sarkac + DC motor + sensor modeli. Yalniz PC simulasyonu icin.
#pragma once
#include <cmath>
#include <random>

#include "cdtp_config.h"
#include "plant_nominal.h"
#include "robot_logic.h"

struct PlantParams {
  double m_body = PlantNominal::m_body, l_com = PlantNominal::l_com, I_body = PlantNominal::I_body;
  double m_wheel = PlantNominal::m_wheel, r = PlantNominal::r, J_rotor = PlantNominal::J_rotor, gear = PlantNominal::gear;
  double Ke = PlantNominal::Ke, R = PlantNominal::R, b_visc = PlantNominal::b_visc, tau_c = PlantNominal::tau_coulomb;
  double track = PlantNominal::track, I_yaw = PlantNominal::I_yaw;
  double voc = 12.0, r_int = 0.15;   // paket bos gerilim, ic direnc
  double box_mass = 0.06, box_h = 0.215;  // ilac kutusu (eksenden yukseklik)
  double h_imu = 0.05;               // IMU'nun eksenden yuksekligi
  double imu_offset_deg = 0.0;       // IMU montaj hatasi
  double gyro_bias_dps = 0.0;        // kalibrasyon sonrasi kalan bias
  double gyro_noise_dps = 0.05, acc_noise = 0.03;  // rms
  double vib_acc = 0.3;              // motor titresimi (m/s^2 rms, motor gerilimiyle olcekli)
  double backlash_deg = 0.0;         // (kullanilmiyor) disli boslugu
};

class Plant {
 public:
  explicit Plant(const PlantParams& p, unsigned seed = 1) : p_(p), rng_(seed) {
    // kutu govdeye eklenir: toplam kutle ve kutle merkezi
    mb_ = p.m_body + p.box_mass;
    l_ = (p.m_body * p.l_com + p.box_mass * p.box_h) / mb_;
    Ib_ = p.I_body + p.m_body * (p.l_com - l_) * (p.l_com - l_) + p.box_mass * (p.box_h - l_) * (p.box_h - l_);
  }

  // durum
  double x = 0, v = 0, th = 0, om = 0, psi = 0, r = 0;  // r: sapma hizi
  double X = 0, Y = 0;                                  // dunya konumu
  double xl = 0, xr = 0;                                 // teker yollari
  double iL = 0, iR = 0, vbat = 12.0;
  bool lying = false;

  // dt boyunca motor gerilimleriyle ilerle (enable=false: surucu yuksek empedans)
  void step(double dutyL, double dutyR, bool enable, double dt) {
    const double r_ = p_.r;
    const double Iw = 2 * 0.5 * p_.m_wheel * r_ * r_;
    const double Jm = 2 * p_.J_rotor * p_.gear * p_.gear;
    const double vl = v - r * p_.track / 2, vr = v + r * p_.track / 2;
    const double wl = vl / r_ - om, wr = vr / r_ - om;  // bagil mil hizi
    auto torque = [&](double duty, double w, double& cur) {
      double t;
      if (enable) {
        double V = duty * vbat;
        cur = (V - p_.Ke * w) / p_.R;
        t = p_.Ke * cur;
      } else {
        cur = 0;
        t = 0;
      }
      return t - p_.b_visc * w - p_.tau_c * std::tanh(w / 0.3);
    };
    const double tl = torque(dutyL, wl, iL), tr = torque(dutyR, wr, iR);
    vbat = p_.voc - p_.r_int * (std::fabs(iL) + std::fabs(iR));
    const double tau = tl + tr;

    if (lying) {  // yere dustu: tekerler yerinde
      v *= 0.9;
      om = 0;
      r = 0;
      return;
    }
    const double a = 2 * p_.m_wheel * r_ * r_ + Iw + mb_ * r_ * r_ + Jm;
    const double c = mb_ * l_ * l_ + Ib_ + Jm;
    const double bc = mb_ * r_ * l_ * std::cos(th) - Jm;
    const double f1 = tau + mb_ * r_ * l_ * std::sin(th) * om * om;
    const double f2 = -tau + mb_ * 9.81 * l_ * std::sin(th);
    const double det = a * c - bc * bc;
    const double phidd = (c * f1 - bc * f2) / det;
    const double thdd = (a * f2 - bc * f1) / det;
    // sapma
    const double Iy = p_.I_yaw + (p_.track * p_.track / (4 * r_ * r_)) * (Iw + Jm);
    const double rd = ((p_.track / (2 * r_)) * (tr - tl) - 0.02 * r) / Iy;

    v += r_ * phidd * dt;
    om += thdd * dt;
    r += rd * dt;
    x += v * dt;
    th += om * dt;
    psi += r * dt;
    xl += (v - r * p_.track / 2) * dt;
```

```
    xr += (v + r * p_.track / 2) * dt;
    X += v * std::cos(psi) * dt;
    Y += v * std::sin(psi) * dt;
    accX_ = r_ * phidd;
    thdd_ = thdd;
    if (std::fabs(th) > 80 * M_PI / 180) lying = true;
  }

  // IMU olcumu (ham eksen, MPU6050 X ileri / Y sol / Z yukari): acc[0]=ileri, acc[2]=yukari, gyro[1]=+yunuslama, gyro[2]=sapma
  cdtp::ImuSample imu(bool motorsOn) {
    std::normal_distribution<double> n(0, 1);
    const double h = p_.h_imu;
    // IMU noktasinin ivmesi (dunya: ileri, yukari)
    const double ax = accX_ + h * (thdd_ * std::cos(th) - om * om * std::sin(th));
    const double az = h * (-thdd_ * std::sin(th) - om * om * std::cos(th));
    const double fx = ax, fz = az + 9.81;
    const double thm = th + p_.imu_offset_deg * M_PI / 180;
    double fwd = fx * std::cos(thm) - fz * std::sin(thm);
    double up = fx * std::sin(thm) + fz * std::cos(thm);
    const double vib = motorsOn ? p_.vib_acc * (0.3 + std::fabs(iL + iR) / 2) : 0;
    fwd += (p_.acc_noise + vib) * n(rng_);
    up += (p_.acc_noise + vib) * n(rng_);
    double side = (p_.acc_noise + vib) * n(rng_);
    const double gn = p_.gyro_noise_dps * M_PI / 180;
    double gp = om + p_.gyro_bias_dps * M_PI / 180 + gn * n(rng_);
    double gy = r + gn * n(rng_);
    // MPU6050 DLPF (~42 Hz) birinci derece
    const double a = 1.0 - std::exp(-2 * M_PI * 42.0 / 200.0);
    f_[0] += a * (fwd - f_[0]);
    f_[1] += a * (side - f_[1]);
    f_[2] += a * (up - f_[2]);
    f_[3] += a * (gp - f_[3]);
    f_[4] += a * (gy - f_[4]);
    cdtp::ImuSample s;
    s.acc[0] = (float)f_[0];
    s.acc[1] = (float)f_[1];
    s.acc[2] = (float)f_[2];
    s.gyro[0] = 0;
    s.gyro[1] = (float)f_[3];
    s.gyro[2] = (float)f_[4];
    return s;
  }

  void initFilter() {
    f_[0] = -9.81 * std::sin(th);
    f_[2] = 9.81 * std::cos(th);
  }

  int32_t encCount(bool right) const {
    const double mpc = 2 * M_PI * cdtp::cfg::WHEEL_RADIUS_M / cdtp::cfg::ENC_COUNTS_PER_REV;
    // gercek enkoder teker ile govde arasindaki bagil aciyi sayar: yol - r*theta
    double c = std::floor(((right ? xr : xl) - p_.r * th) / mpc);
    bool inv = right ? cdtp::cfg::ENC_RIGHT_INVERT : cdtp::cfg::ENC_LEFT_INVERT;
    return (int32_t)(inv ? -c : c);
  }

  const PlantParams& params() const { return p_; }

 private:
  PlantParams p_;
  std::mt19937 rng_;
  double mb_, l_, Ib_;
  double accX_ = 0, thdd_ = 0;
  double f_[5] = {0, 0, 9.81, 0, 0};
};
```

sim/harness.h 193 satır

```
// Simulasyon duzenegi: fiziksel model + istasyon IR isareti + eldiven paketleri + robot mantigi.
#pragma once
#include <cstdio>
#include <functional>
#include <string>
#include <vector>

#include "plant.h"
#include "robot_logic.h"

struct GloveEvent {
  double t;
  cdtp::Cmd cmd;  // bu andan itibaren eldivenin yolladigi komut
};

struct Trace {
  double t, x, v, th, psi, X, Y, dutyL, dutyR;
  int mode, dock, nL, nR;
};

class Harness {
 public:
  Harness(const PlantParams& pp, unsigned seed = 1) : plant(pp, seed), rng_(seed * 7919u + 13) {}
```

```
 Plant plant;
 cdtp::RobotLogic logic;
 cdtp::RobotOutputs out;
 std::vector<Trace> trace;
 bool record = false;

 // istasyon: dunya orijininde, yuzu +X'e bakar
 bool stationOn = false;
 bool docked = false;       // plakalar pogo'ya degiyor
 bool hitWallBad = false;   // istasyona yanlis aciyla carpti
 double dockTime = -1, chargeStart = -1;
 double frontOffset = 0.06;  // robot merkezinden on yuze
 double eyeAxisDeg = 15;     // gozlerin disa bakis acisi (+-)
 double eyeHalfDeg = 30;     // goz tupunun kabul yari acisi

 // eldiven
 std::vector<GloveEvent> glove;
 bool gloveOn = false;
 double gloveLoss = 0.03;    // paket kaybi orani
 double gloveCutFrom = 1e9, gloveCutTo = -1;  // bu aralikta baglanti yok

 bool motorPower = true;  // acil stop sonrasi hat
 double t = 0;

 void run(double duration, const std::function<void(Harness&)>& each = nullptr) {
   const int sub = 10;  // fizik 2 kHz
   const double dtc = cdtp::cfg::DT, dtp = dtc / sub;
   const double tEnd = t + duration;
   while (t < tEnd - 1e-9) {
     // --- sensorler ---
     cdtp::RobotInputs in;
     in.t = (float)t;
     in.imu = plant.imu(out.motorEnable);
     in.encL = plant.encCount(false);
     in.encR = plant.encCount(true);
     in.vbat = (float)(plant.vbat + noise() * 0.02);
     in.vcharge = docked ? 12.4f : 0.0f;
     in.irL = irCount_[0];
     in.irR = irCount_[1];
     in.motorPower = motorPower;
     gloveInput(in);
     out = logic.step(in);
     if (logic.mode() == cdtp::Mode::CHARGING && docked && chargeStart < 0) chargeStart = t;
     // --- fizik ---
     double dl = out.dutyL * (cdtp::cfg::MOTOR_LEFT_INVERT ? -1 : 1);
     double dr = out.dutyR * (cdtp::cfg::MOTOR_RIGHT_INVERT ? -1 : 1);
     dl = std::round(dl * 1023) / 1023;  // 10 bit PWM
     dr = std::round(dr * 1023) / 1023;
     for (int i = 0; i < sub; ++i) {
       plant.step(dl, dr, out.motorEnable && motorPower, dtp);
       t += dtp;
       beacon(t);
       stationContact();
     }
     if (record) {
       trace.push_back({t, plant.x, plant.v, plant.th, plant.psi, plant.X, plant.Y, out.dutyL, out.dutyR,
                        (int)logic.mode(), (int)logic.dockState(), logic.irNL(), logic.irNR()});
     }
     if (each) each(*this);
   }
 }

 void writeCsv(const std::string& path) const {
   FILE* f = std::fopen(path.c_str(), "w");
   if (!f) return;
   std::fprintf(f, "t,x,v,theta_deg,psi_deg,X,Y,dutyL,dutyR,mode,dock,nL,nR\n");
   for (const auto& s : trace)
     std::fprintf(f, "%.4f,%.4f,%.4f,%.3f,%.2f,%.4f,%.4f,%.4f,%.3f,%.3f,%d,%d,%d,%d\n", s.t, s.x, s.v,
                  s.th * 180 / M_PI, s.psi * 180 / M_PI, s.X, s.Y, s.dutyL, s.dutyR, s.mode, s.dock, s.nL, s.nR);
   std::fclose(f);
 }

private:
 std::mt19937 rng_;
 int32_t irCount_[2] = {0, 0};
 double nextBurst_ = 0;
 uint16_t seq_ = 0;
 double nextPacket_ = 0;

 double noise() {
   std::normal_distribution<double> n(0, 1);
   return n(rng_);
 }
 double uni() {
   std::uniform_real_distribution<double> u(0, 1);
   return u(rng_);
 }

 // Istasyon isareti: 78 ms cerceve, ilk 45 ms'de ~1 kHz patlama (44 adet), sonra 33 ms sessizlik
 void beacon(double now) {
   if (!stationOn) return;
   while (nextBurst_ <= now) {
     const double phase = std::fmod(nextBurst_, 0.078);
```

```
      if (phase < 0.045) {
        for (int e = 0; e < 2; ++e)
          if (uni() < eyeGain(e)) irCount_[e]++;
      }
      nextBurst_ += 0.0010122;
      if (std::fmod(nextBurst_, 0.078) >= 0.045) nextBurst_ = (std::floor(nextBurst_ / 0.078) + 1) * 0.078;
    }
  }

  // goz kazanci: 0..1 (patlamayi algilama olasiligi)
  double eyeGain(int eye) const {
    const double fx = plant.X + frontOffset * std::cos(plant.psi), fy = plant.Y + frontOffset * std::sin(plant.psi);
    const double dist = std::hypot(fx, fy);
    if (dist > 1.2 || fx < -0.01) return 0;
    const double gamma = std::atan2(fy, fx);  // istasyon normaline gore robotun acisi
    const double emit = std::max(0.0, 1 - std::pow(gamma / (70 * M_PI / 180), 2));
    double bearing = std::atan2(-fy, -fx) - plant.psi;  // istasyonun robota gore yonu, sola +
    while (bearing > M_PI) bearing -= 2 * M_PI;
    while (bearing < -M_PI) bearing += 2 * M_PI;
    const double axis = (eye == 0 ? eyeAxisDeg : -eyeAxisDeg) * M_PI / 180;
    const double acc = std::max(0.0, 1 - std::pow((bearing - axis) / (eyeHalfDeg * M_PI / 180), 2));
    const double g = emit * acc;
    if (g < 0.12) return 0;  // alici esigi
    return std::min(1.0, g * 1.4);
  }

  // Istasyon: orijinde dikey yuvarlak direk (cap 40 mm), uzerinde iki bakir halka (ust +, alt -) ve IR isaret.
  // Robotun onunde V-yuva var; yuvanin icindeki iki yayli pim halkalara degir. Direk yuvarlak oldugu icin
  // yaklasma acisi onemsizdir; V-yuva yanal +-3 cm hatayi kendisi ortalar.
  void stationContact() {
    if (!stationOn) return;
    const double fx = plant.X + frontOffset * std::cos(plant.psi), fy = plant.Y + frontOffset * std::sin(plant.psi);
    // direk merkezinin robot eksenine gore konumu
    const double dx = -fx, dy = -fy;
    const double along = dx * std::cos(plant.psi) + dy * std::sin(plant.psi);
    const double lateral = -dx * std::sin(plant.psi) + dy * std::cos(plant.psi);
    const double postR = 0.02;
    const bool touching = along <= postR + 0.003 && along > -0.05 && std::fabs(lateral) < 0.09;
    if (touching) {
      if (std::fabs(lateral) < 0.03) {
        if (!docked) dockTime = t;
        docked = true;
      } else {
        hitWallBad = true;  // yuvanin disindan, govde kenariyla carpti
      }
      if (plant.th > 6 * M_PI / 180) {  // direge yaslanir
        plant.th = 6 * M_PI / 180;
        if (plant.om > 0) plant.om = 0;
      }
      if (plant.v > 0) plant.v = 0;  // direk ileri gitmeyi engeller
    } else if (docked && along > postR + 0.01) {
      docked = false;
    }
  }

  void gloveInput(cdtp::RobotInputs& in) {
    in.gloveNew = false;
    if (!gloveOn || t < nextPacket_) return;
    nextPacket_ = t + 0.05;  // 20 Hz
    if (t >= gloveCutFrom && t <= gloveCutTo) return;
    if (uni() < gloveLoss) return;
    cdtp::Cmd c = cdtp::Cmd::STOP;
    for (const auto& e : glove)
      if (e.t <= t) c = e.cmd;
    cdtp::GlovePacket p{};
    p.cmd = (uint8_t)c;
    p.seq = seq_++;
    cdtp::sealPacket(p);
    cdtp::GlovePacket chk;
    if (cdtp::checkPacket(reinterpret_cast<const uint8_t*>(&p), sizeof(p), chk)) {
      in.gloveNew = true;
      in.glove = chk;
    }
  }
};
```

sim/sim_main.cpp 411 satır

```
// Sartnamedeki kabul testlerinin simulasyonu. Robot ESP32'sindeki ayni RobotLogic kodu kosar.
// Derle: tools/run_sim.sh   Cikti: sim/results/
#include <algorithm>
#include <cstdarg>
#include <cstdio>
#include <cstring>
#include <string>
#include <vector>

#include "harness.h"
#include "unit_tests.h"

using cdtp::Cmd;
```

```
using cdtp::Mode;

static const double D2R = M_PI / 180.0, R2D = 180.0 / M_PI;
static std::string OUT = "sim/results/";

struct Result {
  std::string id, name;
  bool pass;
  std::string detail;
};
static std::vector<Result> results;
static double g_lastD = 0, g_lastDx = 0, g_lastTh = 0;

static void report(const std::string& id, const std::string& name, bool pass, const char* fmt, ...) {
  char buf[512];
  va_list ap;
  va_start(ap, fmt);
  std::vsnprintf(buf, sizeof buf, fmt, ap);
  va_end(ap);
  results.push_back({id, name, pass, buf});
  std::printf("[%s] %-6s %-44s %s\n", pass ? "GECTI" : "KALDI", id.c_str(), name.c_str(), buf);
}

// Robotu dik baslatip dengeye aldirir (elle dik tutup birakma)
static void arm(Harness& h) {
  h.plant.initFilter();
  h.run(1.0);
}

struct Stats {
  double maxTh = 0, maxX = 0, minX = 1e9, maxXabs = 0;
  bool fell = false;
  void operator()(Harness& h) {
    maxTh = std::max(maxTh, std::fabs(h.plant.th));
    maxX = std::max(maxX, h.plant.x);
    minX = std::min(minX, h.plant.x);
    maxXabs = std::max(maxXabs, std::fabs(h.plant.x));
    if (h.logic.mode() == Mode::FALLEN || h.plant.lying) fell = true;
  }
};

// ---------- C1: denge ve temel hareket ----------
static bool testStatic(const PlantParams& pp, unsigned seed, bool rep, double dur = 30) {
  Harness h(pp, seed);
  h.logic.setTask(cdtp::Task::STATIC);
  h.record = rep;
  arm(h);
  Stats s;
  h.run(dur, std::ref(s));
  bool ok = !s.fell && h.logic.mode() == Mode::BALANCE && s.maxTh < 5 * D2R && s.maxXabs < 0.15;
  if (rep) {
    h.writeCsv(OUT + "c1_statik.csv");
    report("C1.1", "30 s statik denge (kutulu)", ok, "en buyuk egim %.2f deg, en buyuk kayma %.1f cm", s.maxTh * R2D,
           s.maxXabs * 100);
  }
  return ok;
}

static bool testMove1m(const PlantParams& pp, unsigned seed, bool rep, bool forward) {
  Harness h(pp, seed);
  h.logic.setTask(cdtp::Task::GUIDANCE);
  h.gloveOn = true;
  h.record = rep;
  Cmd c = forward ? Cmd::FWD : Cmd::BACK;
  // robot once 5 s dengede bekler (denge acisini ogrenir), sonra iki adim komutu: 0,5 + 0,5 m
  h.glove = {{0, Cmd::STOP}, {6.0, c}, {6.4, Cmd::STOP}, {9.5, c}, {9.9, Cmd::STOP}};
  arm(h);
  Stats s;
  h.run(13.0, std::ref(s));
  // 3 s durma
  double x0 = h.plant.x, maxV = 0, maxTh = 0, maxDx = 0;
  h.run(3.0, [&](Harness& hh) {
    maxV = std::max(maxV, std::fabs(hh.plant.v));
    maxTh = std::max(maxTh, std::fabs(hh.plant.th));
    maxDx = std::max(maxDx, std::fabs(hh.plant.x - x0));
  });
  double d = std::fabs(h.plant.x);
  // "durdu": 3 s boyunca +-3 cm icinde (disli surtunmesi + sensor gurultusu kucuk bir titresim birakir)
  g_lastD = d;
  g_lastDx = maxDx;
  g_lastTh = s.maxTh;
  bool ok = !s.fell && d > 0.9 && d < 1.1 && maxDx < 0.03;
  if (rep) {
    h.writeCsv(OUT + (forward ? "c1_ileri.csv" : "c1_geri.csv"));
    report(forward ? "C1.2" : "C1.3", forward ? "1 m ileri + 3 s dur" : "1 m geri + 3 s dur", ok,
           "yol %.3f m, 3 s durmada en buyuk kayma %.1f mm (hiz tepe %.3f m/s), en buyuk egim %.1f deg", d,
           maxDx * 1000, maxV, s.maxTh * R2D);
  }
  return ok;
}

static bool testRecovery(const PlantParams& pp, unsigned seed, bool rep, double deg, double* tRec = nullptr,
                         double* travel = nullptr) {
  Harness h(pp, seed);
```

```
  h.logic.setTask(cdtp::Task::STATIC);
  h.record = rep;
  arm(h);
  // elle +-10 deg egilip birakildi
  h.plant.th = deg * D2R;
  h.plant.om = 0;
  double t0 = h.t, recovered = -1, maxDx = 0, maxAfter = 0;
  double x0 = h.plant.x;
  Stats s;
  // toparlanma: |theta| < 3 deg ve |v| < 0,1 m/s'ye ilk varis; sonrasinda egim 5 deg'yi hic asmamali
  h.run(10.0, [&](Harness& hh) {
    s(hh);
    maxDx = std::max(maxDx, std::fabs(hh.plant.x - x0));
    if (recovered < 0 && std::fabs(hh.plant.th) < 3 * D2R && std::fabs(hh.plant.v) < 0.1) recovered = hh.t - t0;
    if (recovered >= 0) maxAfter = std::max(maxAfter, std::fabs(hh.plant.th));
  });
  if (maxAfter > 5 * D2R) recovered = -1;
  bool ok = !s.fell && recovered >= 0 && recovered < 10;
  if (tRec) *tRec = recovered;
  if (travel) *travel = maxDx;
  if (rep) h.writeCsv(OUT + (deg > 0 ? "c1_toparlanma_arti.csv" : "c1_toparlanma_eksi.csv"));
  return ok;
}

// ---------- On test: enerjisizken pasif kararsizlik ----------
static void testPassive() {
  bool ok = true;
  double worst = 0;
  for (double deg : {10.0, -10.0}) {
    Harness h(PlantParams(), 3);
    h.plant.initFilter();
    h.plant.th = deg * D2R;
    double tFall = -1;
    // motorlar enerjisiz: mantigi hic calistirmadan yalniz fizik
    for (int i = 0; i < 5 * 2000 && !h.plant.lying; ++i) h.plant.step(0, 0, false, 0.0005);
    tFall = h.plant.lying ? 0 : -1;
    double tt = 0;
    Harness h2(PlantParams(), 3);
    h2.plant.th = deg * D2R;
    while (tt < 5 && std::fabs(h2.plant.th) < 40 * D2R) {
      h2.plant.step(0, 0, false, 0.0005);
      tt += 0.0005;
    }
    if (tt >= 5) ok = false;
    worst = std::max(worst, tt);
    (void)tFall;
  }
  report("ON", "Enerjisiz +-10 deg'den dik kalmamali", ok, "40 deg'yi gecme suresi en fazla %.2f s (sinir: 5 s icinde dusmeli)",
         worst);
}

// ---------- C2: eldiven komutlari ----------
static void testGlove() {
  struct Case { const char* id; const char* name; Cmd c; bool lin; double sign; };
  Case cs[] = {{"C2.1", "Eldiven: ileri ~0,5 m", Cmd::FWD, true, 1},
               {"C2.2", "Eldiven: geri ~0,5 m", Cmd::BACK, true, -1},
               {"C2.3", "Eldiven: sola ~90 deg", Cmd::LEFT, false, 1},
               {"C2.4", "Eldiven: saga ~90 deg", Cmd::RIGHT, false, -1}};
  for (auto& k : cs) {
    Harness h(PlantParams(), 11);
    h.logic.setTask(cdtp::Task::GUIDANCE);
    h.gloveOn = true;
    h.gloveLoss = 0.05;
    h.glove = {{0, Cmd::STOP}, {1.5, k.c}, {2.0, Cmd::STOP}};
    arm(h);
    const double tCmd = 1.5;  // eldiven jesti bu anda baslar
    double tResp = -1, x0 = h.plant.x;
    Stats s;
    h.run(6.0, [&](Harness& hh) {
      s(hh);
      if (hh.t <= tCmd) x0 = hh.plant.x;
      // tepki: komut yonunde 3 cm yol ya da 5 deg donus (denge titremesi tetiklemesin)
      bool moving = k.lin ? (hh.plant.x - x0) * k.sign > 0.03 : hh.plant.psi * k.sign > 5 * D2R;
      if (moving && tResp < 0 && hh.t > tCmd) tResp = hh.t - tCmd;
    });
    double val = k.lin ? h.plant.x : h.plant.psi * R2D;
    bool ok = !s.fell && tResp >= 0 && tResp < 2.0 &&
              (k.lin ? (std::fabs(val - 0.5 * k.sign) < 0.1) : (std::fabs(val - 90 * k.sign) < 15));
    report(k.id, k.name, ok, k.lin ? "tepki %.2f s, yol %.3f m" : "tepki %.2f s, donus %.1f deg", tResp, val);
  }
  // dur: bes komut art arda, sonra dur; robot durur ve yerinde kalir
  {
    Harness h(PlantParams(), 12);
    h.logic.setTask(cdtp::Task::GUIDANCE);
    h.gloveOn = true;
    h.gloveLoss = 0.05;
    h.record = true;
    h.glove = {{0, Cmd::STOP},  {1.5, Cmd::FWD},    {1.9, Cmd::STOP},  {5.0, Cmd::BACK}, {5.4, Cmd::STOP},
               {8.5, Cmd::LEFT}, {8.9, Cmd::STOP}, {11.0, Cmd::RIGHT}, {11.4, Cmd::STOP}};
    arm(h);
    Stats s;
    h.run(13.5, std::ref(s));
    double x0 = h.plant.x, p0 = h.plant.psi;
    h.run(3.0, std::ref(s));
```

```
    bool ok = !s.fell && std::fabs(h.plant.x - x0) < 0.03 && std::fabs(h.plant.psi - p0) < 3 * D2R;
    h.writeCsv(OUT + "c2_eldiven.csv");
    report("C2.5", "Dur: tum komutlardan sonra yerinde kalir", ok, "son 3 s kayma %.1f mm, donme %.2f deg",
           std::fabs(h.plant.x - x0) * 1000, std::fabs(h.plant.psi - p0) * R2D);
  }
  // baglanti kopmasi: adim ortasinda eldiven susarsa robot durur
  {
    Harness h(PlantParams(), 13);
    h.logic.setTask(cdtp::Task::GUIDANCE);
    h.gloveOn = true;
    h.glove = {{0, Cmd::STOP}, {1.5, Cmd::FWD}, {1.9, Cmd::STOP}};
    h.gloveCutFrom = 2.5;
    h.gloveCutTo = 99;
    arm(h);
    Stats s;
    h.run(5.0, std::ref(s));
    bool ok = !s.fell && std::fabs(h.plant.v) < 0.1 && h.plant.x < 0.45;
    report("GUV.1", "Eldiven sustu (0,3 s): adim kesilir, robot durur", ok, "durdugu yol %.3f m (tam adim 0,5 m)",
           h.plant.x);
  }
}

// ---------- C3: otonom yanasma ----------
struct DockRes { bool ok; double tDock, tCharge; bool bad; };

static DockRes runDock(double rho, double alphaDeg, double headOffDeg, unsigned seed, bool rec = false,
                       const PlantParams& pp = PlantParams()) {
  Harness h(pp, seed);
  h.logic.setTask(cdtp::Task::CHARGING);
  h.stationOn = true;
  h.record = rec;
  const double a = alphaDeg * D2R;
  h.plant.X = rho * std::cos(a);
  h.plant.Y = rho * std::sin(a);
  h.plant.psi = std::atan2(-h.plant.Y, -h.plant.X) + headOffDeg * D2R;  // istasyona bakis + sapma
  double psiW0 = h.plant.psi;
  arm(h);
  (void)psiW0;
  double tStart = h.t;
  bool fell = false;
  h.run(60.0, [&](Harness& hh) {
    if (hh.plant.lying || hh.logic.mode() == Mode::FALLEN) fell = true;
    if (hh.logic.mode() == Mode::CHARGING && hh.chargeStart > 0 && hh.t > hh.chargeStart + 0.5) hh.t = 1e9;
  });
  DockRes r;
  r.ok = !fell && h.docked && h.logic.mode() == Mode::CHARGING && !h.hitWallBad;
  r.tDock = h.dockTime > 0 ? h.dockTime - tStart : -1;
  r.tCharge = (h.chargeStart > 0 && h.dockTime > 0) ? h.chargeStart - h.dockTime : -1;
  r.bad = h.hitWallBad;
  if (rec) h.writeCsv(OUT + "c3_yanasma.csv");
  return r;
}

static void testDock() {
  int n = 0, okN = 0;
  double worstT = 0, worstCharge = 0;
  std::string fails;
  for (double rho : {0.15, 0.22, 0.30})
    for (double al : {-45.0, -30.0, -15.0, 0.0, 15.0, 30.0, 45.0})
      for (double ho : {-30.0, 0.0, 30.0}) {
        DockRes r = runDock(rho, al, ho, 100 + n);
        ++n;
        if (r.ok) {
          ++okN;
          worstT = std::max(worstT, r.tDock);
          worstCharge = std::max(worstCharge, r.tCharge);
        } else {
          char b[96];
          std::snprintf(b, sizeof b, " (%.2f m, %.0f deg, bakis %+.0f%s)", rho, al, ho, r.bad ? ", yuva disi carpma" : "");
          fails += b;
        }
      }
  report("C3.1-2", "IR ile bul, hizalan, yerles (63 baslangic, 30 cm/90 deg)", okN == n, "%d/%d basarili, en uzun %.1f s%s", okN, n,
         worstT, fails.c_str());
  report("C3.3", "Temastan sonra sarj + motor durur <= 5 s", okN > 0 && worstCharge < 5.0,
         "temas -> SARJ modu en fazla %.2f s", worstCharge);
  // arkasi istasyona donuk baslangic: ARA durumu tam tur atmali
  int backOk = 0;
  for (double ho : {90.0, -90.0, 180.0}) backOk += runDock(0.25, 10, ho, 300 + (int)ho).ok;
  report("C3.x", "Istasyon gorus disindayken arayip bulma", backOk == 3, "%d/3 basarili (90, -90, 180 deg ters bakis)",
         backOk);
  runDock(0.30, 35, 25, 7, true);
}

// ---------- C5: butunlesik gorev ----------
static void testIntegrated(int rep, bool record) {
  Harness h(PlantParams(), 500 + rep);
  h.logic.setTask(cdtp::Task::INTEGRATED);
  h.stationOn = true;
  h.gloveOn = true;
  h.gloveLoss = 0.05;
  h.record = record;
  // istasyon (0,0) +X'e bakar. Robot (0.27,-1.0)'dan +Y'ye bakarak baslar.
  h.plant.X = 0.27;
```

```
  h.plant.Y = -1.0;
  h.plant.psi = M_PI / 2;
  // 5 s denge -> 2 x ileri (1 m, hedef nokta) -> hedefte 5 s bekle -> sola don -> dur -> otonom yanasma
  h.glove = {{0, Cmd::STOP}, {5.0, Cmd::FWD}, {5.4, Cmd::STOP}, {8.6, Cmd::FWD}, {9.0, Cmd::STOP},
             {17.0, Cmd::LEFT}, {17.4, Cmd::STOP}};
  arm(h);
  bool fell = false, earlyDock = false;
  double targetWaitStart = -1;
  h.run(70.0, [&](Harness& hh) {
    if (hh.plant.lying || hh.logic.mode() == Mode::FALLEN) fell = true;
    if (hh.t < 17.0 && hh.logic.mode() == Mode::DOCK) earlyDock = true;
    if (targetWaitStart < 0 && hh.t > 9 && std::fabs(hh.plant.v) < 0.02) targetWaitStart = hh.t;
    if (hh.logic.mode() == Mode::CHARGING && hh.chargeStart > 0 && hh.t > hh.chargeStart + 0.5) hh.t = 1e9;
  });
  bool ok = !fell && !earlyDock && h.docked && h.logic.mode() == Mode::CHARGING;
  if (record) h.writeCsv(OUT + "c5_butunlesik.csv");
  char nm[64];
  std::snprintf(nm, sizeof nm, "Butunlesik gorev, tekrar %d", rep);
  report("C5." + std::to_string(rep), nm, ok, "hedefte bekleme %.1f s, erken yanasma %s, sarj %s, temas->sarj %.2f s",
         17.0 - targetWaitStart, earlyDock ? "VAR" : "yok", h.docked ? "basladi" : "YOK",
         h.chargeStart > 0 ? h.chargeStart - h.dockTime : -1.0);
}

// ---------- Saglamlik: parametre belirsizligi ----------
static void testRobust() {
  std::mt19937 g(2026);
  auto U = [&](double a, double b) { return std::uniform_real_distribution<double>(a, b)(g); };
  const int N = 40;
  int okS = 0, okR = 0, okM = 0, okM5 = 0;
  double worstRec = 0;
  std::string worst;
  for (int i = 0; i < N; ++i) {
    PlantParams p;
    p.m_body *= U(0.75, 1.3);
    p.l_com *= U(0.75, 1.6);  // kutle merkezi: tahminin %75'i ile %160'i arasi
    p.I_body *= U(0.7, 1.4);
    p.R *= U(0.8, 1.25);
    p.Ke *= U(0.9, 1.1);
    p.tau_c *= U(0.5, 2.0);
    p.b_visc *= U(0.5, 2.0);
    p.voc = U(10.4, 12.6);
    p.box_mass = U(0.03, 0.10);
    p.imu_offset_deg = U(-1.5, 1.5);
    p.gyro_bias_dps = U(-0.5, 0.5);
    p.h_imu = U(0.0, 0.12);
    unsigned seed = 1000 + i;
    bool s = testStatic(p, seed, false, 15);
    double tr1 = 0, tr2 = 0;
    bool r = testRecovery(p, seed, false, 10, &tr1) && testRecovery(p, seed, false, -10, &tr2);
    bool m3 = testMove1m(p, seed, false, true);
    // modelden uzak robotlarda durma olcutu +-5 cm (nominal testte +-3 cm)
    bool m = g_lastD > 0.9 && g_lastD < 1.1 && g_lastDx < 0.05;
    okS += s;
    okR += r;
    okM += m3;
    okM5 += m;
    if (r) worstRec = std::max(worstRec, std::max(tr1, tr2));
    if (!(s && r && m)) {
      char b[160];
      std::snprintf(b, sizeof b, " [m=%.2f l=%.3f R=%.1f V=%.1f ofs=%.1f: %s%s%s d=%.2f dx=%.0fmm th=%.0f]", p.m_body,
                    p.l_com, p.R, p.voc, p.imu_offset_deg, s ? "" : "S", r ? "" : "T", m ? "" : "H", g_lastD,
                    g_lastDx * 1000, g_lastTh * R2D);
      worst += b;
    }
  }
  report("SAG", "Belirsizlik: kutle +-25%, KM -25/+60%, ...", okS == N && okR == N && okM5 == N,
         "statik %d/%d, toparlanma %d/%d (en uzun %.1f s), 1 m: +-5 cm durmada %d/%d, +-3 cm durmada %d/%d%s", okS, N,
         okR, N, worstRec, okM5, N, okM, N, worst.c_str());
}

int main(int argc, char** argv) {
  if (argc > 1) OUT = argv[1];
  std::printf("== Birim testleri ==\n");
  int fails = runUnitTests();
  std::printf("\n== Kabul testleri (simulasyon) ==\n");
  testPassive();
  testStatic(PlantParams(), 1, true);
  testMove1m(PlantParams(), 2, true, true);
  testMove1m(PlantParams(), 3, true, false);
  {
    double tp, tm, dp, dm;
    bool a = testRecovery(PlantParams(), 4, true, 10, &tp, &dp);
    bool b = testRecovery(PlantParams(), 5, true, -10, &tm, &dm);
    report("C1.4", "+10 ve -10 deg'den 10 s icinde toparlanma", a && b,
           "+10: %.2f s (yol %.0f cm), -10: %.2f s (yol %.0f cm)", tp, dp * 100, tm, dm * 100);
  }
  {
    Harness h(PlantParams(), 21);
    h.logic.setTask(cdtp::Task::STATIC);
    arm(h);
    h.motorPower = false;  // acil stopa basildi
    h.run(0.02);
    bool off = h.logic.mode() == Mode::STANDBY && !h.out.motorEnable;
    h.motorPower = true;   // birakildi; robot yatik -> kendiliginden donmemeli
```

```
    h.run(0.3);
    bool stay = h.logic.mode() != Mode::BALANCE || std::fabs(h.plant.th) < 3 * D2R;
    report("GUV.2", "Acil stop: denge birakilir, motorlar kapanir", off && stay, "stop sonrasi mod %s, bekleme %s",
           off ? "BEKLE" : "YANLIS", stay ? "dogru" : "yanlis");
  }
  testGlove();
  testDock();
  for (int i = 1; i <= 4; ++i) testIntegrated(i, i == 1);
  testRobust();

  int nOk = 0;
  for (auto& r : results) nOk += r.pass;
  std::printf("\nOZET: %d/%d kabul testi gecti, birim testi hatasi: %d\n", nOk, (int)results.size(), fails);

  FILE* f = std::fopen((OUT + "summary.tsv").c_str(), "w");
  if (f) {
    for (auto& r : results) std::fprintf(f, "%s\t%s\t%s\t%s\n", r.id.c_str(), r.name.c_str(), r.pass ? "GECTI" : "KALDI", r.detail.c_str());
    std::fclose(f);
  }
  return (nOk == (int)results.size() && fails == 0) ? 0 : 1;
}
```

sim/unit_tests.h 101 satır

```
// Ortak cekirdek icin birim testleri (eldiven jesti, IR cerceve, paket, profil, telemetri).
#pragma once
#include <cstdio>
#include <cstring>

#include "gesture.h"
#include "ir_frame.h"
#include "protocol.h"
#include "robot_logic.h"

static int g_fail = 0;
#define EXPECT(c, msg)                                   \
  do {                                                   \
    if (!(c)) {                                          \
      std::printf("  HATA: %s (%s:%d)\n", msg, __FILE__, __LINE__); \
      ++g_fail;                                          \
    }                                                    \
  } while (0)

static cdtp::Cmd feed(cdtp::GestureClassifier& g, float p, float r, float sec) {
  cdtp::Cmd c = cdtp::Cmd::NONE;
  for (float t = 0; t < sec; t += 0.02f) c = g.update(p, r, 0.02f);
  return c;
}

static int runUnitTests() {
  using cdtp::Cmd;
  // --- jest siniflandirici ---
  {
    cdtp::GestureClassifier g;
    EXPECT(feed(g, 0, 0, 0.5) == Cmd::STOP, "notr el = dur");
    EXPECT(feed(g, 30, 0, 0.3) == Cmd::FWD, "one 30 deg = ileri");
    EXPECT(feed(g, 18, 0, 0.5) == Cmd::FWD, "histerezis: 18 deg'de ileri surer");
    EXPECT(feed(g, 5, 0, 0.5) == Cmd::STOP, "5 deg'de dur");
    EXPECT(feed(g, -30, 0, 0.3) == Cmd::BACK, "geri");
    feed(g, 0, 0, 0.5);
    EXPECT(feed(g, 0, -30, 0.3) == Cmd::LEFT, "sol");
    feed(g, 0, 0, 0.5);
    EXPECT(feed(g, 0, 30, 0.3) == Cmd::RIGHT, "sag");
    feed(g, 0, 0, 0.5);
    EXPECT(feed(g, 28, 26, 0.5) == Cmd::STOP, "capraz el (28/26) belirsiz: komut degismez");
    EXPECT(feed(g, 30, 0, 0.06) == Cmd::STOP, "0,06 s'lik titreme komut sayilmaz");
    EXPECT(feed(g, 20, 0, 1.0) == Cmd::STOP, "20 deg giris esiginin altinda");
  }
  // --- IR cerceve sayaci ---
  {
    cdtp::IrFrameCounter c;
    int32_t cnt = 0;
    int got = -1;
    for (int ms = 0; ms < 400; ms += 1) {
      double ph = std::fmod(ms, 78.0);
      if (ph < 45 && ms % 1 == 0) cnt++;  // ~44-45 patlama
      if (ms % 5 == 0) {
        c.sample(cnt, ms / 1000.0f);
        if (c.n() > 0) got = c.n();
      }
    }
    EXPECT(got >= 40 && got <= 46, "tam gorus ~44 darbe/cerceve");
    for (int ms = 400; ms < 800; ms += 5) c.sample(cnt, ms / 1000.0f);
    EXPECT(c.n() == 0, "isaret kesilince 0,25 s icinde n=0");
  }
  // --- paket ---
  {
    cdtp::GlovePacket p{};
    p.cmd = (uint8_t)Cmd::LEFT;
    p.seq = 77;
    cdtp::sealPacket(p);
```

```
    cdtp::GlovePacket o;
    EXPECT(cdtp::checkPacket((const uint8_t*)&p, sizeof p, o) && o.cmd == (uint8_t)Cmd::LEFT, "gecerli paket");
    uint8_t bad[sizeof p];
    std::memcpy(bad, &p, sizeof p);
    bad[5] ^= 1;
    EXPECT(!cdtp::checkPacket(bad, sizeof p, o), "bozuk CRC reddedilir");
    cdtp::GlovePacket q = p;
    q.key = 0x12345678;
    q.crc = cdtp::crc8((const uint8_t*)&q, sizeof q - 1);
    EXPECT(!cdtp::checkPacket((const uint8_t*)&q, sizeof q, o), "baska takimin anahtari reddedilir");
  }
  // --- trapez profil ---
  {
    cdtp::Trapezoid tr;
    tr.start(0.5f, 0.25f, 0.4f);
    float vmax = 0;
    int n = 0;
    while (tr.active() && n < 4000) { tr.step(0.005f); vmax = std::max(vmax, tr.vel()); ++n; }
    EXPECT(std::fabs(tr.pos() - 0.5f) < 1e-3f, "profil 0,5 m'de biter");
    EXPECT(vmax <= 0.2501f, "hiz siniri");
  }
  // --- telemetri ---
  {
    cdtp::RobotLogic lg;
    char buf[320];
    int n = lg.telemetryJson(buf, sizeof buf, 5);
    EXPECT(n > 0 && n < (int)sizeof buf && buf[0] == '{' && std::strstr(buf, "\"tilt\":") && std::strstr(buf, "\"chg\":"),
           "telemetri JSON");
    std::printf("  ornek telemetri (%d bayt): %s\n", n, buf);
  }
  std::printf("  birim testleri: %s\n", g_fail ? "HATA VAR" : "hepsi gecti");
  return g_fail;
}
```

MuJoCo dijital ikizi

twin/twin.py 634 satır

```
"""MuJoCo dijital ikizi.

C++ simulasyonundan (sim/plant.h) bagimsiz ikinci bir model:
  * Govde tools/mass_budget.py'deki her parcadan ayri kutu olarak kurulur (kutle, konum, atalet MuJoCo hesaplar).
  * Teker-zemin temasi, kayma, ilac kutusunun cepte durmasi, V-yuvanin direge oturmasi gercek temasla cozulur.
  * Sensorler fiziksel kurallarla uretilir: MPU6050 (X ileri, Y sol, Z yukari) MuJoCo'nun ivme/jiroskop
    sensorunden; enkoder teker ile govde arasindaki BAGIL acidan (gercek enkoder gibi).
  * Karar mantigi: robot firmware'indeki RobotLogic, build/twin_bridge.exe uzerinden (ayni C++ kodu).
Calistir:  <venv>/python twin/twin.py   ->  twin/results/*.csv ve summary.tsv
"""
import math
import pathlib
import subprocess
import sys

import mujoco
import numpy as np

ROOT = pathlib.Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "tools"))
import mass_budget  # noqa: E402

OUT = ROOT / "twin/results"
OUT.mkdir(parents=True, exist_ok=True)
BRIDGE = ROOT / "build/twin_bridge.exe"

# ---- fiziksel parametreler (lqr_design.py ile ayni kaynaklar; motor degerleri veri sayfasi tahmini) ----
R_WHEEL, TRACK = 0.0325, 0.19
KE, RA = 0.216, 4.5
J_ROTOR, GEAR = 5.0e-7, 19.0
B_VISC, TAU_C = 0.002, 0.02
CPR = 836
VOC, RINT = 12.0, 0.15
MAGNET_N, MAGNET_RANGE = 10.0, 0.006  # V-yuvadaki miknatis ile direkteki celik serit: temasta ~10 N, 6 mm'de sifir
PHYS_DT, CTRL_EVERY = 0.0005, 10  # 2 kHz fizik, 200 Hz kontrol
TASKS = {"statik": 0, "yonlendirme": 1, "sarj": 2, "butunlesik": 3}
CMD = {"yok": 0, "ileri": 1, "geri": 2, "sol": 3, "sag": 4, "dur": 5}

# parca boyutlari (yari olcu, m) mass_budget isimlerine gore
SIZES = {
    "Kat": (0.055, 0.075, 0.0015),
    "BTS": (0.025, 0.030, 0.015),
    "3x18650": (0.035, 0.040, 0.012),
    "Ana kart": (0.025, 0.050, 0.008),
    "Acil": (0.020, 0.020, 0.020),
    "Kutu yuvasi": (0.038, 0.025, 0.003),
    "LM2596": (0.020, 0.030, 0.010),
    "MPU": (0.010, 0.010, 0.002),
    "IR": (0.008, 0.020, 0.010),
    "16x": (0.050, 0.070, 0.004),
}


def size_for(name):
    for k, v in SIZES.items():
        if name.startswith(k):
            return v
    return (0.02, 0.02, 0.02)


def build_xml(post_xy=None, box=True):
    geoms = []
    for name, m, z, x, _ in mass_budget.PARTS:
        if name.startswith("2x JGB37"):
            # iki motor teker ekseninde, y boyunca
            for s in (1, -1):
                geoms.append(f'<geom type="cylinder" size="0.0185 0.035" pos="0 {s*0.045} 0" euler="90 0 0" mass="{m/2}" rgba=".3 .5 .5 1"/>')
            continue
        sx, sy, sz = size_for(name)
        # yalniz katlar carpisir (yere dusme, direk); diger parcalar kutle/atalet tasir, kutu cebine girmez
        coll = "" if name.startswith("Kat") else ' contype="0" conaffinity="0"'
        geoms.append(f'<geom type="box" size="{sx} {sy} {sz}" pos="{x} 0 {z}" mass="{m}" rgba=".6 .7 .8 1"{coll}/>')
    # kutu cebi (kat 3 ustunde, 4 duvar, kutle kutu yuvasi parcasinda sayildi)
    pocket = []
    zc = 0.184 + 0.0015 + 0.0125
    for px, py, hx, hy in ((0.0365, 0, 0.0015, 0.025), (-0.0365, 0, 0.0015, 0.025), (0, 0.024, 0.038, 0.0015), (0, -0.024, 0.038, 0.0015)):
        pocket.append(f'<geom type="box" size="{hx} {hy} 0.0125" pos="{px} {py} {zc}" mass="0.0001" rgba=".8 .8 .9 .5"/>')
    # V-yuva (belgedeki gibi): arkada 20 mm genis dikey serit (pimler burada, x=0.058), iki yanda 45 derece kanat
    vee = ['<geom type="box" size="0.0015 0.010 0.025" pos="0.0575 0 0.06" mass="0.0001" rgba=".2 .3 .9 1"/>']
    for s_ in (1, -1):
        # kanat (0.058, +-0.010) noktasindan 45 derece disa, 42 mm
        cx, cy = 0.058 + 0.0148, s_ * (0.010 + 0.0148)
        vee.append(f'<geom type="box" size="0.021 0.0015 0.025" pos="{cx} {cy} 0.06" euler="0 0 {s_*45}" mass="0.0001" rgba=".2 .3 .9 1"/>')
    post = ""
    if post_xy is not None:
        post = (f'<body name="post" pos="{post_xy[0]} {post_xy[1]} 0.1"><geom type="cylinder" size="0.02 0.1" '
                f'rgba=".5 .5 .5 1"/></body>')
    box_body = ""
```

```
    if box:
        box_body = (f'<body name="pillbox" pos="0 0 {R_WHEEL + zc + 0.001}"><freejoint/>'
                    f'<geom type="box" size="0.035 0.0225 0.0125" mass="0.06" rgba=".9 .3 .3 1" friction="0.6 0.005 0.0001"/></body>')
    arm = J_ROTOR * GEAR**2
    xml = f"""
<mujoco model="cdtp_twin">
  <option timestep="{PHYS_DT}" gravity="0 0 -9.81" integrator="implicitfast"/>
  <default><geom friction="1.0 0.01 0.001" condim="3"/></default>
  <worldbody>
    <geom name="floor" type="plane" size="5 5 0.1" rgba=".9 .9 .9 1"/>
    {post}
    <body name="chassis" pos="0 0 {R_WHEEL}">
      <freejoint name="root"/>
      {''.join(geoms)}
      {''.join(pocket)}
      {''.join(vee)}
      <site name="imu" pos="0 0 0.030"/>
      <site name="pin" pos="0.062 0 0.07"/>
      <body name="wheel_L" pos="0 {TRACK/2} 0">
        <joint name="wl" type="hinge" axis="0 1 0" armature="{arm}" damping="{B_VISC}" frictionloss="{TAU_C}"/>
        <geom type="cylinder" size="{R_WHEEL} 0.013" euler="90 0 0" mass="0.045" rgba=".1 .1 .1 1"/>
      </body>
      <body name="wheel_R" pos="0 {-TRACK/2} 0">
        <joint name="wr" type="hinge" axis="0 1 0" armature="{arm}" damping="{B_VISC}" frictionloss="{TAU_C}"/>
        <geom type="cylinder" size="{R_WHEEL} 0.013" euler="90 0 0" mass="0.045" rgba=".1 .1 .1 1"/>
      </body>
    </body>
    {box_body}
  </worldbody>
  <actuator>
    <motor name="ml" joint="wl" gear="1" ctrllimited="false"/>
    <motor name="mr" joint="wr" gear="1" ctrllimited="false"/>
  </actuator>
  <sensor>
    <accelerometer name="acc" site="imu"/>
    <gyro name="gyro" site="imu"/>
  </sensor>
</mujoco>"""
    return xml


class Twin:
    def __init__(self, x=0.0, y=0.0, yaw=0.0, post=None, task="statik", seed=1, box=True, bridge=True):
        self.m = mujoco.MjModel.from_xml_string(build_xml(post, box))
        self.d = mujoco.MjData(self.m)
        self.rng = np.random.default_rng(seed)
        self.post = post
        self.task = TASKS[task]
        q = self.d.qpos
        q[0:3] = [x, y, R_WHEEL]
        q[3:7] = [math.cos(yaw / 2), 0, 0, math.sin(yaw / 2)]
        if box:
            bj = self.m.joint(self.m.body("pillbox").jntadr[0])
            a = bj.qposadr[0]
            q[a:a + 3] = [x, y, q[a + 2]]
            q[a + 3:a + 7] = q[3:7]
            self.box_adr = a
        self.has_box = box
        self.jl, self.jr = self.m.joint("wl"), self.m.joint("wr")
        mujoco.mj_forward(self.m, self.d)
        self.t = 0.0
        self.duty = (0.0, 0.0)
        self.en = False
        self.mode, self.dock, self.theta_logic = 0, 0, 0.0
        self.filt = None
        self.ir = [0, 0]
        self.frame, self.burst = 0, 0
        self.glove = []        # [(t, cmd)]
        self.glove_next = 0.0
        self.motor_power = True
        self.hold = True       # elle dik tutuluyor
        self.magnet = True     # V-yuva miknatisi
        self.vbat = VOC
        self.trace = []
        self.proc = subprocess.Popen([str(BRIDGE)], stdin=subprocess.PIPE, stdout=subprocess.PIPE, text=True,
                                     bufsize=1) if bridge else None

    # ---- durum ----
    def rot(self):
        return self.d.xmat[self.m.body("chassis").id].reshape(3, 3)

    def pitch(self):
        R = self.rot()
        zb, xb = R[:, 2], R[:, 0]
        h = np.array([xb[0], xb[1], 0.0])
        h /= np.linalg.norm(h) + 1e-12
        return math.atan2(zb @ h, zb[2])

    def yaw(self):
        R = self.rot()
        return math.atan2(R[1, 0], R[0, 0])

    def pos(self):
        return self.d.qpos[0], self.d.qpos[1]
```

```
    def fwd_dist(self, x0, y0, yaw0):
        x, y = self.pos()
        return (x - x0) * math.cos(yaw0) + (y - y0) * math.sin(yaw0)

    def charging(self):
        if self.post is None:
            return False
        p = self.d.site("pin").xpos
        dist = math.hypot(p[0] - self.post[0], p[1] - self.post[1])
        return dist < 0.02 + 0.0015  # pim ucu seritten 3 mm disarida, yay yolu ~1,5 mm

    def box_ok(self):
        if not self.has_box:
            return True
        bp = self.d.qpos[self.box_adr:self.box_adr + 3]
        cid = self.m.body("chassis").id
        local = self.rot().T @ (bp - self.d.xpos[cid])
        return abs(local[0]) < 0.03 and abs(local[1]) < 0.03 and local[2] > 0.18

    # ---- sensorler ----
    def sensors(self):
        acc = self.d.sensor("acc").data.copy()
        gyro = self.d.sensor("gyro").data.copy()
        i_mot = abs(self.cur[0]) + abs(self.cur[1]) if hasattr(self, "cur") else 0
        vib = 0.3 * (0.3 + i_mot / 2) if self.en else 0
        acc += self.rng.normal(0, 0.03 + vib, 3)
        gyro += self.rng.normal(0, math.radians(0.05), 3)
        a = 1 - math.exp(-2 * math.pi * 42 / 200)  # MPU6050 DLPF
        raw = np.concatenate([acc, gyro])
        self.filt = raw if self.filt is None else self.filt + a * (raw - self.filt)
        return self.filt

    def enc(self):
        # gercek enkoder: teker ile govde (motor) arasindaki bagil aci
        mpc = 2 * math.pi / CPR
        cl = math.floor(self.d.qpos[self.jl.qposadr[0]] / mpc)
        cr = math.floor(self.d.qpos[self.jr.qposadr[0]] / mpc)
        return cl, -cr  # sag enkoder ayna takili

    def eye_gain(self, eye):
        if self.post is None:
            return 0.0
        R = self.rot()
        cid = self.m.body("chassis").id
        front = self.d.xpos[cid] + R @ np.array([0.06, 0, 0])
        dx, dy = self.post[0] - front[0], self.post[1] - front[1]
        dist = math.hypot(dx, dy)
        if dist > 1.2 or dist < 0.01:
            return 0.0
        gamma = math.atan2(-dy, -dx)  # direk normaline (+X) gore robotun acisi
        emit = max(0.0, 1 - (gamma / math.radians(70)) ** 2)
        bearing = math.atan2(dy, dx) - self.yaw()
        bearing = (bearing + math.pi) % (2 * math.pi) - math.pi
        axis = math.radians(15 if eye == 0 else -15)
        acc = max(0.0, 1 - ((bearing - axis) / math.radians(30)) ** 2)
        g = emit * acc
        return 0.0 if g < 0.12 else min(1.0, 1.4 * g)

    def beacon(self):
        # 78 ms cerceve: ilk 45 ms'de 1,012 ms arayla 44 patlama, sonra 33 ms sessizlik (tamsayi sayacla)
        while True:
            tb = self.frame * 0.078 + self.burst * 0.0010122
            if tb > self.t:
                return
            for e in (0, 1):
                if self.rng.random() < self.eye_gain(e):
                    self.ir[e] += 1
            self.burst += 1
            if self.burst >= 44:
                self.burst = 0
                self.frame += 1

    def glove_cmd(self):
        c = "dur"
        for t, cmd in self.glove:
            if t <= self.t:
                c = cmd
        return CMD[c]

    # ---- adim ----
    def control(self):
        s = self.sensors()
        el, er = self.enc()
        gnew = 0
        if self.glove and self.t >= self.glove_next:
            self.glove_next = self.t + 0.05
            gnew = 1 if self.rng.random() > 0.05 else 0
        vchg = 12.4 if self.charging() else 0.0
        line = (f"{self.t:.4f} {s[0]:.5f} {s[1]:.5f} {s[2]:.5f} {s[3]:.6f} {s[4]:.6f} {s[5]:.6f} {el} {er} "
                f"{self.vbat:.3f} {vchg:.2f} {self.ir[0]} {self.ir[1]} {int(self.motor_power)} {gnew} {self.glove_cmd()} {self.task}\n")
        self.proc.stdin.write(line)
        out = self.proc.stdout.readline().split()
        self.en = out[0] == "1"
```

```
        dl, dr = float(out[1]), float(out[2])
        self.duty = (round(dl * 1023) / 1023, -round(dr * 1023) / 1023)  # sag motor ayna
        self.mode, self.dock, self.theta_logic = int(out[3]), int(out[4]), float(out[5])

    def physics(self):
        # DC motor: tork = Ke*(V - Ke*w)/R ; kuru/viskoz surtunme eklem parametrelerinde
        wl = self.d.qvel[self.jl.dofadr[0]]
        wr = self.d.qvel[self.jr.dofadr[0]]
        on = self.en and self.motor_power
        il = (self.duty[0] * self.vbat - KE * wl) / RA if on else 0.0
        ir = (self.duty[1] * self.vbat - KE * wr) / RA if on else 0.0
        self.cur = (il, ir)
        self.vbat = VOC - RINT * (abs(il) + abs(ir))
        self.d.ctrl[0] = KE * il
        self.d.ctrl[1] = KE * ir
        self.d.qfrc_applied[:] = 0
        if self.post is not None and self.magnet:
            p = self.d.site("pin").xpos
            dx, dy = self.post[0] - p[0], self.post[1] - p[1]
            dist = math.hypot(dx, dy)
            gap = dist - 0.02
            if gap < MAGNET_RANGE and dist > 1e-6:
                f = MAGNET_N * (1 - max(gap, 0) / MAGNET_RANGE)
                force = np.array([dx / dist * f, dy / dist * f, 0.0])
                mujoco.mj_applyFT(self.m, self.d, force, np.zeros(3), p, self.m.body("chassis").id, self.d.qfrc_applied)
        if self.hold:  # elle dik tutma: govdeyi dik ve hareketsiz tut
            yaw = self.yaw()
            self.d.qpos[3:7] = [math.cos(yaw / 2), 0, 0, math.sin(yaw / 2)]
            self.d.qvel[0:6] = 0
        mujoco.mj_step(self.m, self.d)
        self.t += PHYS_DT
        self.beacon()

    def run(self, dur, each=None):
        n = int(round(dur / PHYS_DT))
        for i in range(n):
            if i % CTRL_EVERY == 0:
                self.control()
                if self.hold and self.mode == 1:  # DENGE'ye girdi: birak
                    self.hold = False
                x, y = self.pos()
                self.trace.append((self.t, x, y, math.degrees(self.pitch()), math.degrees(self.yaw()), self.duty[0],
                                   self.duty[1], self.mode, self.dock))
                if each:
                    each(self)
            self.physics()

    def arm(self):
        self.run(1.0)

    def tilt(self, deg):
        # govdeyi teker ekseni etrafinda egerek birak (elle egip birakma)
        R = self.rot()
        yb = R[:, 1]
        q = np.zeros(4)
        mujoco.mju_axisAngle2Quat(q, yb, math.radians(deg))
        q0 = self.d.qpos[3:7].copy()
        qn = np.zeros(4)
        mujoco.mju_mulQuat(qn, q, q0)
        cid = self.m.body("chassis").id
        axle = self.d.xpos[cid].copy()
        if self.has_box:  # kutu govdeyle birlikte egilir (elde tutulan robot)
            a = self.box_adr
            Rq = np.zeros(9)
            mujoco.mju_quat2Mat(Rq, q)
            rel = self.d.qpos[a:a + 3] - axle
            self.d.qpos[a:a + 3] = axle + Rq.reshape(3, 3) @ rel
            qb = np.zeros(4)
            mujoco.mju_mulQuat(qb, q, self.d.qpos[a + 3:a + 7].copy())
            self.d.qpos[a + 3:a + 7] = qb
            self.d.qvel[6:12] = 0
        self.d.qpos[3:7] = qn
        self.d.qpos[0:3] = axle
        self.d.qvel[0:6] = 0
        mujoco.mj_forward(self.m, self.d)

    def lying(self):
        return abs(self.pitch()) > math.radians(40)  # govde ~45-50 derecede plakayla yere degiyor

    def close(self):
        if self.proc:
            self.proc.stdin.write("quit\n")
            self.proc.stdin.close()
            self.proc.wait()

    def csv(self, name):
        with open(OUT / name, "w", encoding="utf-8") as f:
            f.write("t,x,y,theta_deg,yaw_deg,dutyL,dutyR,mode,dock\n")
            for r in self.trace:
                f.write(",".join(f"{v:.5f}" if isinstance(v, float) else str(v) for v in r) + "\n")


# ====================== senaryolar ======================
RESULTS = []
```

```
def report(tid, name, ok, detail):
    RESULTS.append((tid, name, "GECTI" if ok else "KALDI", detail))
    print(f"[{'GECTI' if ok else 'KALDI'}] {tid:7s} {name:44s} {detail}", flush=True)


class Watch:
    def __init__(self):
        self.max_th, self.fell, self.box = 0.0, False, True

    def __call__(self, tw):
        if not tw.hold:
            self.max_th = max(self.max_th, abs(tw.pitch()))
        self.fell |= tw.lying() or tw.mode == 4
        self.box &= tw.box_ok()


def t_passive():
    worst = 0.0
    ok = True
    for deg in (10, -10):
        tw = Twin(bridge=False)
        tw.hold = False
        tw.tilt(deg)
        t = 0.0
        while t < 5 and not tw.lying():
            mujoco.mj_step(tw.m, tw.d)
            t += PHYS_DT
        ok &= t < 5
        worst = max(worst, t)
    report("ON", "Enerjisiz +-10 deg'den dik kalmamali", ok, f"40 deg'ye en gec {worst:.2f} s'de dustu")


def t_static():
    tw = Twin(task="statik", seed=1)
    tw.arm()
    w = Watch()
    x0, y0 = tw.pos()
    maxd = [0.0]

    def each(t):
        w(t)
        maxd[0] = max(maxd[0], math.hypot(t.pos()[0] - x0, t.pos()[1] - y0))
    tw.run(30, each)
    tw.csv("ikiz_statik.csv")
    tw.close()
    ok = not w.fell and w.box and w.max_th < math.radians(5) and maxd[0] < 0.15
    report("C1.1", "30 s statik denge (kutulu)", ok,
           f"en buyuk egim {math.degrees(w.max_th):.2f} deg, kayma {maxd[0]*100:.1f} cm, kutu {'yerinde' if w.box else 'DUSTU'}")


def t_move(forward=True):
    c = "ileri" if forward else "geri"
    tw = Twin(task="yonlendirme", seed=2 if forward else 3)
    tw.glove = [(0, "dur"), (6.0, c), (6.4, "dur"), (9.5, c), (9.9, "dur")]
    tw.arm()
    w = Watch()
    tw.run(4.9, w)  # 5 s dengede bekleme
    yaw0 = tw.yaw()
    x0, y0 = tw.pos()
    tw.run(8.1, w)  # C++ testiyle ayni: durma penceresi t=14 s'de baslar
    d0 = tw.fwd_dist(x0, y0, yaw0)
    maxdx = [0.0]

    def each(t):
        w(t)
        maxdx[0] = max(maxdx[0], abs(t.fwd_dist(x0, y0, yaw0) - d0))
    tw.run(3.0, each)
    d = abs(tw.fwd_dist(x0, y0, yaw0))
    tw.csv("ikiz_ileri.csv" if forward else "ikiz_geri.csv")
    tw.close()
    ok = not w.fell and w.box and 0.9 < d < 1.1 and maxdx[0] < 0.03
    report("C1.2" if forward else "C1.3", f"1 m {c} + 3 s dur", ok,
           f"yol {d:.3f} m, 3 s durmada kayma {maxdx[0]*1000:.1f} mm, egim tepe {math.degrees(w.max_th):.1f} deg, kutu {'yerinde' if w.box else 'DUSTU'}")


def t_recovery():
    res = []
    for deg in (10, -10):
        tw = Twin(task="statik", seed=4)
        tw.arm()
        tw.tilt(deg)
        t0 = tw.t
        rec = [None]
        after = [0.0]
        w = Watch()

        # toparlanma: |theta| < 3 deg ve |v| < 0,1 m/s'ye ilk varis; sonrasinda egim 5 deg'yi hic asmamali (C++ testiyle ayni olcut)
        def each(t):
            w(t)
            th = abs(t.pitch())
            v = np.linalg.norm(t.d.qvel[0:2])
            if rec[0] is None and th < math.radians(3) and v < 0.1:
```

```
                rec[0] = t.t - t0
            if rec[0] is not None:
                after[0] = max(after[0], th)
        tw.run(10, each)
        if after[0] > math.radians(5):
            rec[0] = None
        tw.close()
        res.append((deg, rec[0], w.fell, w.box))
    ok = all(r[1] is not None and r[1] < 10 and not r[2] and r[3] for r in res)
    report("C1.4", "+10 ve -10 deg'den 10 s icinde toparlanma", ok,
           ", ".join(f"{r[0]:+d}: {r[1]:.2f} s" if r[1] is not None else f"{r[0]:+d}: TOPARLANMADI" for r in res))


def t_turn():
    out = []
    for c, sgn in (("sol", 1), ("sag", -1)):
        tw = Twin(task="yonlendirme", seed=5)
        tw.glove = [(0, "dur"), (1.5, c), (1.9, "dur")]
        tw.arm()
        y0 = tw.yaw()
        w = Watch()
        tw.run(5, w)
        dy = math.degrees((tw.yaw() - y0 + math.pi) % (2 * math.pi) - math.pi)
        tw.close()
        out.append((c, dy, not w.fell and w.box and abs(dy - 90 * sgn) < 15))
    report("C2.3-4", "Eldiven: sola / saga ~90 deg", all(o[2] for o in out),
           ", ".join(f"{o[0]} {o[1]:+.1f} deg" for o in out))


def t_estop():
    tw = Twin(task="statik", seed=6)
    tw.arm()
    tw.motor_power = False
    tw.run(0.05)
    ok = tw.mode == 0 and not tw.en
    tw.close()
    report("GUV.2", "Acil stop: denge birakilir, motorlar kapanir", ok, f"mod={['BEKLE','DENGE','YANASMA','SARJ','DUSTU'][tw.mode]}")


def dock_once(rho, alpha_deg, head_deg, seed, record=False):
    # direk dunya orijininde; yaklasma bolgesi direk normali (+X) etrafinda
    a = math.radians(alpha_deg)
    x, y = rho * math.cos(a), rho * math.sin(a)
    yaw = math.atan2(-y, -x) + math.radians(head_deg)
    tw = Twin(x=x, y=y, yaw=yaw, post=(0.0, 0.0), task="sarj", seed=seed)
    tw.arm()
    t0 = tw.t
    w = Watch()
    st = {"contact": None, "charge": None}

    def each(t):
        w(t)
        if st["contact"] is None and t.charging():
            st["contact"] = t.t
        if st["charge"] is None and t.mode == 3:
            st["charge"] = t.t
    tw.run(45, each)
    if record:
        tw.csv("ikiz_yanasma.csv")
    tw.close()
    ok = st["charge"] is not None and tw.charging() and not w.box is False
    return ok, (st["contact"] - t0) if st["contact"] else None, (st["charge"] - st["contact"]) if st["charge"] and st["contact"] else None


def t_dock():
    cases = [(r, a, h) for r in (0.18, 0.30) for a in (-45, -20, 0, 20, 45) for h in (-25, 0, 25)]
    good, worst, wc = 0, 0.0, 0.0
    fails = []
    for i, (r, a, h) in enumerate(cases):
        ok, tc, tchg = dock_once(r, a, h, 100 + i, record=(i == len(cases) - 2))
        if ok:
            good += 1
            worst = max(worst, tc)
            wc = max(wc, tchg)
        else:
            fails.append(f"({r:.2f} m, {a} deg, bakis {h:+d})")
    report("C3", f"IR ile yanasma, V-yuva + direk ({len(cases)} baslangic)", good == len(cases),
           f"{good}/{len(cases)} basarili, en uzun temas {worst:.1f} s, temas->sarj modu {wc:.2f} s " + " ".join(fails))


def t_undock():
    """Sarjdayken eldivenle 'geri': robot miknatistan kopup dengeye girer ve 0,5 m geri gider."""
    tw = Twin(x=0.25, y=0.0, yaw=math.pi, post=(0.0, 0.0), task="sarj", seed=8)
    tw.arm()
    tw.run(15)
    docked = tw.mode == 3 and tw.charging()
    tw.glove = [(0, "dur"), (tw.t + 0.5, "geri"), (tw.t + 0.9, "dur")]
    w = Watch()
    tw.run(8, w)
    p = tw.d.site("pin").xpos
    gap = math.hypot(p[0], p[1]) - 0.02
    ok = docked and tw.mode == 1 and gap > 0.25 and not w.fell
    tw.close()
    report("C3.5", "Eldivenle 'geri': miknatistan kopup istasyondan ayrilir", ok,
```

```
          f"once {'SARJ' if docked else 'sarjda degil'}, sonra mod {['BEKLE','DENGE','YANASMA','SARJ','DUSTU'][tw.mode]}, direge uzaklik {gap*100:.0f} c
m")


def t_integrated():
    # direk (0,0); robot (0.27,-1.0)'dan +Y'ye bakarak baslar: 2 x ileri, 5 s bekle, sola don, otonom yanasma
    tw = Twin(x=0.27, y=-1.0, yaw=math.pi / 2, post=(0.0, 0.0), task="butunlesik", seed=7)
    tw.glove = [(0, "dur"), (5.0, "ileri"), (5.4, "dur"), (8.6, "ileri"), (9.0, "dur"), (17.0, "sol"), (17.4, "dur")]
    tw.arm()
    w = Watch()
    early = [False]

    def each(t):
        w(t)
        if t.t < 17 and t.mode == 2:
            early[0] = True
    tw.run(60, each)
    tw.csv("ikiz_butunlesik.csv")
    ok = tw.mode == 3 and tw.charging() and not early[0] and w.box
    tw.close()
    report("C5", "Butunlesik gorev (kutu, eldiven, yanasma)", ok,
           f"son mod {['BEKLE','DENGE','YANASMA','SARJ','DUSTU'][tw.mode]}, erken yanasma {'VAR' if early[0] else 'yok'}, kutu {'yerinde' if w.box else 'D
USTU'}")


def t_reference_gains():
    """GitHub: suelioalencar/Two-Wheels-Inverted-Pendulum... gercek robotta calisan LQI kazanclari
    (k1 = [-100.5, -69.9, -83.6, -9.31] V, 65 mm teker, enkoderli DC motor). Bu kazanclari ikizde, kendi
    durum olcumleriyle (dogrudan fizikten) kosup modelin gercekci bir bolgede oldugunu kontrol eder."""
    K = np.array([-100.493, -69.900, -83.636, -9.314])
    res = []
    for deg in (0, 8):
        tw = Twin(task="statik", seed=9, bridge=False)
        tw.hold = False
        tw.en = True
        if deg:
            tw.tilt(deg)
        x0 = tw.d.qpos[0]
        ok = True
        prev = tw.pitch()
        for i in range(int(8 / PHYS_DT)):
            if i % CTRL_EVERY == 0:
                th = tw.pitch()
                om = tw.d.sensor("gyro").data[1]
                wl = tw.d.qpos[tw.jl.qposadr[0]]
                wr = tw.d.qpos[tw.jr.qposadr[0]]
                x = R_WHEEL * ((wl + wr) / 2 + th) - 0  # teker yolu
                v = R_WHEEL * ((tw.d.qvel[tw.jl.dofadr[0]] + tw.d.qvel[tw.jr.dofadr[0]]) / 2 + om)
                u = -(K @ np.array([x, v, th, om]))
                u = max(-11.0, min(11.0, u))
                tw.duty = (u / tw.vbat, u / tw.vbat)
            tw.physics()
            if tw.lying():
                ok = False
                break
        res.append((deg, ok, math.degrees(tw.pitch())))
    report("REF", "GitHub referans LQI kazanclari ikizde dengeliyor", all(r[1] for r in res),
           ", ".join(f"baslangic {r[0]} deg: {'dengede' if r[1] else 'DUSTU'} (son {r[2]:+.1f} deg)" for r in res))


if __name__ == "__main__":
    which = sys.argv[1:] or ["passive", "static", "move", "recovery", "turn", "estop", "ref", "dock", "undock", "integrated"]
    if "passive" in which: t_passive()
    if "static" in which: t_static()
    if "move" in which: t_move(True); t_move(False)
    if "recovery" in which: t_recovery()
    if "turn" in which: t_turn()
    if "estop" in which: t_estop()
    if "ref" in which: t_reference_gains()
    if "dock" in which: t_dock()
    if "undock" in which: t_undock()
    if "integrated" in which: t_integrated()
    with open(OUT / "summary.tsv", "w", encoding="utf-8") as f:
        for r in RESULTS:
            f.write("\t".join(r) + "\n")
    n = sum(r[2] == "GECTI" for r in RESULTS)
    print(f"\nIKIZ OZET: {n}/{len(RESULTS)} gecti")
```

twin/bridge.cpp 52 satır

```
// Dijital ikiz koprusu: robot firmware'indeki RobotLogic'i stdin/stdout uzerinden MuJoCo ikizine baglar.
// Her satir bir 5 ms adimi:
//   girdi : t ax ay az gx gy gz encL encR vbat vchg irL irR motorPower gloveNew gloveCmd task
//   cikti : en dutyL dutyR mode dock thetaDeg
#include <cstdio>
#include <cstring>

#include "robot_logic.h"

int main() {
  cdtp::RobotLogic logic;
```

```
  char line[512];
  uint16_t seq = 0;
  int lastTask = -1;
  while (std::fgets(line, sizeof line, stdin)) {
    if (!std::strncmp(line, "quit", 4)) break;
    cdtp::RobotInputs in;
    int en_mp, gnew, gcmd, task;
    long eL, eR, iL, iR;
    int n = std::sscanf(line, "%f %f %f %f %f %f %f %ld %ld %f %f %ld %ld %d %d %d %d", &in.t, &in.imu.acc[0],
                        &in.imu.acc[1], &in.imu.acc[2], &in.imu.gyro[0], &in.imu.gyro[1], &in.imu.gyro[2], &eL, &eR,
                        &in.vbat, &in.vcharge, &iL, &iR, &en_mp, &gnew, &gcmd, &task);
    if (n != 17) {
      std::printf("ERR %d\n", n);
      std::fflush(stdout);
      continue;
    }
    if (task != lastTask) {
      logic.setTask((cdtp::Task)task);
      lastTask = task;
    }
    in.encL = eL;
    in.encR = eR;
    in.irL = iL;
    in.irR = iR;
    in.motorPower = en_mp != 0;
    in.gloveNew = gnew != 0;
    if (in.gloveNew) {
      cdtp::GlovePacket p{};
      p.cmd = (uint8_t)gcmd;
      p.seq = seq++;
      cdtp::sealPacket(p);
      in.glove = p;
    }
    cdtp::RobotOutputs o = logic.step(in);
    std::printf("%d %.5f %.5f %d %d %.3f\n", o.motorEnable ? 1 : 0, o.dutyL, o.dutyR, (int)logic.mode(),
                (int)logic.dockState(), logic.thetaDeg());
    std::fflush(stdout);
  }
  return 0;
}
```

tools/plot_twin.py 74 satır

```
"""C++ simulasyonu ile MuJoCo dijital ikizini karsilastiran grafikler -> docs/img/ikiz_*.png"""
import csv
import math
import pathlib

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt

ROOT = pathlib.Path(__file__).resolve().parents[1]
OUT = ROOT / "docs/img"


def load(p):
    with open(p, encoding="utf-8") as f:
        rows = list(csv.DictReader(f))
    return {k: [float(r[k]) for r in rows] for k in rows[0]}


def style(ax):
    ax.grid(True, alpha=0.3)
    ax.spines[["top", "right"]].set_visible(False)


def compare():
    c = load(ROOT / "sim/results/c1_ileri.csv")
    t = load(ROOT / "twin/results/ikiz_ileri.csv")
    tx0, ty0 = t["x"][0], t["y"][0]
    td = [math.hypot(x - tx0, y - ty0) for x, y in zip(t["x"], t["y"])]
    fig, ax = plt.subplots(2, 1, figsize=(8, 5.4), sharex=True)
    ax[0].plot(c["t"], c["x"], color="#2a6fdb", label="C++ simülasyonu")
    ax[0].plot(t["t"], td, color="#d9822b", label="MuJoCo dijital ikiz", lw=1.2)
    ax[0].set_ylabel("yol (m)")
    ax[0].legend(frameon=False, loc="upper left")
    ax[1].plot(c["t"], c["theta_deg"], color="#2a6fdb")
    ax[1].plot(t["t"], t["theta_deg"], color="#d9822b", lw=1.2)
    ax[1].set_ylabel("eğim (°)")
    ax[1].set_xlabel("zaman (s)")
    for a in ax:
        style(a)
    fig.suptitle("Aynı firmware mantığı, iki bağımsız model: 5 s bekle, 2 × ileri, 3 s dur")
    fig.tight_layout()
    fig.savefig(OUT / "ikiz_karsilastirma.png", dpi=110)
    plt.close(fig)


def dock():
```

```
    p = ROOT / "twin/results/ikiz_yanasma.csv"
    if not p.exists():
        return
    d = load(p)
    fig, ax = plt.subplots(figsize=(5.5, 4.6))
    ax.plot(d["x"], d["y"], color="#d9822b", label="robot merkezi (ikiz)")
    ax.plot(d["x"][0], d["y"][0], "o", color="#d9822b")
    ax.add_patch(plt.Circle((0, 0), 0.02, color="#333", label="direk Ø40"))
    th = [math.radians(a) for a in range(-45, 46)]
    ax.plot([0.3 * math.cos(a) for a in th], [0.3 * math.sin(a) for a in th], "--", color="#999", lw=0.8, label="30 cm / 90° bölge")
    ax.set_aspect("equal")
    ax.set_xlabel("X (m)")
    ax.set_ylabel("Y (m)")
    ax.legend(frameon=False, fontsize=8)
    style(ax)
    ax.set_title("Dijital ikiz: IR ile yanaşma")
    fig.tight_layout()
    fig.savefig(OUT / "ikiz_yanasma.png", dpi=110)
    plt.close(fig)


if __name__ == "__main__":
    compare()
    dock()
    print("cizildi")
```